



System Specifications

3

Communication

3

Application Layer

7

Summary

This document specifies the Application Layer of the KNX System.
Version 02.01.01 is a KNX Approved Standard.

Document updates

Version	Date	Modifications
1.0	2001.12.19	Preparation of the Approved Standard.
1.0	2005.08.22	AN045 → 3.3.2 “A_DeviceDescriptor_InfoReport-service”
1.0	2007.01.08	S15 “Easy Common Parts” – A_Link_Read and A_Link_Write integrated.
1.0	2007.01.12	• AN038 “Function Services” integrated.
1.0	2007.01.24	• AN028 “A_NetworkParameter_Write” integrated.
1.0	2007.08.21	• AN022 “A_PropertyDescription_Read-service” integrated.
1.0	2007.08.21	• AN026 “DoA on RF” integrated. • Introducing of grouping of AL-services in “System Broadcast”
1.0	2007.09.19	• AN026 “Connectionless A_Restart” integrated.
1.0	2007.09.20	• AN065 “Number field in memory services” integrated.
1.0	2008.01.17	• AN044 “RF specification complements” integrated
1.0	2008.05.05	• AN044 “APCI_ServiceInformation_Indication_Write” integrated: service A_ServiceInformation_Indication_Write-Service removed.
1.0	2008.05.07	• AN072 “A_Authorize correction and configuration” integrated.
1.0	2008.05.14	• AN046 “Device Descriptor error handling” integrated.
1.0	2008.05.23	• Update of A_Link_Write handling according [KSG00105-14]
1.0	2008.08.12	• AN101 “File Transfer Protocol” integrated.
1.1	2008.12.24	Finalisation of the Approved Standard v1.1.
	2009.02.24	Indication about variable size of A_Link_Response-PDU ([Ref20090224ab])
1.2.00	2009.11.05	• AN116 “connectionless use of manufacturer specific user messages” integrated.
1.2.01	2010.08.02	• AN124 “Interface Object Index Discovery” integrated.
1.2.02	2010.09.27	Corrected reference to definition of TMedia from [06] to [05] in 3.2.6. Checked all instances.
1.3.00	2010.10.22	• AN127 “Master Reset” integrated.
01.05.00	2013.07.22	• AN132 “A_NetworkParameter_InfoReport” integrated.
01.05.02	2013.09.27	• Editorial update to include APCI_FileStream_InfoReport in Table 1.
01.06.00	2013.10.23	• AN162 “System aspects of RF S-Mode” integrated.
01.06.01	2013.10.28	Editorial updates for the publication of KNX Specifications 2.1.
01.06.02	2013.12.10	Vertical alignment of cover page corrected. Final editorial review in view of publication of the KNX Specifications v2.1.
01.06.03	2014.11.10	Editorial corrections. – Correction of font colour in 3.4.2.2. – Correction of the APCI-values of A_Memory_-services in Table 1.
01.06.04	2015.04.24	Editorial corrections. – Correction of allowed communication modes of A_NetworkParameter_Read in Table 1.
01.06.05	2015.07.07	Editorial corrections. – Correction of the error handling of A_SystemNetwork-Parameter_Read in 3.3.8
02.01.01	2019.12.17	• AN163 “Extended Interface Object Addressing” integrated.

Version	Date	Modifications
	2020.01.03	• AN158 “KNX Data security” integrated.
	2020.01.07	• AN177 “Extended Memory services” integrated.
	2020.01.08	• AN189 “New routing algorithm for hc 7” integrated.
	2020.10.14	• AN178 “Specification of services using EFF” integration started. - Names of Coupler services; specification of nte Coupler services - Support of max APDU length for management (2.2)
	2020.10.26	• AN183 “KNXnet/IP System Broadcast” integration started
	2021.05.18	• AN193 “Access Policies” integration started
	2021.08.19	• AN160 “KNX RF S-Mode device Profiles” integration started and completed.
	2021.09.28	• Editorial corrections. • Preparation for publication in the KNX Specifications v3.0.

References

- [01] Chapter 3/3/3 “Network Layer”
- [02] Chapter 3/3/4 “Transport Layer”
- [03] Chapter 3/4/1 “Application Interface Layer”
- [04] Chapter 3/5/1 “Resources”
- [05] Chapter 3/5/2 “Management Procedures”
- [06] Volume 6 “Profiles”
- [07] RFC 3610 September 2003 “Counter with CBC-MAC (CCM)”
- [08] NIST SP 800-38C May 2004 “Recommendation for Block Cipher modes of Operation: The CCM Mode for Authentication and Confidentiality”
- [09] RFC 3610 September 2003 “Counter with CBC-MAC (CCM)”

Filename: 03_03_07 Application Layer v02.01.01 AS.docx
Version: 02.01.01
Status: Approved Standard
Savedate: 2021.09.28
Number of pages: 191

Contents

1	Introduction	6
1.1	Application Layer services and Transport Layer communication modes	6
1.2	Service Primitives of the Application Layer	6
2	APDU	8
2.1	General	8
2.2	APDU length	8
3	Application Layer services	12
3.1	Application Layer services on Multicast Communication Mode	12
3.1.1	The relation between TSAPs and ASAPs	12
3.1.2	A_GroupValue_Read-service	13
3.1.3	A_GroupValue_Write-service	16
3.2	Application Layer services on Broadcast Communication Mode	18
3.2.1	Goal	18
3.2.2	A_IndividualAddress_Write-service	18
3.2.3	A_IndividualAddress_Read-service	19
3.2.4	A_IndividualAddressSerialNumber_Read-service	21
3.2.5	A_IndividualAddressSerialNumber_Write-service	23
3.2.6	A_NetworkParameter_Read-service	25
3.2.7	A_NetworkParameter_Write-service	29
3.2.8	A_NetworkParameter_InfoReport	30
3.3	Application Layer Services on System Broadcast communication mode	32
3.3.1	Introduction	32
3.3.2	A_DeviceDescriptor_InfoReport-service	33
3.3.3	A_DomainAddress_Write-service	34
3.3.4	A_DomainAddress_Read-service	36
3.3.5	A_DomainAddressSelective_Read-service	38
3.3.6	A_DomainAddressSerialNumber_Read-service	39
3.3.7	A_DomainAddressSerialNumber_Write-Service	41
3.3.8	A_SystemNetworkParameter_Read	43
3.3.9	A_SystemNetworkParameter_Write	46
3.4	Application Layer Services on Point-to-point Connectionless Communication Mode	48
3.4.1	Introduction	48
3.4.2	Common services	48
3.4.3	Property Description services	55
3.4.4	Data Property services	63
3.4.5	Data Property Extended services	67
3.4.6	Link services	80
3.4.7	Function Property services	83
3.4.8	Function Property Extended services	88
3.4.9	Memory services	97
3.5	Application Layer Services on Point-to-point Connection-Oriented Communication Mode	106
3.5.1	Introduction	106
3.5.2	A_ADC_Read-service	107
3.5.3	A_Memory_Read-service	109
3.5.4	A_Memory_Write-service	111
3.5.5	A_MemoryBit_Write-service	114
3.5.6	A_UserData	117

3.5.7	A_Authorize_Request-service	127
3.5.8	A_Key_Write-service	129
3.6	Coupler specific Application Layer Services on Point-to-Point connection oriented Communication Mode	131
3.6.1	A_FilterTable_Open-service.....	131
3.6.2	A_FilterTable_Read-service.....	132
3.6.3	A_FilterTable_Write-service	134
3.6.4	A_RouterMemory_Read-service	137
3.6.5	A_RouterMemory_Write-service	139
4	Parameters of Application Layer	142
4.1	Group Object Association Table	142
4.2	Verify flag.....	142
5	Secure Application Layer	143
5.1	General requirements and overview	143
5.1.1	Embedding of the S-AL within the Application Layer and basic functionality	143
5.1.2	S-AL – overview.....	144
5.1.3	AES-128 with CTR operation mode and AES-CBC-MAC signature (CCM).....	145
5.1.4	Other security algorithms.....	150
5.2	Support the S-A_Data-service	150
5.2.1	S-A_Data-service.....	150
5.3	Handling of the Sequence Number.....	154
5.3.1	Runtime handling.....	154
5.3.2	S-A_Sync-service	157
5.4	Registration of Security Failures	164
5.5	Secure Handling of the Transport Layer services.....	165
5.5.1	Common requirements.....	165
5.5.2	Common error handling.....	166
5.5.3	Secure AL-service on Multicast Communication Mode - T_Data_Group.....	167
5.5.4	Secure AL-service on Multicast Communication Mode - T_Data_Tag_Group	169
5.5.5	Secure AL-Service on point-to-point communication mode	171
5.5.6	Secure AL-Service on system broadcast communication mode.....	175
5.5.7	Secure AL-services on broadcast communication mode	176
5.5.8	Overview of allowed combinations of keys and communication modes	178
5.6	Access Policies at service level	178
A.1	Goal.....	181
A.2	Definitions	181
A.3	CCM operation	181
B.1	Runtime - symmetric security algorithm	185
B.1.1	Algorithm.....	185
B.1.2	Operation Mode	185
B.2	Sequence Number field – calculation of size	185
B.3	Timing calculation of KNX secure communication.....	186
C.1	Full encoding of a KNX Secure APDU.....	187
C.1.1	S-A_Data-PDU	187
C.1.2	S-A_Data-PDU	188
C.1.3	S-A_Sync.req.....	189
C.1.4	S-A_Sync.res	190

1 Introduction

1.1 Application Layer services and Transport Layer communication modes

The Application Layer provides a large variety of application services to the application process. Application processes in different devices interoperate by using services of Application Layer over communication modes. According to Transport Layer, different types of communication modes exist:

- point-to-multi-point, connection-less (multicast)
- point-to-all-points, connection-less (broadcast)
- point-to-point connection-less
- point-to-point connection-oriented.

Depending on the communication mode, different Application Layer services are offered.

Some services can be used on the point-to-point connection-oriented, as well as the point-to-point connection-less communication mode, although Application Layer services are always mapped to Transport Layer services depending on the type of the communication mode.

1.2 Service Primitives of the Application Layer

Each specified Application Layer service shall be invoked by the Transport Layer primitives Request(req), Indication(ind) and confirm(con). For a remote confirmed Service the remote device shall use the same Transport Layer primitives to respond to the service.

The Transport Layer confirm primitive shall only be a confirmation from the Transport Layer instance and shall include all data from the request plus the state that shall indicate whether the service is sent successfully or not. The Application Layer shall map the Transport Layer confirm to a Local confirm (Lcon).

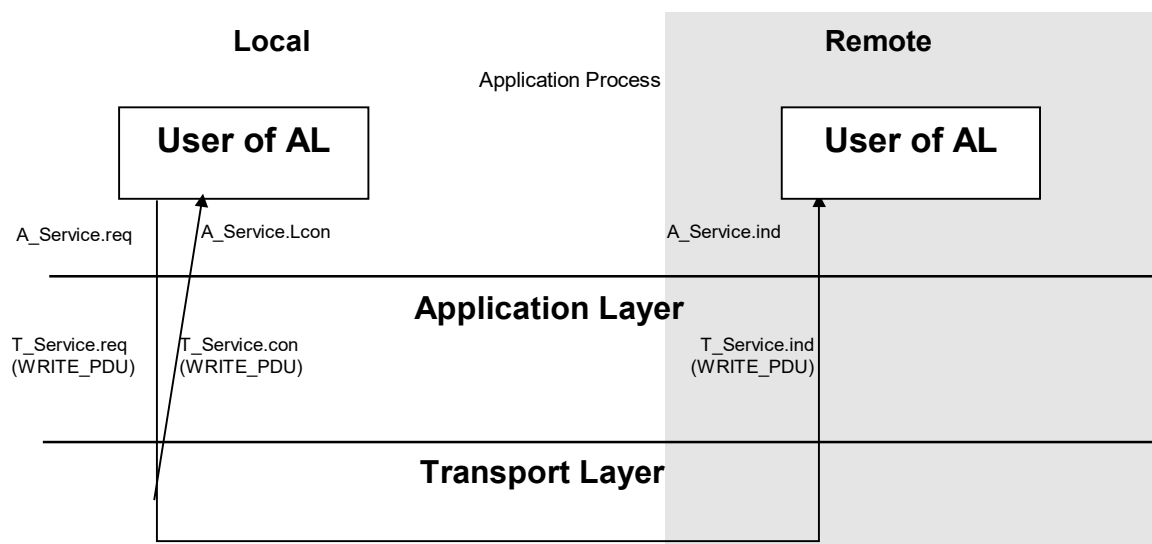
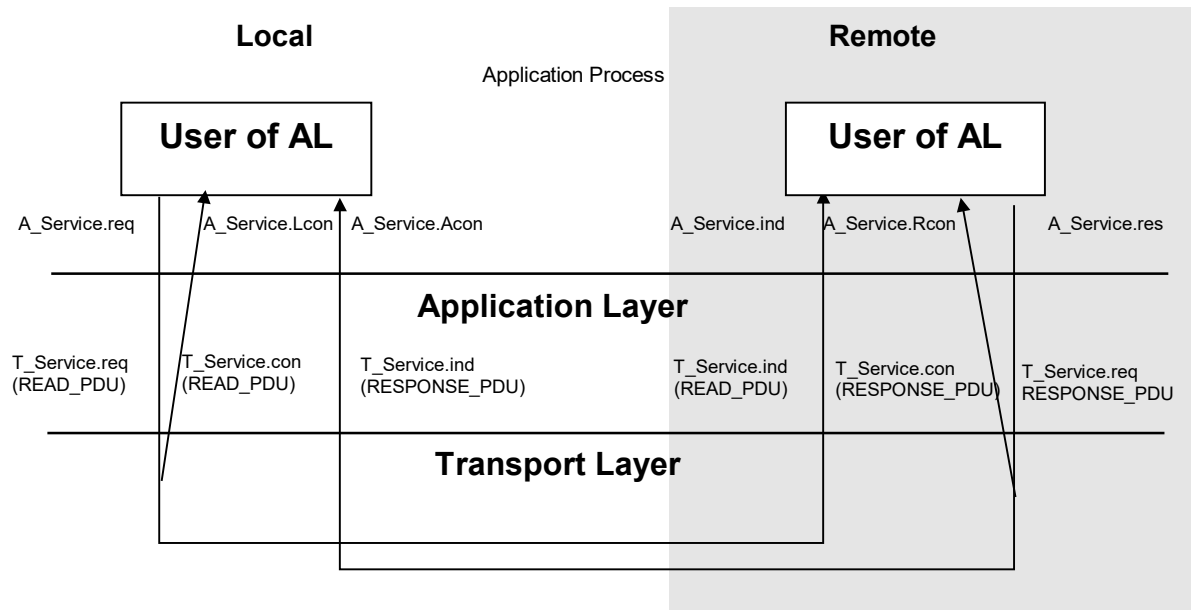


Figure 1 - Interactivity of the Application Layer for services that are not remote confirmed

In case of a remote confirmed service the remote device shall initiate the Response(res) primitive and the Application layer shall map this service to a Transport Layer request primitive. The local Application Layer shall receive the TL indication primitive and shall map it to an Application Layer Confirm(Acon). The Transport Layer confirmation in the remote device shall be mapped by the Application Layer to a Response Confirm (Rcon). In the following service descriptions the Local Confirm(Lcon) and the Response Confirm (Rcon) are not described.



**Figure 2 - Interactivity of the Application Layer
for services that are remote confirmed**

2 APDU

2.1 General

The APDU is shown in Figure 3 below.

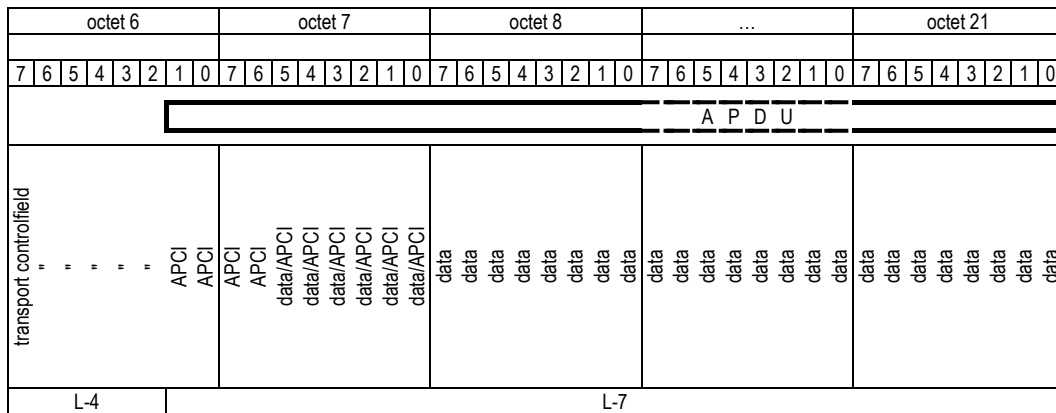


Figure 3 - Format of the APDU (example)

The APDU shall correspond to the TPDU, but shall be reduced by the Transport Control field. The application control field shall be encoded and decoded by Application Layer and shall contain the Application Layer service codes. The Application Control field shall have a length of 4 - or 10 bits, depending on the Application Layer service. The codes for the application control field are shown in Table 1.

2.2 APDU length

Devices supporting EFF (Extended Frame Format) for management shall support all services with service maximum available length up to the maximum supported APDU-length.

The complete PDU for each service primitive is shown in the description of every service. Not defined and not supported Application Layer services shall be ignored by the Application Layer.

The symbols in Table 1 shall be interpreted as follows. Whether or not an Application Layer service is mandatory, is specified in [06] either explicitly or through the requirements of other features than Application Layer services. If for a certain Profile it is required that an Application Layer service be implemented, then additionally Table 1 applies, with the following definition.

- If the service primitive is marked with "M" then this service primitive shall be implemented for this communication mode.
- If the service primitive is marked with "O" then this service primitive may be implemented for this communication mode.
- If the service primitive is marked with "X" then this service primitive shall not be implemented for this communication mode.

Communication modes

1) This service shall not be used for future Profile definitions.

Table 1 – Application Layer control field[illegible]

2) This APCI-value 3DFh has been used in the past for the service A_ServiceInformation_Indication. This APCI shall not be used for new implementations.

Table 1 – Application Layer control field

Communication
modes

7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
						APCI									
						APCI									
						APCI/dat									
						APCI/dat									
						APCI/dat									
						APCI/dat									
						APCI/dat									
						APCI/dat									
						APCI/dat									
						APCI/dat									

A_DomainAddressSelective_Read-PDU

						1	1	1	1	1	0	0	1	0	0
--	--	--	--	--	--	---	---	---	---	---	---	---	---	---	---

A_NetworkParameter_Write-PDU

						1	1	1	1	1	0	0	1	0	1
						1	1	1	1	1	0	0	1	1	0
						1	1	1	1	1	0	0	1	1	1

A_Link_Read-PDU

A_Link_Response-PDU

A_Link_Write-PDU

						1	1	1	1	1	0	1	0	0	0
						1	1	1	1	1	0	1	0	0	1
						1	1	1	1	1	0	1	0	1	0
						1	1	1	1	1	0	1	0	1	1

A_GroupPropValue_Read-PDU

A_GroupPropValue_Response-PDU

A_GroupPropValue_Write-PDU

A_GroupPropValue_InfoReport-PDU

						1	1	1	1	1	0	1	1	0	0
						1	1	1	1	1	0	1	1	0	1
						1	1	1	1	1	0	1	1	1	0

A_DomainAddressSerialNumber_Read-PDU

A_DomainAddressSerialNumber_Response-PDU

A_DomainAddressSerialNumber_Write-PDU

						1	1	1	1	1	1	0	0	0	0
--	--	--	--	--	--	---	---	---	---	---	---	---	---	---	---

A_FileStream_InforReport-PDU

T_Data_Group	T_Data_Tag_Group	T_Data_Broadcast	T_Data_SystemBroadcast	T_Data_Individual	T_Data_Connected
x	x	M	M	x	x

x	x	M	x	M	M
---	---	---	---	---	---

x	x	x	x	M	M
x	x	x	x	M	M
x	x	x	x	M	M

x	M	x	x	x	x
x	M	x	x	x	x
x	M	x	x	x	x
x	M	x	x	x	x

x	x	M	M	x	x
x	x	M	M	x	x
x	x	M	M	x	x

x	x	x	x	M	M
---	---	---	---	---	---

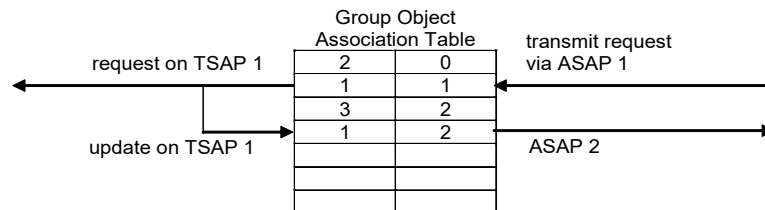
3 Application Layer services

3.1 Application Layer services on Multicast Communication Mode

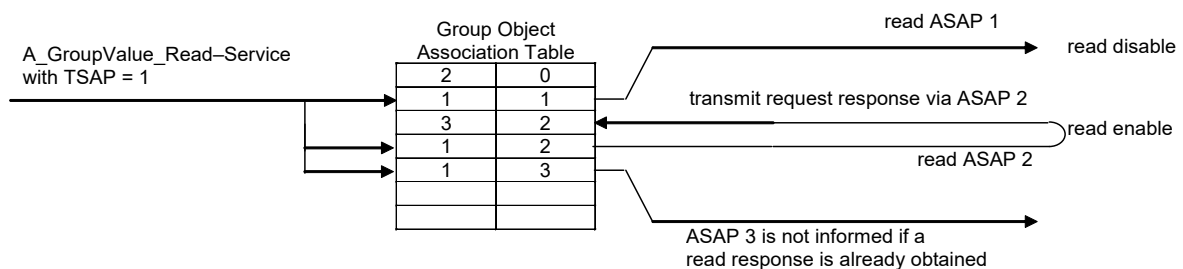
3.1.1 The relation between TSAPs and ASAPs

A multicast communication mode shall connect Transport Layer Service Access Points (TSAP) to Application Layer Service Access Points (ASAP). If one device sends an A_GroupValue_Service each device that is member of this group shall receive the A_GroupValue_Service.

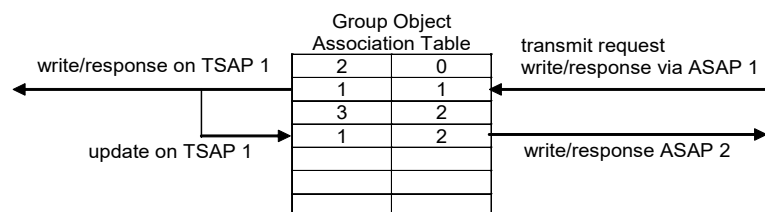
If the Application Layer of a device receives an A_GroupValue_Write-Service, it shall search the TSAP in all entries of the Group Object Association Table and informs all the associated ASAP.



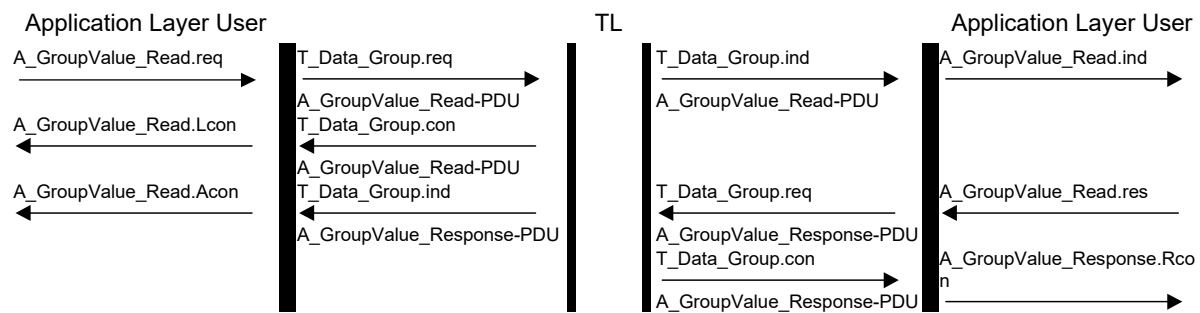
If the Application Layer of a device receives an A_GroupValue_Read-service, it shall search the TSAP in all entries of the Group Object Association Table and shall inform all the associated ASAPs. Only one read response shall be generated by the user.



If a transmission is requested (read response or write) via an ASAP, the Application Layer shall take the TSAP from the Group Object Association Table, shall update all the ASAPs with the same TSAP and shall generate an A_GroupValue-Service-Request.



3.1.2 A_GroupValue_Read-service



The `A_GroupValue_Read.req` primitive shall be applied by the user of Application Layer, to receive an update of the value of its ASAP by making a communication partner respond with an `A_GroupValue_Read.res`, i.e. the service shall be confirmed by the remote application process. The ASAP shall be associated to the TSAP via the Group Object Association Table, i.e. with a Group Address (see Transport Layer). All other group members shall receive the `A_GroupValue_Response-PDU` as well.

The local Application Layer shall accept the service request and map the ASAP to the TSAP.

- If none of the security service parameters *par_auth* or *par_conf* is set, then the local Application Layer shall forward the APDU and pass it with a `T_Data_Group.req` to the local Transport Layer. The user shall decide during configuration about this mapping. The parameters TSAP and priority shall be mapped to the corresponding parameters of the `T_Data_Group.req` primitive; the TSDU shall be an `A_GroupValue_Read-PDU`.
- If one or more of the security service parameters is set, then the local Application Layer shall forward the APDU with an `S-A_Data.req` to the Secure Application Layer. Please refer to the specifications of the S-AL for the handling of Secure AL-services (see 5.5.3.3) for the further handling by the S-AL.

The remote Application Layer shall map a `T_Data_Group.ind` primitive with TSDU = `A_GroupValue_Read-PDU` to an `A_GroupValue_Read.ind` primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the `A_GroupValue_Read.ind` primitive. One `A_GroupValue_Read.ind` primitive shall be generated per ASAP that is assigned to the corresponding TSAP (i.e. Group Address).

The application process may respond to the `A_GroupValue_Read.ind` primitive with an `A_GroupValue_Read.res` primitive containing the value of the ASAP. The user can decide during configuration, whether or not the `A_GroupValue_Read.res` primitive shall be generated, although it makes sense that exactly one ASAP generates the `A_GroupValue_Read.res` primitive.

NOTE 1 It is left to the user application programmer to decide whether an `A_GroupValue_Read.Acon` time-out supervision is necessary.

Two different formats of the `A_GroupValue_Response-PDU` are used depending on the length of the value. The maximum length of the value is 14 octets. Unused data bits shall be set to zero.

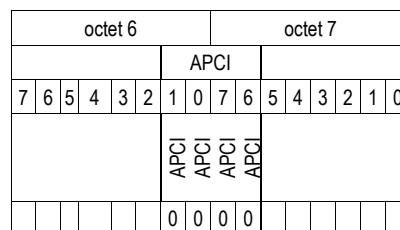
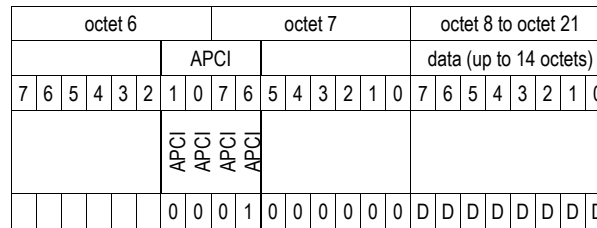
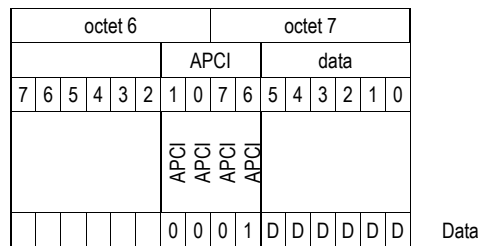


Figure 4 - `A_GroupValue_Read-PDU` (example)



**Figure 5 - A_GroupValue_Response-PDU,
length of ASAP data is more than 6 bit (example)**

Values that only consist of 6 bits or less shall have the following optimized A_GroupValue_Response-PDU format.



**Figure 6 - A_GroupValue_Response-PDU,
length of ASAP data is 6 bit or less (example)**

The remote Application Layer shall accept the service response, map the ASAP to the TSAP and pass it with a T_Data_Group.req to the local Transport Layer. The parameters ack_request, TSAP, hop_count_type and priority shall be mapped to the corresponding parameters of the T_Data_Group.req primitive; the TSDU shall be an A_GroupValue_Response-PDU.

The local Application Layer shall map a T_Data_Group.ind primitive with TSDU = A_GroupValue_Response-PDU to an A_GroupValue_Read.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_GroupValue_Read.Acon primitive. More than one A_GroupValue_Read.Acon primitive may occur depending on the number of group members that have been configured to respond.

A_GroupValue_Read.req(ack_request, ASAP, priority, hop_count_type)

- ack_request: Data Link Layer Acknowledge requested or don't care
- ASAP: local reference of the Service Access Point
- hop_count_type: standard NL hop count value
- par_auth: This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
- par_conf: This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
- priority: system, urgent, normal or low priority

A_GroupValue_Read.Lcon(ack_request ,ASAP, priority, hop_count_type, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
ASAP:	local reference of the Service Access Point
hop_count_type:	standard NL hop count value
par_auth:	This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
par_conf:	This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
priority:	system, urgent, normal or low priority
a_status:	ok: A_GroupValue_Read.req sent successfully with T_Data_Group service not_ok: transmission of the associated T_Data_Group request frame did not succeed

A_GroupValue_Read.ind(ASAP, priority, hop_count_type)

ASAP:	local reference of the Service Access Point
hop_count_type:	standard NL hop count value
par_auth:	This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
par_conf:	This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
priority:	system, urgent, normal or low priority

A_GroupValue_Read.res(ack_request, ASAP, priority, hop_count_type, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
ASAP:	local reference of the Service Access Point
data:	the value of the associated Service Access Point
hop_count_type:	standard NL hop count value
par_auth:	This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
par_conf:	This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
priority:	system, urgent, normal or low priority

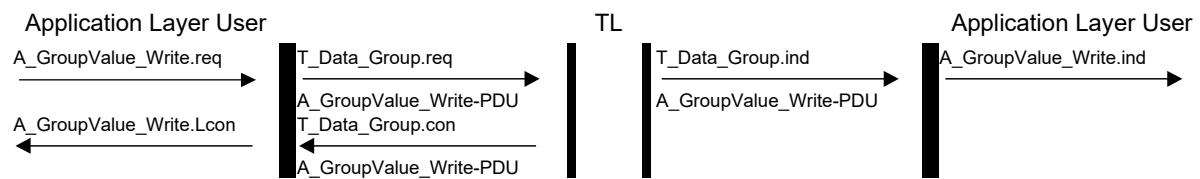
A_GroupValue_Read.Rcon(ack_request, ASAP, priority, hop_count_type, data, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
ASAP:	local reference of the Service Access Point
hop_count_type:	standard NL hop count value
par_auth:	This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
par_conf:	This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
priority:	system, urgent, normal or low priority
data:	the value of the associated Service Access Point
a_status:	ok: A_GroupValue_Read.res sent successfully with T_Data_Group service not_ok: transmission of the associated T_Data_Group request frame did not succeed

A_GroupValue_Read.Acon(ASAP, priority, hop_count_type, data)

ASAP:	local reference of the Service Access Point
hop_count_type:	standard NL hop count value
priority:	system, urgent, normal or low priority
par_auth:	This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
par_conf:	This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
data:	the value of the associated Service Access Point

3.1.3 A_GroupValue_Write-service



The A_GroupValue_Write.req primitive shall be applied by the user of Application Layer, to send an update of its ASAP to all connected ASAPs. The service shall not be confirmed by the remote application process, the confirmation shall be caused by the local T_Data_Group.con. The ASAP shall be associated to the TSAP via the Group Object Association Table, i.e. with a Group Address (see [02]). All group members shall receive the A_GroupValue_Write-PDU.

The local Application Layer shall accept the service request and map the ASAP to the TSAP (IA_Index).

- If none of the security service parameters *par_auth* or *par_conf* is set, then the local Application Layer shall forward the APDU with a T_Data_Group.req to the local Transport Layer. The user decides during configuration about this mapping. The parameters TSAP and priority shall be mapped to the corresponding parameters of the T_Data_Group.req primitive, the TSDU shall be an A_GroupValue_Write-PDU.
- If one or more of the security service parameters is set, then the local Application Layer shall forward the APDU with an S-A_Data.req to the Secure Application Layer. Please refer to the specifications of the S-AL for the handling of Secure AL-services (see 5.5.3.3) for the further handling by the S-AL.

The remote Application Layer shall map a T_Data_Group.ind primitive with TSDU = A_GroupValue_Write-PDU to an A_GroupValue_Write.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_GroupValue_Write.ind primitive. One A_GroupValue_Write.ind primitive shall be generated per ASAP that is assigned to the corresponding TSAP (i.e. Group Address).

Two different formats of the A_GroupValue_Write-PDU shall be used depending on the length of the value. The maximum length of the value shall be 14 octets. Unused data bits shall be set to zero.

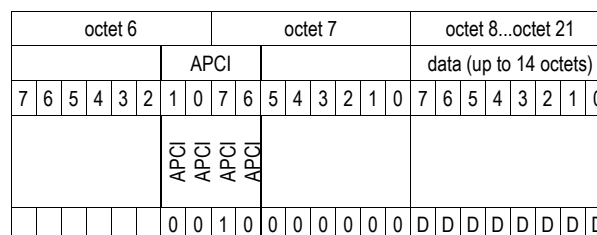
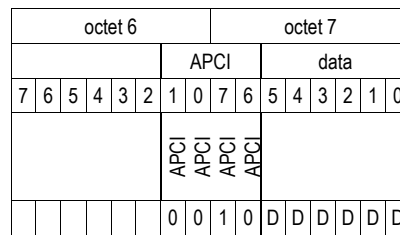


Figure 7 - A_GroupValue_Write-PDU, length of ASAP data is more than 6 bit (example)

Values that only consist of 6 bits or less have the following optimized A_GroupValue_Write-PDU format:



**Figure 8 - A_GroupValue_Write-PDU,
length of ASAP data is 6 bit or less (example)**

Prior to passing an A_GroupValue_Write.Lcon primitive to the local application process, the local Application Layer shall need a T_Data_Group.con from the local Transport Layer. If the confirmation is positive (t_status = ok), the local Application Layer shall pass a positive A_GroupValue_Write.Lcon (a_status = ok) to the local application process. If the confirmation is negative (t_status = not_ok), the local Application Layer shall pass an A_GroupValue_Write.Lcon (a_status = not_ok) to the local user indicating that the transmission of the associated T_Data_Group.req did not succeed.

A_GroupValue_Write.req(ack_request, ASAP, priority, hop_count_type, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
ASAP:	local reference of the Service Access Point
par_auth:	This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
par_conf:	This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
data:	data of the associated application layer Service Access Point

A_GroupValue_Write.Lcon(ack_request, ASAP, priority, hop_count_type, data, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
ASAP:	local reference of the Service Access Point
par_auth:	This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
par_conf:	This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
data:	data of the associated application layer Service Access Point
a_status:	ok: A_GroupValue_Write sent successfully with T_Data_Group service
	not_ok: transmission of the associated T_Data_Group request frame did not succeed

A_GroupValue_Write.ind(ASAP, priority, hop_count_type, data)

ASAP:	local reference of the Service Access Point
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
par_auth:	This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
par_conf:	This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.

3.2 Application Layer services on Broadcast Communication Mode

3.2.1 Goal

A broadcast communication mode shall be connectionless and shall connect one device with all others.

3.2.2 A_IndividualAddress_Write-service

The A_IndividualAddress_Write.req primitive shall be applied by the user of Application Layer to modify the Individual Address in a communication partner. The communication partner shall not be identified in the service, i.e. the destination must be defined by selecting a destination manually. This can be done by pressing a button on exactly one device that brings this device into a 'programming mode', i.e. only the device where the button is pressed shall accept the A_IndividualAddress_Write.ind, others shall ignore it. The way that a product is set to 'programming mode' may be manufacturer specific.

The local Application Layer shall accept the service request and shall pass it with a T_Data_Broadcast.req to the local Transport Layer. The parameter priority, implicitly with value 'system', shall be mapped to the corresponding parameter of the T_Data_Broadcast.req primitive, the TSDU shall be an A_IndividualAddress_Write-PDU.

octet 6						octet 7						octet 8						octet 9					
						APCI						newaddress (high)						newaddress (low)					
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
						APCI	APCI	APCI	APCI														
						0	0	1	1	0	0	0	0	0	0								

Figure 9 - A_IndividualAddress_Write-PDU (example)

The remote Application Layer shall map a T_Data_Broadcast.ind primitive with TSDU = A_IndividualAddress_Write-PDU to an A_IndividualAddress_Write.ind primitive. The argument priority, implicitly with value 'system', shall be mapped to the corresponding argument priority of the A_IndividualAddress_Write.ind primitive.

The application process shall ignore the A_IndividualAddress_Write.ind primitive if the device is not in 'programming mode'. Otherwise the local Individual Address shall be set to the new address.

Prior to passing an A_IndividualAddress_Write.Lcon primitive to the local application process, the local Application Layer shall need a T_Data_Broadcast.con from the local Transport Layer. If the confirmation is positive (t_status = ok), the local Application Layer shall pass a positive A_IndividualAddress_Write.Lcon(a_status = ok) to the local application process. If the confirmation is negative (t_status = not_ok), the local Application Layer shall pass an A_IndividualAddress_Write.Lcon (a_status = not_ok) to the local user indicating that the transmission of the associated T_Data_Broadcast.req did not succeed.

A_IndividualAddress_Write.req(ack_request, priority, hop_count_type, newaddress)

ack_request: Data Link Layer Acknowledge requested or don't care
 priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 newaddress: the new value of the Individual Address

A_IndividualAddress_Write.Lcon(ack_request, priority, hop_count_type, newaddress, a_status)

ack_request: Data Link Layer Acknowledge requested or don't care
 priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 newaddress: the new value of the Individual Address
 a_status: ok: A_IndividualAddress_Write sent successfully with T_Data_Broadcast service
 not_ok: transmission of the associated T_Data_Broadcast request frame did not succeed

A_IndividualAddress_Write.ind(priority, hop_count_type, newaddress)

priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 newaddress: the new value of the Individual Address

3.2.3 A_IndividualAddress_Read-service

The A_IndividualAddress_Read.req primitive shall be applied by the user of Application Layer to read the Individual Address in a communication partner. The communication partner shall not be identified in the service, i.e. the destination must be defined by selecting a destination manually. This can be done by pressing a button on one or more devices that brings these devices into a 'programming mode', i.e. only a device where the button is pressed shall accept the A_IndividualAddress_Read.ind, others shall ignore it. The way that a product is set to 'programming mode' may be manufacturer specific.

The local Application Layer shall accept the service request and shall pass it with a T_Data_Broadcast.req to the local Transport Layer. The parameter priority, implicitly with value 'system', shall be mapped to the corresponding parameter of the T_Data_Broadcast.req primitive; the TSDU shall be an A_IndividualAddress_Read-PDU.

The remote Application Layer shall map a T_Data_Broadcast.ind primitive with TSDU = A_IndividualAddress_Read-PDU to an A_IndividualAddress_Read.ind primitive. The argument priority, implicitly with value 'system', shall be mapped to the corresponding argument priority of the A_IndividualAddress_Read.ind primitive.

The application process shall respond to the A_IndividualAddress_Read.ind primitive with an A_IndividualAddress_Read.res primitive only if the device is in 'programming mode'.

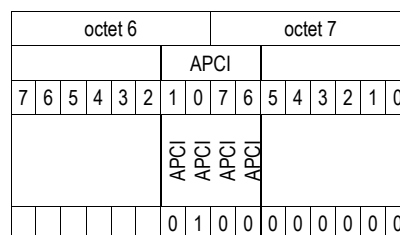


Figure 10 - A_IndividualAddress_Read-PDU (example)

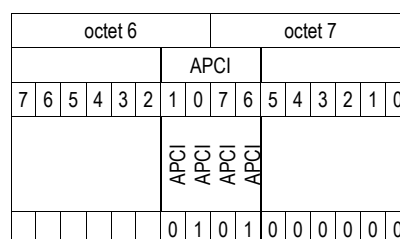


Figure 11 - A_IndividualAddress_Response-PDU (example)

The remote Application Layer shall accept the service response and pass it with a T_Data_Broadcast.req to the Transport Layer; the TSDU shall be an A_IndividualAddress_Response-PDU. The local Application Layer shall map a T_Data_Broadcast.ind primitive with TSDU = A_IndividualAddress_Response-PDU to an A_IndividualAddress_Read.Acon primitive. The argument priority, implicitly with value 'system', shall be mapped to the corresponding argument priority of the A_IndividualAddress_Read.Acon primitive. The argument source_address shall be mapped to the corresponding argument individual_address of the A_IndividualAddress_Read.Acon primitive.

A_IndividualAddress_Read.req(ack_request, priority, hop_count_type)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value

A_IndividualAddress_Read.Lcon(ack_request, priority, hop_count_type, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
a_status:	ok: A_IndividualAddress_Read sent successfully with T_Data_Broadcast service
	not_ok: transmission of the associated T_Data_Broadcast request frame did not succeed

A_IndividualAddress_Read.ind(priority, hop_count_type)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value

A_IndividualAddress_Read.res(ack_request, priority, hop_count_type, Individual Address)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
individual_address:	the value of the Individual Address

A_IndividualAddress_Read.Rcon(ack_request, priority, hop_count_type, individual_address, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
individual_address:	the value of the Individual Address
a_status:	ok: A_IndividualAddress_Read sent successfully with T_Data_Broadcast service
	not_ok: transmission of the associated T_Data_Broadcast request frame did not succeed

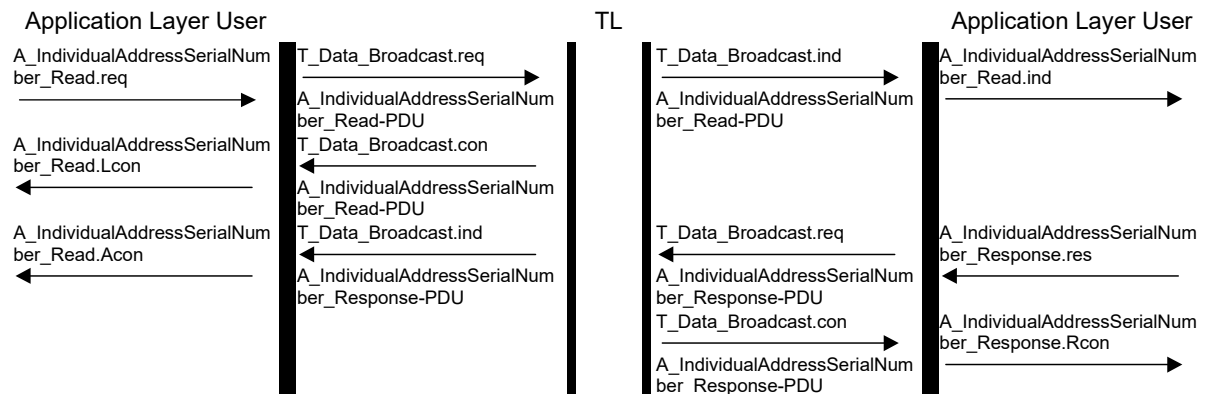
A_IndividualAddress_Read.Acon(priority, hop_count_type, Individual Address)

priority: system, urgent, normal or low priority

hop_count_type:	standard NL hop count value
-----------------	-----------------------------

individual address: the value of the Individual Address

3.2.4 A IndividualAddressSerialNumber_Read-service



The A_IndividualAddressSerialNumber_Read.req primitive shall be applied by the user of Application Layer to read the Individual Address in a communication partner. The communication partner shall be identified using the unique KNX Serial Number (6 octets) of the device. KNX Serial numbers are administered by the KNX Association.

The local Application Layer shall accept the service request and pass it with a T_Data_Broadcast.req to the local Transport Layer. The parameter priority, implicitly with value ‘system’, shall be mapped to the corresponding parameter of the T_Data_Broadcast.req primitive; the TSDU shall be an A_Individual-AddressSerialNumber_Read-PDU.

Prior to passing an A_IndividualAddressSerialNumber_Read.Lcon to the local user, the local Application Layer shall need a T_Data_Broadcast.con from the local Transport Layer. If the confirmation is positive (t_status = ok), the local Application Layer shall pass a positive A_IndividualAddressSerialNumber_Read.Lcon (a_status = ok) to the local user. If the confirmation is negative (a_status = not_ok), the local Application Layer shall pass an A_IndividualAddressSerialNumber_Read.Lcon (a_status = not_ok) to the local user indicating that the transmission of the associated A_IndividualAddressSerialNumber_Read.req did not succeed.

The remote Application Layer shall map a T_Data_Broadcast.ind primitive with TSDU = A_IndividualAddressSerialNumber_Read-PDU to an A_IndividualAddressSerialNumber_Read.ind primitive. The argument priority, implicitly with value 'system', shall be mapped to the corresponding argument priority of the A_IndividualAddressSerialNumber_Read.ind primitive.

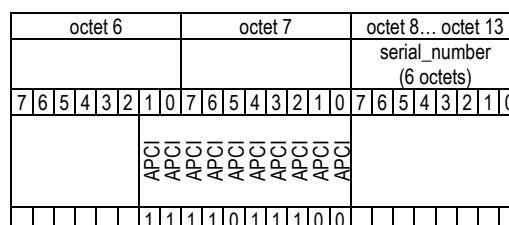


Figure 12 - A IndividualAddressSerialNumber Read-PDU (example)

The application process shall respond to the A_IndividualAddressSerialNumber_Read.ind primitive with an A_IndividualAddressSerialNumber_Read.res primitive, if the KNX Serial Number received is equal to the KNX Serial Number of the device.

The remote Application Layer shall accept the service response and pass it with a T_Data_Broadcast.req to the remote Transport Layer; the TSDU shall be an A_IndividualAddressSerialNumber_Response-PDU.

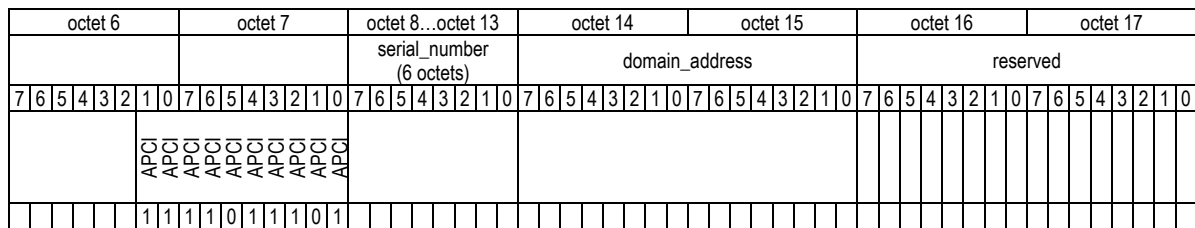


Figure 13 - A_IndividualAddressSerialNumber_Response-PDU (example)

Prior to passing an A_IndividualAddressSerialNumber_Read.Rcon to the remote user, the remote Application Layer shall need a T_Data_Broadcast.con from the remote Transport Layer. If the confirmation is positive (t_status = ok), the remote Application Layer shall pass a positive A_IndividualAddressSerialNumber_Read.Rcon (a_status = ok) to the remote user. If the confirmation is negative (a_status = not_ok), the remote Application Layer shall pass an A_IndividualAddressSerialNumber_Read.Rcon (a_status = not_ok) to the remote user indicating that the transmission of the associated A_IndividualAddressSerialNumber_Read.res did not succeed.

The local Application Layer shall map a T_Data_Broadcast.ind primitive with TSDU = A_IndividualAddressSerialNumber_Response-PDU to an A_IndividualAddressSerialNumber_Read.Acon primitive. The argument priority, implicitly with value 'system', shall be mapped to the corresponding argument priority of the A_IndividualAddressSerialNumber_Read.Acon primitive. The argument source_address shall be mapped to the argument individual_address.

A_IndividualAddressSerialNumber_Read.req (ack_request, priority, hop_count_type, serial_number)

ack_request: Data Link Layer Acknowledge requested or don't care
 priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 serial_number: the KNX Serial Number

A_IndividualAddressSerialNumber_Read.Lcon (ack_request, priority, hop_count_type, serial_number, a_status)

ack_request: Data Link Layer Acknowledge requested or don't care
 priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 serial_number: the KNX Serial Number
 a_status ok: A_IndividualAddressSerialNumber_Read sent successfully with T_Data_Broadcast service
 not_ok: transmission of the associated T_Data_Broadcast request frame did not succeed

A_IndividualAddressSerialNumber_Read.ind (priority, hop_count_type, serial_number)

priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 serial_number: the KNX Serial Number

A_IndividualAddressSerialNumber_Read.res (ack_request, priority, hop_count_type, serial_number, domain_address)

ack_request: Data Link Layer Acknowledge requested or don't care
 priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 serial_number: the KNX Serial Number
 domain_address: the Domain Address of the remote device

A_IndividualAddressSerialNumber_Read.Rcon (ack_request, priority, hop_count_type, serial_number, a_status)

ack_request: Data Link Layer Acknowledge requested or don't care
 priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 serial_number: the KNX Serial Number
 a_status ok: A_IndividualAddressSerialNumber_Response sent successfully with T_Data_Broadcast service
 not_ok: transmission of the associated T_Data_Broadcast request frame did not succeed

A_IndividualAddressSerialNumber_Read.Acon (priority, hop_count_type, serial_number, individual_address, domain_address)

priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 serial_number: the KNX Serial Number
 individual_address: the value of the Individual Address
 domain_address: the Domain Address of the remote device

3.2.5 A_IndividualAddressSerialNumber_Write-service

The A_IndividualAddressSerialNumber_Write.req primitive shall be applied by the user of Application Layer, to modify the Individual Address in a communication partner. The communication partner shall be identified using the unique KNX Serial Number (6 octets) of the device.

The local Application Layer shall accept the service request and pass it with a T_Data_Broadcast.req to the local Transport Layer. The parameter priority, implicitly with value 'system', shall be mapped to the corresponding parameter of the T_Data_Broadcast.req primitive; the TSDU shall be an A_Individual-AddressSerialNumber_Write-PDU.

octet 6						octet 7						octet 8 ... octet 13						octet 14						octet 15						octet 16 ... octet 19									
						APCI						serial_number (6 octets)						newaddress (high)						newaddress (low)						reserved (4 octets)									
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																								
						1	1	1	1	0	1	1	1	1	0																								

Figure 14 - A_IndividualAddressSerialNumber_Write-PDU (example)

The remote Application Layer shall map a T_Data_Broadcast.ind primitive with TSDU = A_IndividualAddressSerialNumber_Write-PDU to an A_IndividualAddressSerialNumber_Write.ind primitive. The argument priority shall be mapped to the corresponding argument priority of the A_IndividualAddressSerialNumber_Write.ind primitive.

Prior to passing an A_IndividualAddressSerialNumber_Write.Lcon primitive to the local application process, the local Application Layer shall need a T_Data_Broadcast.con from the local Transport Layer. If the confirmation is positive (t_status = ok), the local Application Layer shall pass a positive A_IndividualAddressSerialNumber_Write.Lcon(a_status = ok) to the local application process. If the confirmation is negative (t_status = not_ok), the local Application Layer shall pass an A_IndividualAddressSerialNumber_Write.Lcon(a_status = not_ok) to the local user indicating that the transmission of the associated T_Data_Broadcast.req did not succeed.

A_IndividualAddressSerialNumber_Write.req(ack_request, priority, hop_count_type, serial_number, newaddress)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
serial_number:	the serial number
newaddress:	the new value of the Individual Address

A_IndividualAddressSerialNumber_Write.Lcon(ack_request, priority, hop_count_type, newaddress, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
newaddress:	the new value of the Individual Address
a_status:	ok: A_IndividualAddressSerialNumber_Write sent successfully with T_Data_Broadcast service
	not_ok: transmission of the associated T_Data_Broadcast request frame did not succeed

A_IndividualAddressSerialNumber_Write.ind(priority, hop_count_type, serial_number, newaddress)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
serial_number:	the serial number
newaddress:	the new value of the Individual Address

3.2.6 A_NetworkParameter_Read-service

The `A_NetworkParameter_Read.req` primitive shall be applied by the user of Application Layer of the management client to check about the configuration of a network parameter. The service request shall be communicated on either point-to-point connectionless or on point-to-all-points (broadcast) communication mode. This shall be specified in the parameter `comm_mode_req` of the `A_NetworkParameter_Read.req` primitive. A device shall respond to the service if it complies with the conditions specified in the service parameters.

The local Application Layer shall accept the service request. If the parameter `comm_mode_req` equals “point-to-point, connectionless”, it shall pass the service with a `T_Data_Individual.req` to the local Transport Layer; the parameter `ASAP` shall be mapped to the corresponding parameter `TSAP` of the `T_Data_Individual.req` primitive. If the parameter `comm_mode_req` equals “point-to-all-points connectionless”, it shall pass the service with a `T_Data_Broadcast.req` to the local Transport Layer. The TSDU shall in both cases always be an `A_NetworkParameter_Read-PDU`. The parameter `priority`, implicitly with value ‘system’, shall be mapped to the corresponding parameter of the `T_Data_Broadcast.req` primitive, the TSDU shall be an `A_NetworkParameter_Read-PDU`.

If the remote Application Layer receives a T_Data_Individual.ind or a T_Data_Broadcast.ind with TSDU = A_NetworkParameter_Read-PDU, it shall map the service primitive to an A_NetworkParameter_Read.ind primitive. The parameter priority, implicitly with value 'system', shall be mapped to the corresponding parameter priority of the A_NetworkParameter_Read.ind primitive.

octet 6							octet 7							octet 8							octet 9							octet 10							octet 11...n														
							APCI							parameter_type																												test_info							
															object_type														PID																				
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0		
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI																																										
							APCI						</																																				

Figure 15 - A NetworkParameter Read-PDU (example)

If the remote application process finds the conditions for replying are not fulfilled, this is if it does not support the read network parameter or the check of its investigated parameters against the test information is negative, it shall ignore the service.

If the remote application process accepts the service, it shall respond to the A_NetworkParameter_Read.ind primitive with an A_NetworkParameter_Read.res primitive after a random wait time in the range of 0 s ... 10 x T_{media} s³⁾. This random wait time is per parameter_type specified in [05]. For parameter types read in point-to-point communication mode, the random wait time is 0 s. The data in the response shall depend on the network parameter type being read.

The remote Application Layer shall accept the service response. If the parameter `comm_mode` equals “point-to-point, connectionless”, it shall pass the service with a `T_Data_Individual.req` to the local Transport Layer; the parameter `ASAP` shall be mapped to the corresponding parameter `TSAP` of the `T_Data_Individual.req` primitive. If the parameter `comm_mode` equals “point-to-all-points connectionless”, it shall pass the service with a `T_Data_Broadcast.req` to the local Transport Layer. The TSDU shall always be an `A_NetworkParameter_Response-PDU`.

3) T_{media} is specified in [05].

[illegible]

A_NetworkParameter_Read.req(ASAP, comm_mode_req, hop_count_type, parameter_type, priority, test_info)

ASAP:	local reference of the Service Access Point or Individual Address
comm_mode_req:	point-to-all-points connectionless communication mode or point-to-point connectionless communication mode
hop_count_type:	standard NL hop count value
parameter_type:	network parameter type that is verified, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
test_info:	value against which the resource indicated by parameter_type is tested

A_NetworkParameter_Read.Lcon(ASAP, comm_mode_req, hop_count_type, parameter_type, priority, test_info, a_status)

ASAP:	local reference of the Service Access Point or Individual Address
comm_mode_req:	point-to-all-points connectionless communication mode or point-to-point connectionless communication mode
hop_count_type:	standard NL hop count value
parameter_type:	network parameter type that is verified, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
test_info	value against which the resource indicated by parameter_type is tested
a_status:	ok: A_NetworkParameter_Read.req sent successfully with T_Data_Broadcast service
	not_ok: transmission of the associated T_Data_Broadcast request frame did not succeed

A_NetworkParameter_Read.ind(ASAP, comm_mode_req, hop_count_type, parameter_type, priority, test_info)

ASAP:	local reference of the Service Access Point or individual address
comm_mode_req:	point-to-all-points connectionless communication mode or point-to-point connectionless communication mode
hop_count_type:	standard NL hop count value
parameter_type:	network parameter type that is verified, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
test_info:	value against which the resource indicated by parameter_type is tested

A_NetworkParameter_Read.res(ASAP, comm_mode, hop_count_type, parameter_type, priority, test_info, test_result)

ASAP:	local reference of the Service Access Point or individual address
comm_mode:	point-to-all-points connectionless communication mode or point-to-point connectionless communication mode
hop_count_type:	standard NL hop count value
individual_address:	the destination address for the service
parameter_type:	network parameter type that is verified, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
test_info:	value against which the resource indicated by parameter_type is tested
test_result:	parameter_type dependent response

A_NetworkParameter_Read.Rcon(ASAP, comm_mode, hop_count_type, parameter_type, priority, test_info, test_result, a_status)

ASAP:	local reference of the Service Access Point or individual address
comm_mode:	point-to-all-points connectionless communication mode or point-to-point connectionless communication mode
hop_count_type:	standard NL hop count value
individual_address:	the destination address for the service
parameter_type:	network parameter type that is verified, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
test_info:	value against which the resource indicated by parameter_type is tested
test_result:	parameter_type dependent response
a_status: ok:	A_NetworkParameter_Read.res sent successfully with T_Data_Broadcast or T_Data_Individual service
not_ok:	transmission of the associated T_Data_Broadcast or T_Data_Individual request frame did not succeed

A_NetworkParameter_Read.Acon(ASAP, hop_count_type, individual_address, parameter_type, priority, test_info, test_result)

ASAP:	local reference of the Service Access Point or individual address
hop_count_type:	standard NL hop count value
individual_address:	the individual address of the device that has sent the response
parameter_type:	network parameter type that is verified, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
test_info:	value against which the resource indicated by parameter_type is tested
test_result:	parameter_type dependent response

3.2.7 A_NetworkParameter_Write-service

The A_NetworkParameter_Write.req primitive shall be applied by the user of Application Layer of the management client to set network configuration information in one or multiple management servers. The service shall be transmitted on either point-to-point connection-less or on point-to-all points (broadcast) communication mode. This shall be specified in the parameter comm_mode of the A_NetworkParameter_Write.req primitive.

The local Application Layer shall accept the service request. If the parameter `comm_mode` equals “point-to-point, connectionless”, it shall pass the service with a `T_Data_Individual.req` to the local Transport Layer; the parameter `ASAP` shall be mapped to the corresponding parameter `TSAP` of the `T_Data_Individual.req` primitive. If the parameter `comm_mode` equals “point-to-all-points connectionless”, it shall pass the service with a `T_Data_Broadcast.req` to the local Transport Layer. The TSDU shall in both cases always be an `A_NetworkParameter_Write-PDU`.

If the remote Application Layer receives a T_Data_Individual.ind or a T_Data_Broadcast.ind with TSDU = A_NetworkParameter_Write-PDU, it shall map the service primitive to an A_NetworkParameter_Write.ind primitive. The parameter priority, implicitly with value 'system', shall be mapped to the corresponding parameter priority of the A_NetworkParameter_Write.ind primitive.

[illegible]

Figure 17 - A NetworkParameter_Write-PDU (example)

Prior to passing an A_NetworkParameter_Write.Lcon primitive to the local application process, the local Application Layer shall need a T_Data_Individual.con or a T_Data_Broadcast.con, depending on the parameter comm_mode of the service request, from the local Transport Layer. If the confirmation is positive (t_status = ok), the local Application Layer shall pass a positive A_Network-Parameter_Write.Lcon(a_status = ok) to the local application process. If the confirmation is negative (t_status = not_ok), the local Application Layer shall pass an A_NetworkParameter_Write.Lcon(a_status = not_ok) to the local user indicating that the transmission of the associated Transport Layer specification did not succeed.

NOTE 2 This service is as such not confirmed by the remote partner. The A_NetworkParameter_Write.Lcon is only a local confirmation caused by the local Transport Layer confirmation, basically caused by the Data Link Layer confirmation (ok, not ok).

A NetworkParameter Write.req(ASAP, comm mode, hop count type, parameter type, priority, value)

ASAP:	local reference of the Service Access Point or Individual Address
comm_mode:	point-to-all-points connectionless communication mode or point-to-point connectionless communication mode
hop_count_type:	standard NL hop count value
parameter_type:	the network parameter that shall be set, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
value:	value to which the network parameter indicated by parameter_type shall be set

A_NetworkParameter_Write.Lcon(ASAP, comm_mode, hop_count_type, parameter_type, priority, value, a_status)

ASAP:	local reference of the Service Access Point or Individual Address
comm_mode:	point-to-all-points connectionless communication mode or point-to-point connectionless communication mode
hop_count_type:	standard NL hop count value
parameter_type:	the network parameter that shall be set, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
value:	value to which the network parameter indicated by parameter_type shall be set
a_status:	ok: A_NetworkParameter_Write-PDU sent successfully with the requested Transport Layer service
	not_ok: transmission of the requested Transport Layer service did not succeed

A_NetworkParameter_Write.ind(ASAP, parameter_type, priority, value)

ASAP:	local reference of the Service Access Point or Individual Address
parameter_type:	the network parameter that shall be set, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
value:	value to which the network parameter indicated by parameter_type shall be set

Error and exception handling

- **Unknown parameter_type**

If an unknown parameter_type is received, the receiver shall neglect the A_NetworkParameter_Write-PDU without further action.

- **Data consistency**

In general, if over this service data is accessed that is also accessible over additional mechanisms, like over the A_PropertyValue_Read or the A_PropertyValue_Write, via dedicated services like the A_IndividualAddress_Read or the A_IndividualAddress_Write, as a memory mapped Resource or via local access using EMI, the receiver shall take care of consistency between these access modes.

The network – and device Management Procedures (see [05]) that base on this Application Layer services may specify procedure specific error handling.

3.2.8 A_NetworkParameter_InfoReport

The A_NetworkParameter_InfoReport.req primitive shall be applied by the user of Application Layer, which shall be a Property Server, to inform one or all Management Clients about the value of an Interface Object Property. This Interface Object Property shall be identified by the field parameter_type, which shall contain the Object Type and the Property Identifier. The field test_info shall have a value that shall be specified for each use case of this service, that is, for each specific parameter_type.

- If the Management Server may hold more than one instance of the Object Type, then it shall be specified for the specific use of this service if the Object Instance shall be encoded. If it is to be encoded, then the Object Instance shall be encoded as part of the field test_result.
- The Management Server may typically use this service to announce the value of one of its Properties. If it is to be encoded, then the Property Value shall be encoded as part of the field test_result.

NOTE 3 For possible use of the service A_NetworkParameter_InfoReport, please refer to the specification of the Management Procedure DMP_InterfaceObjectInfoReport_RCI in [05] and its examples.

A_NetworkParameter_InfoReport.Lcon(ASAP, comm_mode_req, hop_count_type, parameter_type, priority, test_info, test_result, a_status)

ASAP:	local reference of the Service Access Point or Individual Address
comm_mode_req	point-to-all-points connectionless communication mode or point-to-point connectionless communication mode
hop_count_type:	hop count 0 or Network Layer Parameter
parameter_type:	the network parameter that shall be set, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
test_info:	parameter type dependent value
test_result	parameter type dependent value
a_status:	ok: A_NetworkParameter_InfoReport sent successfully with T_Data_Individual - or T_Data_Broadcast service not_ok: transmission of the associated T_Data_Individual – or T_Data_Broadcast request frame did not succeed

A_NetworkParameter_InfoReport.ind(ASAP, comm_mode_req, hop_count_type, parameter_type, priority, test_info, test_result)

ASAP:	the Individual Address of the device that has sent the A_NetworkParameter_InfoReport-PDU.
comm_mode_req	point-to-all-points connectionless communication mode or point-to-point connectionless communication mode
hop_count_type:	hop count 0 or Network Layer Parameter
parameter_type:	the network parameter that shall be set, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
test_info:	parameter type dependent value
test_result:	parameter type dependent value

Error - and – exception handling

No error handling shall be applied. The service shall not be remote confirmed.

3.3 Application Layer Services on System Broadcast communication mode

3.3.1 Introduction

The Application Layer PDUs specified in this clause 3.3 may be transmitted on

- point-to-all-points, connectionless (system broadcast) communication mode (T_Data_SystemBroadcast), or on
- point-to-domain, connectionless (broadcast) communication mode (T_Data_Broadcast).

If transmitted or forwarded on a closed medium (Twisted Pair, KNX IP) then in any case only point-to-domain, connectionless (broadcast) communication mode (T_Data_Broadcast) can and shall be used. The below conditions apply for the transmission on an open medium (Powerline, Radio Frequency).

For the MaC

The MaC may use system broadcast communication mode or broadcast communication mode in function of the situation, according to the configuration procedures.

For the MaS (end device)

NOTE 4 The system broadcast services are designed for network configuration, in which the MaC is always the initiator. Hence, here only the behaviour of the MaS is given if responding to a service.

It depends on the MaS whether the system broadcast services are supported as well on broadcast communication mode or not.

- If this is not the case, then the MaS shall always respond using system broadcast communication mode.
- If this is the case, then the MaS shall respond with the same communication mode as the preceding incoming request.

For the Media Coupler

The communication mode that shall be used when forwarding the system broadcast messages onto the open medium may either be fixed to system broadcast or may depend on Coupler Parameter settings.

EXAMPLE 1 The KNX TP1/RF Media Coupler can be configured to route TP1 broadcast Frames containing a system broadcast service-PDU on either broadcast or on system broadcast.

3.3.2 A_DeviceDescriptor_InfoReport-service

NOTE 5 This service uses the same 4 bit APCI as APCI_DeviceDescriptor_Response.

The A_DeviceDescriptor_InfoReport.req primitive shall be applied by the local user of Application Layer to send the Device Descriptor to remote communication partners. The remote communication partner shall not be identified in the service, this is, the destination shall be defined by setting it in “teaching mode”. Only remote devices in this mode shall accept the A_DeviceDescriptor_InfoReport-PDU, other device shall ignore it.

The local Application Layer shall accept the service request and pass it with a T_Data_System-Broadcast.req to the local Transport Layer. The parameter priority, implicitly with value “system”, shall be mapped to the corresponding parameter of the T_Data_SystemBroadcast.req primitive, the TSDU shall be an A_DeviceDescriptor_InfoReport-PDU.

octet 6							octet 7							octet 8							...octet n															
							APCI			descriptor_type				device_descriptor																						
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0					
						APCI																														
						APCI																														
						APCI																														
						1	1	0	1																											

Figure 19 - A_DeviceDescriptor_InfoReport-PDU (example)

The remote Application Layer shall map a T_Data_SystemBroadcast.ind primitive with TSDU = A_DeviceDescriptor_InfoReport-PDU to an A_DeviceDescriptor_InfoReport.ind primitive. The argument priority, implicitly with value “system”, shall be mapped to the corresponding argument priority of the A_DeviceDescriptor_InfoReport.ind primitive.

The remote application process shall ignore the A_DeviceDescriptor_InfoReport.ind primitive if the remote device is not in “teaching mode”. Otherwise the received Device Descriptor shall be processed.

Prior to passing an A_DeviceDescriptor_InfoReport.Lcon primitive to the local application process, the local Application Layer shall need a T_Data_SystemBroadcast.con from the local Transport Layer. If the confirmation is positive (t_status = ok), the local Application Layer shall pass a positive A_DeviceDescriptor_InfoReport.Lcon (a_status = ok) to the local Application process. If the confirmation is negative (t_status = not_ok), the local Application Layer shall pass an A_DeviceDescriptor_InfoReport.Lcon (a_status = not_ok) to the local user indicating that the transmission of the associated T_Data_SystemBroadcast.req did not succeed.

A_DeviceDescriptor_InfoReport.req(ack_request, descriptor_type, device_descriptor, hop_count_type, priority)

ack_request:	Data Link Layer Acknowledge requested or don't care
descriptor_type:	type of the device descriptor
device_descriptor:	the device descriptor of the device
hop_count_type:	standard NL hop count value
priority:	system, urgent, normal or low priority

A_DeviceDescriptor_InfoReport.Lcon(ack_request, descriptor_type, device_descriptor, hop_count_type, priority, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
descriptor_type:	type of the descriptor
device_descriptor:	the device descriptor of the device
hop_count_type:	standard NL hop count value
priority:	system, urgent, normal or low priority
a_status:	ok: A_DeviceDescriptor_InfoReport sent successfully with T_Data_SystemBroadcast service
	not_ok: transmission of the associated T_Data_SystemBroadcast request frame did not succeed

A_DeviceDescriptor_InfoReport.ind(descriptor_type device_descriptor, hop_count_type, priority)

descriptor_type:	type of the descriptor
device_descriptor:	the device descriptor of the device
hop_count_type:	standard NL hop count value
priority:	system, urgent, normal or low priority

Error and Exception handling

No error handling is applied. The service is not remote confirmed.

3.3.3 A_DomainAddress_Write-service

The A_DomainAddress_Write.req primitive shall be applied by the user of Application Layer, to modify the Domain Address in a communication partner. The Domain Address shall be encoded according to the Domain Address format used on the medium of the communication partner either as a 2 octet value (KNX-PL110) or a 6 octet value (KNX-RF). The communication partner shall not be identified in the service, i.e. the destination must be defined by selecting a destination manually. This can be done by pressing a button on exactly one device that brings this device into a 'programming' mode, i.e., only the device where the button is pressed shall accept the A_DomainAddress_Write.ind, others shall ignore it. The way that a product is set to 'programming' mode may be manufacturer specific.

The local Application Layer shall accept the service request and shall pass it with a T_Data_SystemBroadcast.req to the local Transport Layer. The parameter priority shall be mapped to the corresponding parameter of the T_Data_SystemBroadcast.req primitive. The TSDU shall be an A_DomainAddress_Write-PDU.

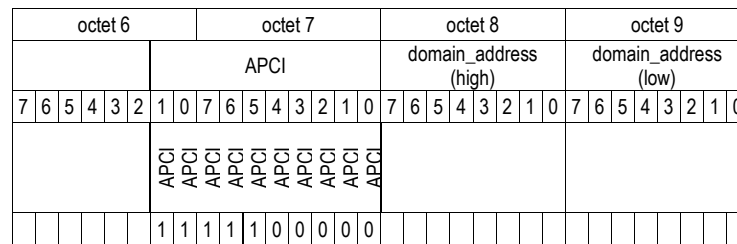


Figure 20 - A_DomainAddress_Write-PDU for a 2 octet DoA format (example)

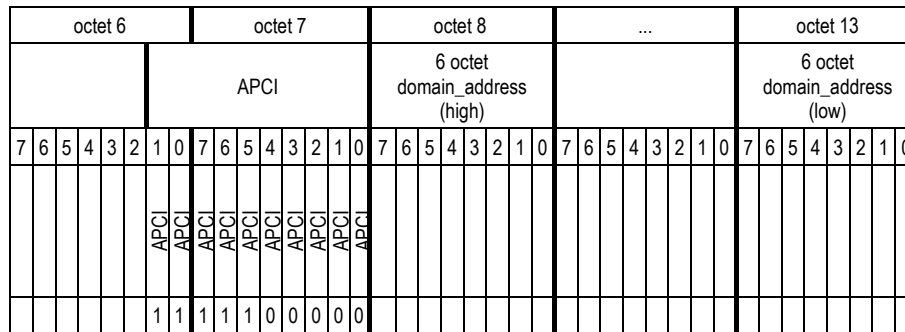


Figure 21 - A_DomainAddress_Write-PDU for a 6 octet DoA format (example)

The remote Application Layer shall map a T_Data_SystemBroadcast.ind primitive with TSDU = A_DomainAddress_Write-PDU to an A_DomainAddress_Write.ind primitive. The arguments priority and domain_address_new shall be mapped to the corresponding arguments of the A_DomainAddress_Write.ind primitive.

The application process shall ignore the A_DomainAddress_Write.ind primitive if the device is not in 'programming' mode.

Prior to passing an A_DomainAddress_Write.Lcon primitive to the local application process, the local Application Layer needs a T_Data_SystemBroadcast.con from the local Transport Layer. If the confirmation is positive (t_status = ok), the local Application Layer shall pass a positive A_DomainAddress_Write.Lcon(a_status = ok) to the local application process. If the confirmation is negative (t_status = not_ok), the local Application Layer shall pass an A_DomainAddress_Write.Lcon(a_status = not_ok) to the local user indicating that the transmission of the associated T_Data_SystemBroadcast.req did not succeed.

A_DomainAddress_Write.req(ack_request, priority, hop_count_type, domain_address_new)

ack_request: Data Link Layer Acknowledge requested or don't care

priority: system, urgent, normal or low priority

hop_count_type: standard NL hop count value

domain_address_new: the new value of the Domain Address

A_DomainAddress_Write.Lcon(ack_request, priority, hop_count_type, domain_address_new, a_status)

ack_request: Data Link Layer Acknowledge requested or don't care

priority: system, urgent, normal or low priority

hop_count_type: standard NL hop count value

domain_address_new: the new value of the Domain Address

a_status: ok: A_DomainAddress_Write sent successfully with T_Data_SystemBroadcast service

not_ok: transmission of the associated T_Data_SystemBroadcast request frame did not succeed

A_DomainAddress_Write.ind(priority, hop_count_type, domain_address_new)

priority: system, urgent, normal or low priority

hop_count_type: standard NL hop count value

domain_address_new: the new value of the Domain Address

3.3.4 A_DomainAddress_Read-service

The A_DomainAddress_Read.req primitive shall be applied by the user of Application Layer, to read the Domain Address from a communication partner. The communication partner shall not be identified in the service, i.e. the destination must be defined by selecting a destination manually. This can be done by pressing a button on one or more devices that brings these devices into a 'programming' mode, i.e. only a device where the button is pressed shall accept the A_DomainAddress_Read.ind, others shall ignore it. The way that a product is set to 'programming' mode may be manufacturer specific.

The local Application Layer shall accept the service request and shall pass it with a T_Data_SystemBroadcast.req to the local Transport Layer. The parameter priority shall be mapped to the corresponding parameter of the T_Data_SystemBroadcast.req primitive, the TSDU shall be an A_DomainAddress_Read-PDU.

octet 6						octet 7									
						APCI									
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
						$\overline{A}$	$\overline{P}$	$\overline{C}$	$\overline{I}$	$\overline{A}$	$\overline{P}$	$\overline{C}$	$\overline{I}$	$\overline{A}$	$\overline{P}$
						1	1	1	1	1	0	0	0	0	1

Figure 22 - A_DomainAddress_Read-PDU (example)

The remote Application Layer shall map a T_Data_SystemBroadcast.ind primitive with TSDU = A_DomainAddress_Read-PDU to an A_DomainAddress_Read.ind primitive. The argument priority shall be mapped to the corresponding argument priority of the A_DomainAddress_Read.ind primitive.

The remote application process shall respond to the A_DomainAddress_Read.ind primitive with an A_DomainAddress_Read.res primitive only if the device is in 'programming' mode. The Domain Address shall be encoded according to the Domain Address format used on the medium of the remote application process either as a 2 octet value (KNX-PL110) or a 6 octet value (KNX-RF).

The remote Application Layer shall accept the service response and shall pass it with a T_Data_SystemBroadcast.req to the Transport Layer; the TSDU shall be an A_DomainAddress_Response-PDU.

octet 6						octet 7						octet 8						octet 9													
						APCI						domain_address (high)						domain_address (low)													
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
						APCI																									
						APCI																									
						APCI																									
						APCI																									
						1	1	1	1	1	0	0	0	1	0																

Figure 23 - A_DomainAddress_Response-PDU for a 2 octet DoA format (example)

octet 6						octet 7						octet 8						...						octet 13							
						APCI						6 octet domain_address (high)						...						6 octet domain_address (low)							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
						APCI APCI APCI APCI APCI APCI APCI APCI																									
						1	1	1	1	1	1	0	0	0	1	0															

Figure 24 - A DomainAddress_Response-PDU for a 6 octet DoA format (example)

The local Application Layer shall map a T_Data_SystemBroadcast.ind primitive with TSDU = A_DomainAddress_Response-PDU to an A_DomainAddress_Read.Acon primitive. The argument priority shall be mapped to the corresponding argument priority of the A_DomainAddress_Read.Acon primitive.

A DomainAddress Read.req(ack request, priority, hop count type)

ack request: Data Link Layer Acknowledge requested or don't care

priority: system, urgent, normal or low priority

hop count type: standard NL hop count value

A DomainAddress Read.Lcon(ack request, priority, hop count type, a status)

ack request: Data Link Layer Acknowledge requested or don't care

priority: system, urgent, normal or low priority

hop_count_type:	standard NL hop count value
-----------------	-----------------------------

```
a_status:      ok:      A_DomainAddress_Read.req sent successfully with T_Data_SystemBroadcast
                  service
```

not_ok: transmission of the associated T_Data_SystemBroadcast request frame did not succeed

A DomainAddress Read.ind(priority, hop count type)

priority: system, urgent, normal or low priority

hop count type: standard NL hop count value

A DomainAddress Read.res(ack request, priority, hop count type, domain address)

ack request: Data Link Layer Acknowledge requested or don't care

priority: system, urgent, normal or low priority

hop count type: standard NL hop count value

domain	address:	the value of the Domain Address
--------	----------	---------------------------------

A_DomainAddress_Read.Acon(priority, hop_count_type, domain_address)

priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 domain_address: the value of the Domain Address

3.3.5 A_DomainAddressSelective_Read-service

The A_DomainAddressSelective_Read.req primitive shall be applied by the user of Application Layer, to read the Domain Address from the communication partner that shall be identified within the service. This service is particularly used to check the existence of any open media devices with the specified Domain Address in possibly neighbouring installations.

NOTE 6 The A_DomainAddressSelective_Read-service is only specified for the 2 octet format DoA. It is not available for the 6 octet format DoA.

The local Application Layer shall accept the service request and shall pass it with a T_Data_System-Broadcast.req to the local Transport Layer. The parameter priority shall be mapped to the corresponding parameter of the T_Data_SystemBroadcast.req primitive, the TSDU shall be an A_DomainAddress-Selective_Read-PDU.

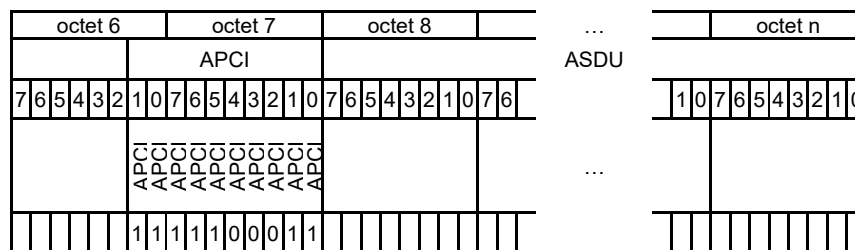


Figure 25 - A_DomainAddressSelective_Read-PDU (example)

The remote Application Layer shall map a T_Data_SystemBroadcast.ind primitive with TSDU = A_DomainAddressSelective_Read-PDU to an A_DomainAddressSelective_Read.ind primitive. The priority and the ASDU shall be passed to the remote Application Layer user.

If the remote Application Layer user confirms the A_DomainAddressSelective_Read.ind primitive, with ASDU specific data and wait times, with an A_DomainAddress_Read.res, then the remote Application Layer shall accept the service response and shall pass it with a T_Data_System-Broadcast.req to the remote Transport Layer; the TSDU shall be an A_DomainAddress_Response PDU.

A_DomainAddressSelective_Read.req(priority, hop_count_type, ASDU)

priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 ASDU: service specific parameters

A_DomainAddressSelective_Read.Lcon(priority, hop_count_type, domain_address, start_address, range, a_status)

priority: system, urgent, normal or low priority

hop_count_type: standard NL hop count value

ASDU: service specific parameters

a_status: ok: A_DomainAddressSelective_Read-PDU sent successfully with T_Data_SystemBroadcast service

not_ok: transmission of the associated T_Data_SystemBroadcast request frame did not succeed.

A_DomainAddressSelective_Read.ind(priority, hop_count_type, domain_address, start_address, range)

priority: system, urgent, normal or low priority

hop_count_type: standard NL hop count value

ASDU: service specific parameters

3.3.6 A_DomainAddressSerialNumber_Read-service

The A_DomainAddressSerialNumber_Read.req primitive shall be applied by the user of the Application Layer to read the Domain Address in a communication partner. The communication partner shall be identified using the unique KNX Serial Number (6 octet) of the communication partner.

The local Application Layer shall accept the service request and shall pass it with a T_Data_SystemBroadcast.req to the local Transport Layer. The parameter priority, implicitly with value 'system', shall be mapped to the corresponding parameter of the T_Data_SystemBroadcast.req primitive; the TSDU shall be an A_DomainAddressSerialNumber_Read-PDU.

The remote Application Layer shall map a T_Data_SystemBroadcast.ind primitive with TSDU = A_DomainAddressSerialNumber_Read-PDU to an A_DomainAddressSerialNumber_Read.ind primitive. The argument priority, implicitly with value 'system', shall be mapped to the corresponding argument priority of the A_DomainAddressSerialNumber_Read.ind primitive.

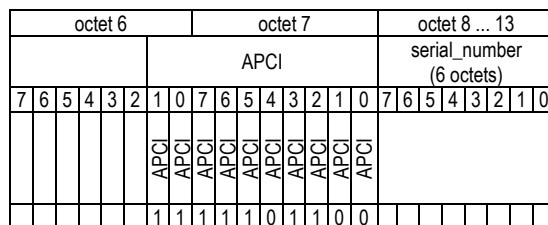


Figure 26 - A_DomainAddressSerialNumber_Read-PDU (example)

The application process shall respond to the A_DomainAddressSerialNumber_Read.ind primitive with an A_DomainAddressSerialNumber_Read.res primitive, if the KNX Serial Number received is equal to the KNX Serial Number of the device. The Domain Address shall be encoded according to the Domain Address format used on the medium of the remote application process either as a 2 octet value (KNX-PL110) or a 6 octet value (KNX-RF).

The remote Application Layer shall accept the service response and shall pass it with a T_Data_SystemBroadcast.req to the remote Transport Layer; the TSDU shall be an A_DomainAddressSerialNumber_Response-PDU.

octet 6						octet 7						octet 8 ... 13						octet 14						octet 15							
						APCI						serial_number (6 octets)						domain_address (high)						domain_address (low)							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																
						1	1	1	1	1	0	1	1	0	1																

Figure 27 - A_DomainAddressSerialNumber_Response-PDU for a 2 octet DoA format (example)

octet 6						octet 7						octet 8 ... 13						octet 14 ... 19					
						APCI						serial_number (6 octets)						domain_address (6 octets)					
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
						APCI	APCI	APCI	APCI	APCI	APCI												
						1	1	1	1	1	0	1	1	0	1								

Figure 28 - A_DomainAddressSerialNumber_Response-PDU for a 6 octet DoA format (example)

The local Application Layer shall map a T_Data_SystemBroadcast.ind primitive with TSDU = A_DomainAddressSerialNumber_Response-PDU to an A_DomainAddressSerialNumber_Read.con primitive. The argument priority, implicitly with value 'system', shall be mapped to the corresponding argument priority of the A_DomainAddressSerialNumber_Read.con primitive.

A_DomainAddressSerialNumber_Read.req (ack_request, hop_count_type, priority, serial_number)

ack_request: Data Link Layer Acknowledge requested or don't care
 hop_count_type: standard NL hop count value
 priority: system
 serial_number: the KNX Serial Number of the remote device

A_DomainAddressSerialNumber_Read.Lcon (ack_request, hop_count_type, priority, serial_number, a_status)

ack_request: Data Link Layer Acknowledge requested or don't care
 hop_count_type: standard NL hop count value
 priority: system
 serial_number: the KNX Serial Number of the remote device
 a_status ok: A_DomainAddressSerialNumber_Read-PDU sent successfully with T_Data_Broadcast service
 not_ok: transmission of the associated T_Data_Broadcast request frame did not succeed

A_DomainAddressSerialNumber_Read.ind (hop_count_type, priority, serial_number)

hop_count_type: standard NL hop count value
 priority: system
 serial_number: the KNX Serial Number of the remote device

A_DomainAddressSerialNumber_Read.res (ack_request, domain_address, hop_count_type, priority, serial_number)

ack_request: Data Link Layer Acknowledge requested or don't care

`domain address:` the Domain Address of the remote device

hop_count_type:	standard NL hop count value
-----------------	-----------------------------

priority: system

serial_number: the KNX Serial Number of the remote device

A_DomainAddressSerialNumber_Read.con (domain_address, hop_count_type, priority, serial_number)

`domain_address:` the Domain Address of the remote device

hop_count_type:	standard NL hop count value
-----------------	-----------------------------

priority: system

serial number: the KNX Serial Number

3.3.7 A_DomainAddressSerialNumber_Write-Service

The A_DomainAddressSerialNumber_Write.req primitive shall be applied by the user of Application Layer, to modify the Domain Address in a communication partner. The communication partner shall be identified using the unique KNX Serial Number (6 octet) of the device.

The Domain Address shall be encoded according to the Domain Address format used on the medium of the remote application process. This shall be one of the following.

- For KNX-PL110 this shall be a 2 octet value

[illegible]

**Figure 29 - A_DomainAddressSerialNumber_Write-PDU
for a 2 octet DoA format (example)**

- For KNX-RF this shall be a 6 octet value.

octet 6								octet 7								octet 8 to 13								octet 14 to 19							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
								APCI								serial_number (6 octets)								domain_address (6 octets)							
						1	1	1	1	1	0	1	1	1	0																

**Figure 30 - A_DomainAddressSerialNumber_Write-PDU
for a 6 octet DoA format (example)**

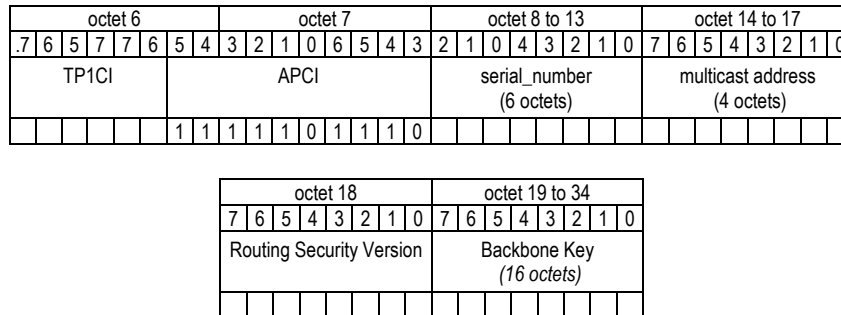
- For KNX IP plain this shall be a 4 octet value that shall consist of the *IP multicast address* only.

octet 6								octet 7								octet 8 to 13								octet 14 to 17							
7	6	5	7	6	5	4	3	2	1	0	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
TPCI								APCI								serial_number (6 octets)								multicast address (4 octets)							
								1	1	1	1	1	1	0	1	1	1	1	0												

Figure 31 – A_DomainAddressSerialNumber_Write-PDU for a 4 octet DoA format (example)

- For KNX IP Secure, this shall be a 21 octet value that shall consist of the *IP multicast address*, the *routing security version* and the *Backbone Key*.

The 21 octet format shall always be sent wrapped as S-A_Data-PDU with authentication and confidentiality. Please refer to the Access Policies for the A_DomainAddressSerialNumber_Write service.



**Figure 32 – A_DomainAddressSerialNumber_Write-PDU
for a 21 octet DoA format (example)**

The local Application Layer shall accept the service request and shall pass it with a T_Data_System-Broadcast.req to the local Transport Layer. The parameter priority, implicitly with value 'system', shall be mapped to the corresponding parameter of the T_Data_SystemBroadcast.req primitive; the TSDU shall be an A_DomainAddressSerialNumber_Write-PDU.

The remote Application Layer shall map a T_Data_SystemBroadcast.ind primitive with TSDU = A_DomainAddressSerialNumber_Write-PDU to an A_DomainAddressSerialNumber_Write.ind primitive. The argument priority shall be mapped to the corresponding argument priority of the A_DomainAddressSerialNumber_Write.ind primitive.

If the KNX Serial Number contained in the A_DomainAddressSerialNumber_Write-PDU is equal to the KNX Serial Number of the device, then the remote application process shall change its Domain Address to the value contained in the A_DomainAddressSerialNumber_Write-PDU; otherwise, it shall ignore the A_DomainAddressSerialNumber_Write.ind.

A_DomainAddressSerialNumber_Write.req(ack_request, domain_address, hop_count_type, priority, serial_number)

ack_request: Data Link Layer Acknowledge requested or don't care

backbone_key: the new security key for secure Routing

domain_address: the new value of the Domain Address (2 octet or 6 octet)

hop_count_type: standard NL hop count value

multicast address: the new routing multicast address for the MaS

priority: system

routing_security_version: the routing security version that the MaS shall activate

serial_number: the KNX Serial Number of the remote device

backbone_key:	the new security key for secure Routing
domain_address:	the new value of the Domain Address (2 octet or 6 octet)
hop_count_type:	standard NL hop count value
multicast address:	the new routing multicast address for the MaS
priority:	system, urgent, normal or low priority
routing_security_version:	the routing security version that the MaS shall activate
serial_number:	the KNX Serial Number of the remote device

The A_SystemNetworkParameter_Read.req primitive shall be applied by the user of Application Layer of the MaC to check about the configuration of a network parameter.

- On an open communication medium, the service request shall be communicated on point-to-all-points (system broadcast) communication mode.
- On a closed communication medium, the service request shall be communicated on point-to-domain, connectionless (broadcast) communication mode.

The local Application Layer shall accept the service request.

- If the local medium is an open medium, then it shall pass the service request with a T_Data_SystemBroadcast.req to the local Transport Layer.
- If the local medium is a closed medium, then it shall pass the service request with a T Data Broadcast.req to the local Transport Layer.

If the remote Application Layer receives a T_Data_SystemBroadcast.ind or a T_Data_Broadcast.ind with TSDU = A_SystemNetworkParameter_Read-PDU, it shall map the service primitive to an A_SystemNetworkParameter_Read.ind primitive. The parameter priority, implicitly with value ‘system’, shall be mapped to the corresponding parameter priority of the A_SystemNetworkParameter_Read.ind primitive.

octet 6						octet 7						octet 8						octet 9						octet 10						octet 11 ... n									
						APCI						parameter_type												test_info															
												object_type												PID						reserved				operand					
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
						APCI	APCI																																
						0	1	1	1	0	0	1	0	0	0																								

Figure 33 - A SystemNetworkParameter Read-PDU (example)

If the remote application process finds the conditions for replying are not fulfilled, this is if it does not support the read network parameter or the check of its investigated parameters against the test information is negative, it shall ignore the service.

If the remote application process accepts the service, it shall respond to the A_SystemNetworkParameter_Read.ind primitive with an A_SystemNetworkParameter_Read.res primitive after a random wait time that is per parameter_type specified in [05]. The data in the response shall depend on the network parameter type being read.

A_SystemNetworkParameter_Read.ind(hop_count_type, parameter_type, priority, test_info)

hop_count_type:	standard NL hop count value
parameter_type:	network parameter type that is verified, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
test_info:	value against which the resource indicated by parameter_type is tested

A_SystemNetworkParameter_Read.res(hop_count_type, parameter_type, priority, test_info, test_result)

hop_count_type:	standard NL hop count value
parameter_type:	network parameter type that is verified, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
test_info:	value against which the resource indicated by parameter_type is tested
test_result:	parameter_type dependent response

A_SystemNetworkParameter_Read.Rcon(hop_count_type, parameter_type, priority, test_info, test_result, a_status)

hop_count_type:	standard NL hop count value
parameter_type:	network parameter type that is verified, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
test_info:	value against which the resource indicated by parameter_type is tested
test_result:	parameter_type dependent response
a_status: ok:	A_NetworkParameter_Read.res sent successfully with T_Data_Broadcast or T_Data_Individual service
not_ok:	transmission of the associated T_Data_Broadcast or T_Data_Individual request frame did not succeed

A_SystemNetworkParameter_Read.Acon(ASAP, hop_count_type, parameter_type, priority, test_info, test_result)

ASAP:	local reference of the Service Access Point or individual address
hop_count_type:	standard NL hop count value
parameter_type:	network parameter type that is verified, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
test_info:	value against which the resource indicated by parameter_type is tested
test_result:	parameter_type dependent response

3.3.9 A_SystemNetworkParameter_Write

The A_SystemNetworkParameter_Write.req primitive shall be applied by the user of Application Layer of the MaC to set network configuration information in one or multiple management servers.

- On an open communication medium, the service request shall be communicated on point-to-all-points (system broadcast) communication mode.
- On a closed communication medium, the service request shall be communicated on point-to-domain, connectionless (broadcast) communication mode.

The local Application Layer shall accept the service request.

- If the local medium is an open medium, then it shall pass the service request with a T_Data_SystemBroadcast.req to the local Transport Layer.
- If the local medium is a closed medium, then it shall pass the service request with a T_Data_Broadcast.req to the local Transport Layer.

The TSDU shall in both cases always be an A_SystemNetworkParameter_Write-PDU. The parameter priority, implicitly with value 'system', shall be mapped to the corresponding parameter of the T_Data_SystemBroadcast.req respectively the T_Data_Broadcast.req primitive; the TSDU shall be an A_SystemNetworkParameter_Write-PDU.

If the remote Application Layer receives a T_Data_SystemBroadcast.ind or a T_Data_Broadcast.ind with TSDU = A_SystemNetworkParameter_Write-PDU, it shall map the service primitive to an A_SystemNetworkParameter_Write.ind primitive. The parameter priority, implicitly with value 'system', shall be mapped to the corresponding parameter priority of the A_SystemNetworkParameter_Write.ind primitive.

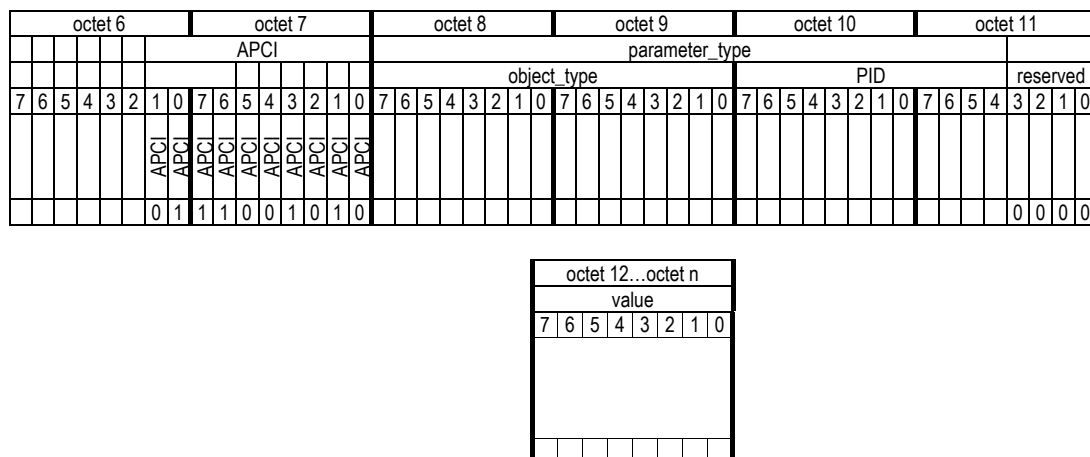


Figure 35 - A_SystemNetworkParameter_Write-PDU (example)

Prior to passing an A_SystemNetworkParameter_Write.Lcon primitive to the local application process, the local Application Layer shall need a T_Data_SystemBroadcast.con or a T_Data_Broadcast.con, depending on the communication mode of the service request, from the local Transport Layer. If the confirmation is positive (t_status = ok), the local Application Layer shall pass a positive A_SystemNetworkParameter_Write.Lcon(a_status = ok) to the local application process. If the confirmation is negative (t_status = not_ok), the local Application Layer shall pass an A_SystemNetworkParameter_Write.Lcon (a_status = not_ok) to the local user indicating that the transmission of the associated Transport Layer specification did not succeed.

NOTE 7 This service is as such not confirmed by the remote partner. The A_SystemNetworkParameter_Write.Lcon is only a local confirmation caused by the local Transport Layer confirmation, basically caused by the Data Link Layer confirmation (ok, not_ok).

A_SystemNetworkParameter_Write.req(hop_count_type, parameter_type, priority, value)

hop_count_type:	standard NL hop count value
parameter_type:	the network parameter that shall be set, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
value:	value to which the network parameter indicated by parameter_type shall be set

A_SystemNetworkParameter_Write.Lcon(hop_count_type, parameter_type, priority, value, a_status)

hop_count_type:	standard NL hop count value
parameter_type:	the network parameter that shall be set, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
value:	value to which the network parameter indicated by parameter_type shall be set
a_status:	ok: A_SystemNetworkParameter_Write.req sent successfully with T_Data_Broadcast – or with T_Data_SystemBroadcast- service not_ok: Transmission of the associated T_Data_Broadcast – or with T_Data_System-Broadcast request frame did not succeed

A_SystemNetworkParameter_Write.ind(hop_count_type, parameter_type, priority, value)

hop_count_type:	standard NL hop count value
parameter_type:	the network parameter that shall be set, structured as Interface Object Type and Property Identifier
priority:	system, urgent, normal or low priority
value:	value to which the network parameter indicated by parameter_type shall be set

Error and exception handling

- **Unknown parameter_type**

If an unknown parameter_type is received, the receiver shall neglect the A_SystemNetwork-Parameter_Write-PDU without further action.

- **Object Type and PID not accessible through A_SystemNetworkParameter_Write**

The A_SystemNetworkParameter_Write-service is no generic function to access any Property. Instead, the accessibility of a Property through this service is limited to those Properties for which this is indicated in [06].

- **Property specific use of A_SystemNetworkParameter_Write**

The service parameters Object Type, PID and Value are inherited from the access to Data Properties using e.g., A_PropertyValue_Write. Yet, this service may not only be used to simply set the Property Value but may Instead, have a use case specific meaning and interpretation in the field “value”, similar to the service A_FunctionProperty_Command. It is for each use of this service per Property specified in Chapter 3/5/1 “Resources” ([04]) or in Chapter 3/5/2 “Management Procedures” ([05]) how the field “value” shall be interpreted.

- **Data consistency**

In general, if over this service data is accessed that is also accessible over additional mechanisms, like over the A_PropertyValue_Read or the A_PropertyValue_Write, via dedicated services like the A_IndividualAddress_Read or the A_IndividualAddress_Write, as a memory mapped Resource or via local access using EMI, the receiver shall take care of consistency between these access modes.

The network – and device Management Procedures (see [05]) that base on this Application Layer service may specify procedure specific error handling.

3.4 Application Layer Services on Point-to-point Connectionless Communication Mode

3.4.1 Introduction

A point-to-point connectionless communication mode shall connect one device with another device. The following services can be applied on the point-to-point connectionless communication mode as well as on the point-to-point connection-oriented communication mode. The following clauses describe the mapping of the services on the point-to-point connectionless communication mode. For using these services on a connection oriented communication mode, the T_Data_Connected service of Transport Layer shall be applied Instead, of the T_Data_Individual service.

- For the connectionless communication mode, the ASAP parameter shall be the Individual Address of the communication partner.
- For the connection-oriented communication mode the ASAP parameter shall be the identifier of the communication relationship

The services on the point-to-point connectionless communication mode allow accessing Properties of Interface Objects in the communication partner. See [03] for a description of the Interface Objects and their structure.

3.4.2 Common services

3.4.2.1 A_DeviceDescriptor_Read-service

The A_DeviceDescriptor_Read.req primitive shall be applied by the user of Application Layer to read the Device Descriptor of the remote Management Server. The service shall be confirmed by the remote Management Server containing the Device Descriptor information.

The local Application Layer shall accept the service request and shall pass it with a T_Data_Individual.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Individual.req primitive, the TSDU shall be an A_DeviceDescriptor_Read-PDU.

The remote Application Layer shall map a T_Data_Individual.ind primitive with TSDU = A_DeviceDescriptor_Read-PDU to an A_DeviceDescriptor_Read.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_DeviceDescriptor_Read.ind primitive.

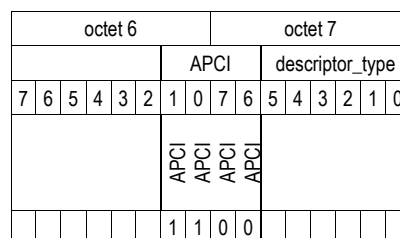


Figure 36 - A_DeviceDescriptor_Read-PDU (example)

The remote Management Server shall respond to the A_DeviceDescriptor_Read.ind primitive with an A_DeviceDescriptor_Read.res primitive containing the Device Descriptor information.

In this, the following shall apply.

- If Security Mode is not enabled in the MaS then the plain handling shall apply.
 - if the descriptor_type = 0 the Management Server shall respond with DD0 if DD0 is supported. If DD0 is not supported by the Management Server then it shall respond with the lowest supported Descriptor Type and corresponding data.
 - if the descriptor_type $\neq$ 0 the Management Server shall respond with the corresponding Descriptor Type and data.
 - if the descriptor_type $\neq$ 0 and this Descriptor Type is not supported by the Management Server, then the Management Server shall respond with the Descriptor Type 3Fh (error) without any data.

This definition is independent of the medium used and whether connection-oriented - or connectionless communication is used.

- If Security Mode is enabled in the MaS then the secure handling shall apply.
 - If the **requested descriptor_type = 0** then the following shall apply.
 - ▲ If the communication partner **does** have the necessary Permissions according to the Access Policies of the Device Descriptor Type, then the MaS shall respond with the descriptor_type = 0 and its Descriptor Type 0 value.
 - ▲ If the communication partner **does not** have the necessary Permissions according to the Access Policies of the Device Descriptor Type, then the MaS shall respond with the descriptor_type = 0 and Descriptor Type 0 value FFFFh.
 - If the **requested descriptor_type $\neq$ 0** then the following shall apply.
 - ▲ If the requested descriptor_type **is supported**, then the following shall apply.
 - If the communication partner **does** have the necessary Permissions according to the Access Policies of the Device Descriptor Type, then the MaS shall respond with the descriptor_type as requested and the corresponding Descriptor Type value.
 - If the communication partner **does not** have the necessary Permissions according to the Access Policies of the Device Descriptor Type, then the MaS shall respond with the descriptor_type = 3Fh without any data.

This is summarized in Figure 37.

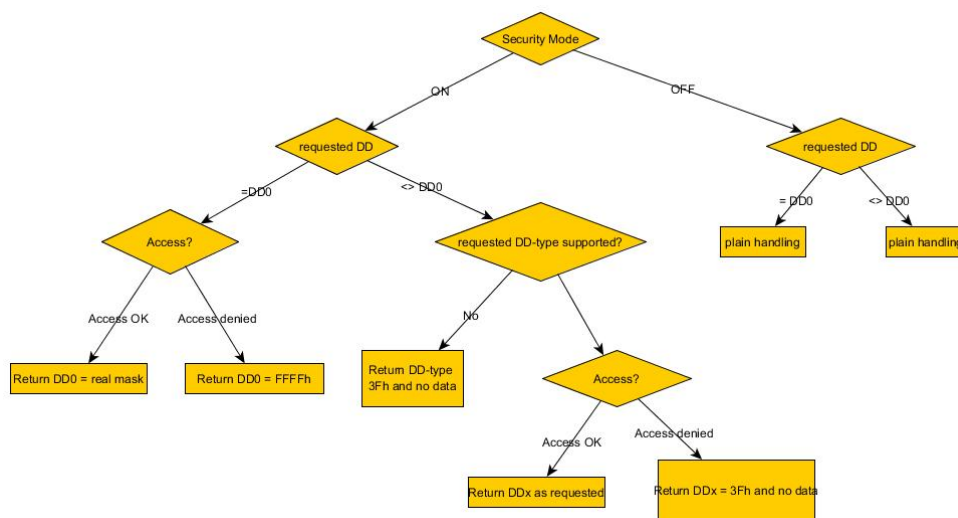


Figure 37 – Permissions and error handling for A_DeviceDescriptor_Read

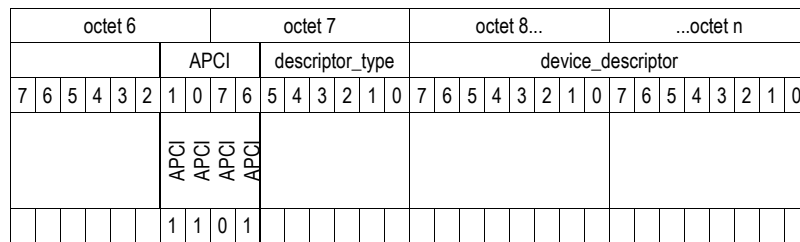


Figure 38 - A_DeviceDescriptor_Response-PDU (example)

The remote Application Layer shall accept the service response and shall pass it with a T_Data_Individual.req to the local Transport Layer. The parameters TSAP and priority shall be mapped to the corresponding parameters of the T_Data_Individual.req primitive, the TSDU shall be an A_DeviceDescriptor_Response-PDU.

The local Application Layer shall map a T_Data_Individual.ind primitive with TSDU = A_DeviceDescriptor_Response-PDU to an A_DeviceDescriptor_Read.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_DeviceDescriptor_Read.Acon primitive. For definition of the Device Descriptors see [04].

A_DeviceDescriptor_Read.req(ack_request, priority, hop_count_type, ASAP, descriptor_type)

ack_request: Data Link Layer Acknowledge requested or don't care
 priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 ASAP: local reference of the Service Access Point
 descriptor_type: type of the Device Descriptor

A_DeviceDescriptor_Read.Lcon(ack_request, priority, hop_count_type, ASAP, descriptor_type, a_status)

ack_request: Data Link Layer Acknowledge requested or don't care
 priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 ASAP: local reference of the Service Access Point
 descriptor_type: type of the Device Descriptor
 a_status: ok: A_DeviceDescriptor_Read-PDU sent successfully with T_Data_Individual service
 not_ok: transmission of the associated T_Data_Connected request frame did not succeed

A_DeviceDescriptor_Read.ind(priority, hop_count_type, ASAP, descriptor_type)

priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 ASAP: local reference of the Service Access Point
 descriptor_type: type of the Device Descriptor

A_DeviceDescriptor_Read.res(ack_request, priority, hop_count_type, ASAP, descriptor_type, device_descriptor)

ack_request: Data Link Layer Acknowledge requested or don't care
 priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 ASAP: local reference of the Service Access Point
 descriptor_type: type of the Device Descriptor
 device_descriptor: the Device descriptor of the communication controller

A_DeviceDescriptor_Read.Acon(priority, hop_count_type, ASAP, descriptor_type, device_descriptor)

priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 ASAP: local reference of the Service Access Point
 descriptor_type: type of the Device Descriptor
 device_descriptor: the Device Descriptor of the communication controller

3.4.2.2 A_Restart-service

The following service can be applied on the point-to-point connectionless communication mode as well as on the point-to-point connection-oriented communication mode. The following clause specifies the mapping of A_Restart on the point-to-point connectionless communication mode. For using this service on a connection-oriented communication mode T_Data_Connected of Transport Layer shall be applied Instead, of T_Data_Individual.

- For the connectionless communication mode the ASAP parameter shall be the Individual Address of the communication partner.
- For the connection-oriented communication mode the ASAP parameter shall be the identifier of the communication relationship.

The local Application Layer user shall apply the A_Restart.req service primitive with service parameter restart_type encoded as follows.

0: To request a Basic Restart of the communication partner.

The A_Restart-PDU shall contain no further data.

1: To request a Master Reset of the communication partner.

The A_Restart-PDU shall contain the fields Erase Code and Channel Number.

NOTE 8 The Erase Codes allows differentiating a Basic Restart and a Master Reset, and within the Master Reset multiple types, to control which Resources shall be reset. This is specified in the cause DM_Restart in [05].

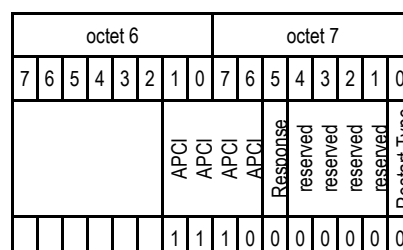


Figure 39 - A_Restart-PDU (example with restart_type = 0)

octet 6								octet 7								octet 8								octet 9							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
						APCI APCI APCI APCI				Response	reserved	reserved	reserved	reserved	Restart Type	Erase Code						Channel Number									
						1	1	1	0	0	0	0	0	0	1																

Figure 40 - A_Restart-PDU (example with restart_type = 1)

The local Application Layer shall accept the service request and pass it with a T_Data_Individual.req (T_Data_Connected.req) to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Individual.req (T_Data_Connected.req) primitive, the TSDU shall be an A_Restart-PDU.

The remote Application Layer shall map a T_Data_Individual.ind (T_Data_Connected.ind) primitive with TSDU = A_Restart-PDU to an A_Restart.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_Restart.ind primitive.

If the remote Application Layer accepts the service and the restart_type indicates a Basic Restart then the remote application process shall not confirm the service.

If the service is not confirmed by the remote application process, then

- for the connectionless communication mode the local Application Layer confirmation shall be caused by the L2-acknowledge of the local Data Link Layer, and
- for the connection-oriented communication mode the local Application Layer confirmation shall be caused by the confirmation of the local Transport Layer.

If the restart_type equals a Master Reset then the remote Application Process shall respond to the A_Restart_ind primitive with an A_Restart.res primitive containing the Error Code and the Process Time. The Process Time shall be encoded as specified in [05] clause “DM_Restart” under “Timing (Management Client and Management Server)”.

octet 6								octet 7								octet 8								octet 9								octet 10							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
						APCI	APCI	APCI	APCI	Response reserved reserved reserved reserved	Restart Type	Error Code								Process Time																			
						1	1	1	0	1	0	0	0	0	1																								

Figure 41 - A_Restart_Response-PDU (example)

The remote Application Layer shall accept the service response and pass it with a T_Data_Individual.req (T_Data_Connect.req) to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters TSAP and priority of the T_Data_Individual.req primitive, the TSDU shall be an A_Restart_Response-PDU.

The local Application Layer shall map a T_Data_Individual.ind (T_Data_Connected.ind) primitive with TSDU = A_Restart_Response-PDU to an A_Restart.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_Restart.Acon primitive.

A_Restart.req(ack_request, channel_number, erase_code, priority, hop_count_type, restart_type, ASAP)

ack_request	Data Link Layer acknowledge requested or don't care
channel_number	number of the application channel that shall be reset or 00h
erase_code	indication of Resources that shall be reset prior to resetting the device
priority:	system, urgent, normal or low priority
hop_count_type	standard NL hop count value
restart_type	Basic Restart or Master Reset
ASAP:	local reference of the service access point or Individual Address

A_Restart.ind(erase_code, channel_number, priority, hop_count_type, ASAP)

channel_number	number of the application channel that shall be reset or 00h
erase_code	indication of Resources that shall be reset prior to resetting the device
priority:	system, urgent, normal or low priority
hop_count_type	standard NL hop count value
restart_type	Basic Restart or Master Reset
ASAP:	local reference of the service access point or Individual Address

A_Restart.res(error_code, priority, process time, hop_count_type, ASAP)

error_code	indication about success or failure of the Master Reset request
priority:	system, urgent, normal or low priority
process time:	worst case time required by the remote Application Layer user needs for the execution of the requested Master Reset
hop_count_type	standard NL hop count value
restart_type	Basic Restart or Master Reset
ASAP:	local reference of the service access point or Individual Address

Error and exception handling

The connectionless usage of A_Restart simplifies the error handling of the restart procedure.

In case the local Application Layer user (Management Client) applies the A_Restart (Basic Restart or Master Reset) using the connection-oriented communication mode, it shall take into account the following.

- The Management Server (device) may or may not break down the Transport Layer connection. If the Management Server breaks down the Transport Layer connection, it may or may not send a T_Disconnect-PDU to the Management Server.

The Management Client shall set the value of the reserved fields in octet 7 to 0. The Management Server (device) shall check these fields to be 0; if these fields differ from 0 then it shall ignore the service totally. (No negative response shall be sent.)

If the Management Server (device) receives an A_Restart-PDU with restart_type = Master Reset and with a value of the Erase Code that is reserved or that it does not support, then it shall respond with an A_Restart_Response-PDU with Error Code = "Unsupported Erase Code".

If the Management Server (device) receives an A_Restart-PDU with restart_type = Master Reset and with a value of the Channel Number that is not valid, then it shall respond with an A_Restart_Response-PDU with Error Code = “Invalid Channel Number”. The Channel Number shall be invalid in the following cases.

1. The Channel Number is not 00h for an Erase Code for which the Channel Number shall be 00h according to “Definition of Erase Code and Channel Number” in [05] clause “DM_Restart”.
2. The Channel Number is not 00h but the device does not support application channels.
3. The Channel Number is not 00h and the device does support application Channels but the device does not have an application channel with the requested Channel Number.

3.4.2.3 A_FileStream_InfoReport

The A_FileStream_InfoReport.req primitive shall be applied by the user of Application Layer to send one file block to the remote partner. The file of the File Server shall be identified by the file handle and the file block shall be identified by the file block sequence number. The APCI shall be the APCI_FileStream_InfoReport. The communication partner shall be addressed with a local ASAP that shall be mapped to an Individual Address by the Transport Layer. This service shall not be confirmed.

The local Application Layer shall accept the service request and shall pass it with a T_Data_Individual.req to the local Transport Layer. The parameter ASAP shall be mapped to the corresponding ASAP of the T_Data_Individual.req primitive. The priority shall be set to low. The TSDU shall be an A_FileStream_InfoReport-PDU.

The remote Application Layer shall map a T_Data_Individual.ind primitive with TSDU = A_FileStream_InfoReport-PDU to an A_FileStream_InfoReport.ind primitive. The argument TSAP shall be mapped to the corresponding argument ASAP of the A_FileStream_InfoReport.ind primitive.

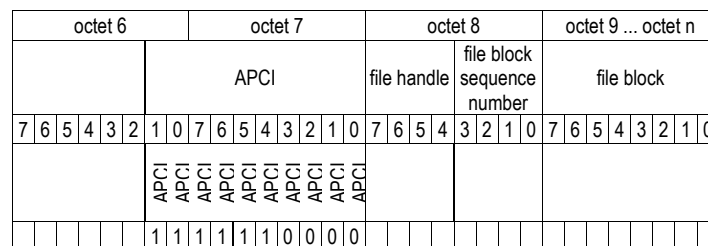


Figure 42 - A_FileStream_InfoReport-PDU (example)

A_FileStream_InfoReport.req(ack_request, ASAP, file_block, file_block_sequence_number, file_handle, hop_count_type)

ack_request:	Data Link Layer Acknowledge requested or don't care
ASAP:	local reference of the Service Access Point or Individual Address
file_block:	part of the file that shall be transferred to the communication partner
file_block_sequence_number:	rolling counter to denote the relative position of the file block in the stream transfer
file_handle:	reference to the file path of file stream to which the file block belongs
hop_count_type:	standard NL hop count value

A_FileStream_InfoReport.Lcon(ack_request, ASAP, file_block, file_block_sequence_number, file_handle, hop_count_type, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
file_block:	part of the file that shall be transferred to the communication partner
file_block_sequence_number:	rolling counter to denote the relative position of the file block in the stream transfer
file_handle:	reference to the file path of file stream to which the file block belongs
ASAP:	local reference of the Service Access Point or Individual Address
hop_count_type:	standard NL hop count value
a_status:	ok: A_FileStream_InfoReport-PDU is sent successfully with T_Data_Individual service
	not_ok: transmission of the associated T_Data_Individual request frame did not succeed

A_FileStream_InfoReport.ind(ASAP, file_block, file_block_sequence_number, file_handle, hop_count_type)

ASAP:	local reference of the Service Access Point or Individual Address
file_block:	part of the file that shall be transferred to the communication partner
file_block_sequence_number:	rolling counter to denote the relative position of the file block in the stream transfer
file_handle:	reference to the file path of file stream to which the file block belongs
hop_count_type:	standard NL hop count value

Error and exception handling

There is no Application Layer error handling specified for this service. This service is not confirmed.

3.4.3 Property Description services

3.4.3.1 A_PropertyDescription_Read-service

The A_PropertyDescription_Read.req primitive shall be applied by the user of Application Layer, to read the description of the Property of an Interface Object. The communication partner shall be addressed with a local ASAP that shall be mapped to an Individual Address by Transport Layer. The Interface Object of the partner shall be addressed with an object_index and the Property of the object shall be addressed with a property_id or with a property_index. The property_index shall be used only if the property_id is zero. The property_index, if evaluated, shall address the Property of the Interface Object with a sequential number, i.e. property_index = 0 shall mean the first Property of the associated Interface Object, property_index = 1 shall mean the second Property. The service shall be confirmed by the remote application process.

The local Application Layer shall accept the service request and shall pass it with a T_Data_Individual.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters TSAP and priority of the T_Data_Individual.req primitive, the TSDU shall be an A_PropertyDescription_Read-PDU.

The remote Application Layer shall map a T_Data_Individual.ind primitive with TSDU = A_PropertyDescription_Read-PDU to an A_PropertyDescription_Read.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_PropertyDescription_Read.ind primitive.

The application process shall respond to the A_PropertyDescription_Read.ind primitive with an A_PropertyDescription_Read.res primitive containing the description of the Property of the associated Property of the Interface Object addressed. The fields of the description of the Property shall be encoded as defined in [03].

If the property_id in the A_PropertyDescription_Read-PDU is zero, the remote application process shall use the indicated property_index to access the Property description. The property_index in the A_PropertyDescription_Response-PDU shall be the value of the field property_index of the received A_PropertyDescription_Read-PDU

If the property_id in the A_PropertyDescription_Read-PDU is not zero, then the field property_index shall be ignored; the remote application process shall use the indicated property_id to access the Property description. The property_index in the A_PropertyDescription_Response-PDU shall in this case be:

- the correct value of the Property index of the addressed Property, or
- the value of the field property_index of the received A_PropertyDescription_Read-PDU.

For new implementations the property_index shall contain the correct value of the addressed Property.

If the remote application process has a problem, e.g., Interface Object or Property does not exist, then the fields write_enable, type, max_nr_of_elem and access of the A_PropertyDescription_Response-PDU shall be zero.

The service shall not be confirmed negative for authorization reasons (see A_Authorize_Request-service).

octet 6						octet 7						octet 8						octet 9						octet 10							
						APCI						object_index						Property_id						property_index							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																		
						1	1	1	1	0	1	1	0	0	0																

Figure 29 - A_PropertyDescription_Read-PDU (example)

octet 6						octet 7						octet 8						octet 9						octet 10									
						APCI						object_index						property_id						property_index									
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0		
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																		
						1	1	1	1	0	1	1	0	0	1																		

octet 11								octet 12								octet 13								octet 14							
w		type										max_nr_of_elem								access											
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
	r							r	r	r	r									read_level				write_level							
	0							0	0	0	0																				

Figure 43 - A_PropertyDescription_Response-PDU (example)

The remote Application Layer shall accept the service response and shall pass it with a T_Data_Individual.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters TSAP and priority of the T_Data_Individual.req primitive, the TSDU shall be an A_PropertyDescription_Response-PDU.

Unused fields in the A_PropertyDescription_Response-PDU are marked with 'r' in below and shall be set to 0⁴⁾.

The local Application Layer shall map a T_Data_Individual.ind primitive with TSDU = A_PropertyDescription_Response-PDU to an A_PropertyDescription_Read.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_PropertyDescription_Read.Acon primitive.

A_PropertyDescription_Read.req(ack_request, priority, hop_count_type, ASAP, object_index, property_id, property_index)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_index:	the object_index of the object addressed
property_id:	the property_id of the Property of the object
property_index:	sequential Property number

A_PropertyDescription_Read.Lcon(ack_request, priority, hop_count_type, ASAP, object_index, property_id, property_index, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_index:	the object_index of the object addressed
property_id:	the property_id of the Property of the object
property_index:	sequential Property number
a_status:	ok: A_PropertyDescription_Read sent successfully with T_Data_Individual service
	not_ok: transmission of the associated T_Data_Individual request frame did not succeed

A_PropertyDescription_Read.ind(priority, hop_count_type, ASAP, object_index, property_id, property_index)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_index:	the object_index of the object addressed
property_id:	the property_id of the Property of the object
property_index:	sequential Property number

⁴⁾ Existing implementations may set bit 6 of octet 11 if the Property Value is an array; this is not allowed for new implementations. This bit shall not be evaluated by a management client; instead, the contents of the field max_nr_of_elem shall be used.

A_PropertyDescription_Read.res(ack_request, priority, hop_count_type, ASAP, object_index, property_id, property_index, write_enable, type, max_nr_of_elem, access)

ack_request: Data Link Layer Acknowledge requested or don't care

priority: system, urgent, normal or low priority

hop_count_type: standard NL hop count value

ASAP: local reference of the Service Access Point or Individual Address

object_index: the object_index of the object addressed

property_id: the property_id of the Property of the object

property_index: the sequential Property number of the addressed Property ⁵⁾

write_enable: specifies whether the Property value can be written or not

type: the Property DataType according to which the Property value is encoded

max_nr_of_elem: maximum number of elements of the array or zero to indicate a problem

access: access level to read or write to the Property value

A_PropertyDescription_Read.Acon(priority, hop_count_type, ASAP, object_index, property_id, property_index, write_enable, type, max_nr_of_elem, access)

priority: system, urgent, normal or low priority

hop_count_type: standard NL hop count value

ASAP: local reference of the Service Access Point or Individual Address

object_index: the object_index of the object addressed

property_id: the property_id of the Property of the object

property_index: the sequential Property number of the addressed Property ⁷⁾

write_enable: specifies whether the Property value can be written or not

type: the Property DataType according to which the Property value is encoded

max_nr_of_elem: maximum number of elements of the array or zero to indicate a problem

access: access level to read or write to the Property value

3.4.3.2 A_PropertyExtDescription_Read-service

The A_PropertyExtDescription_Read.req primitive shall be applied by the user of Application Layer, to read the description of the Property of an Interface Object. The communication partner shall be addressed with a local ASAP that shall be mapped to an Individual Address by the Transport Layer. The Interface Object of the partner shall be addressed with an Interface Object Type and an Object Instance and the Property of the Interface Object shall be addressed with a Property Identifier or with a Property Index. The Property Index shall be used only if the Property Identifier is zero. The Property Description Type shall be zero and is reserved for later extensions e.g., access control with Roles.

NOTE 9 The Format of the A_PropertyExtDescription_Read-PDU is fixed. Values of Property Description Type may be defined in the future. For the responses only the position of the "Property Description Type" field is fixed.

The Property Index, if evaluated, shall address the Property of the Interface Object with a sequential number: Property Index = 0 shall mean the first Property of the associated Interface Object, Property Index = 1 shall mean the second Property.

The local Application Layer shall accept the service request and shall pass it with a T_Data_Individual.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters TSAP and priority of the T_Data_Individual.req or T_Data_Connected.req primitive, the TSDU shall be an A_PropertyExtDescription_Read-PDU.

⁵⁾ Existing implementations may respond with the value of the field property_index contained in the A_PropertyDescription_Read-PDU. This is not allowed for new implementations.

The remote Application Layer shall map a T_Data_Individual.ind primitive with TSDU = A_PropertyExtDescription_Read-PDU to an A_PropertyExtDescription_Read.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_PropertyExtDescription_Read.ind primitive.

The application process shall respond to the A_PropertyExtDescription_Read.ind primitive with an A_PropertyExtDescription_Read.res primitive containing the description of the addressed Property. The fields of the description of the Property shall be encoded as defined in [03]. If no DPT is available, both DPT fields shall be set to zero.

- If the Property Identifier in the A_PropertyExtDescription_Read-PDU is zero then the remote application process shall use the indicated Property Index to access the Property description.
- If the Property Identifier in the A_PropertyExtDescription_Read-PDU is not zero, then the field Property Index shall be ignored (index should be zero) and the remote application process shall use the indicated Property Identifier to access the Property description.

The Property Index in the A_PropertyExtDescription_Response-PDU shall be the correct value of the Property Index of the addressed Property.

The Property Index shall contain the index of the addressed Property.

Error - and exception handling

- If the remote application process has a problem - e.g., Interface Object or Property does not exist - then the fields Property Description Type, DPT, writeable, PDT, max_nr_of_elem, and access of the A_PropertyExtDescription_Response-PDU shall be zero.
- If another Property Description Type than zero is requested, then the MaS shall answer with the A_PropertyExtDescription_Response-PDU of the Property Description Type zero.

The A_PropertyExtDescription_Read-service shall not be subject to authorisation. The remote application process shall not confirm the service negatively for authorisation reasons (see A_Authorize_Request-service).

octet 6						octet 7						octet 8						octet 9						octet 10						octet 11						octet 12											
						APCI						interface_object_type												object_instance						property_id																	
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0								
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																																
						0	1	1	1	0	1	0	0	1	0																																

octet 13								octet 14							
Property Description Type				property_index											
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
0	0	0	0												

Figure 44 - A_PropertyExtDescription_Read-PDU (example)

octet 6				octet 7				octet 8				octet 9				octet 10				octet 11				octet 12															
		APCI						interface_object_type								object_instance								property_id															
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
		APCI APCI APCI APCI APCI APCI APCI APCI																																					
						0	1	1	1	0	1	0	0	1	1																								

octet 13				octet 14				octet 15				octet 16				octet 17				octet 18																			
Property Description Type		property_index						DPT (main number)								DPT (subnumber)																							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
Type zero																																							
0	0	0	0																																				

octet 19				octet 20				octet 21				octet 22											
r				max_nr_of_elem								access											
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
writeable	reserved	PDT														read level		write level					
0																							

Figure 45 - A_PropertyExtDescription Response-PDU (type zero) (example)

The remote Application Layer shall accept the service response and shall pass it with a T_Data_Individual.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters TSAP and priority of the T_Data_Individual.req or the T_Data_Connected.req primitive; the TSDU shall be an A_PropertyExtDescription_Response-PDU.

Unused fields in the A_PropertyExtDescription_Response-PDU are marked with ‘r(erved)’ above and shall be set to 0.

The local Application Layer shall map a T_Data_Individual.ind or a T_Data_Individual.ind primitive with TSDU = A_PropertyExtDescription_Response-PDU to an A_PropertyExtDescription_Read.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_PropertyExtDescription_Read.Acon primitive.

```
A_PropertyExtDescription_Read.req(ack_request, priority, hop_count_type, ASAP, object_type,
                                   object_instance, property_id, property_index, type)
```

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
property_index:	sequential Property number
type:	the Type of the Property description, currently only type 0 defined

A_PropertyExtDescription_Read.Lcon(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, property_index, type, a_status)

ack_request:		Data Link Layer Acknowledge requested or don't care
priority:		system, urgent, normal or low priority
hop_count_type:		standard NL hop count value
ASAP:		local reference of the Service Access Point or Individual Address
object_type:		the Interface Object Type of the addressed Interface Object
object_instance:		the Object Instance of the addressed Interface Object
property_id:		the Property Identifier of the Property of the addressed Interface Object
property_index:		sequential Property number
type:		the Type of the Property description, currently only type 0 defined
a_status:	ok:	A_PropertyExtDescription_Read sent successfully with T_Data_Individual service
	not_ok:	transmission of the associated T_Data_Individual request frame did not succeed

A_PropertyExtDescription_Read.ind(priority, hop_count_type, ASAP, object_type, object_instance, property_id, property_index, type)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
property_index:	sequential Property number
type:	the Type of the Property description, currently only type 0 defined

A_PropertyExtDescription_Read.res(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, property_index, type, dpt, write_enable, pdt, max_nr_of_elem, access)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
property_index:	sequential Property number
type:	the Type of the Property description, currently only type 0 defined
dpt:	the Datapoint Type according to which the Property Value is encoded
write_enable:	specifies whether the Property value can be written or not
pdt:	the Property Datatype according to which the Property value is encoded
max_nr_of_elem:	maximum number of elements of the array or zero to indicate a problem
access:	access level to read or write to the Property value

A_PropertyExtDescription_Read.Acon(priority, hop_count_type, ASAP, object_type, object_instance, property_id, property_index, type, DPT, write_enable, PDT, max_nr_of_elem, access)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
property_index:	sequential Property number
type:	the Type of the Property description, currently only type 0 defined
DPT:	the Datapoint Type according to which the Property Value is encoded
write_enable:	specifies whether the Property value can be written or not
PDT:	the Property Datatype according to which the Property value is encoded
max_nr_of_elem:	maximum number of elements of the array or zero to indicate a problem
access:	access level to read or write to the Property value

3.4.4 Data Property services

3.4.4.1 A_PropertyValue_Read-service

The A_PropertyValue_Read.req primitive shall be applied by the user of Application Layer to read the value of a Property of an Interface Object. The communication partner shall be addressed with a local ASAP that shall be mapped to an Individual Address by Transport Layer. The Interface Object of the partner shall be addressed with an object_index and the Property of the Interface Object shall be addressed with a property_id. The nr_of_elem and start_index shall indicate the number of array elements starting with the given start_index in the Property value that the user wants to read. The user of Application Layer in the partner device shall respond with an A_PropertyValue_Read.res, i.e. the service shall be confirmed by the remote application process.

The local Application Layer shall accept the service request and pass it with a T_Data_Individual.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters ASAP and priority of the T_Data_Individual.req primitive, the TSDU shall be an A_PropertyValue_Read-PDU.

The remote Application Layer shall map a T_Data_Individual.ind primitive with TSDU = A_PropertyValue_Read-PDU to an A_PropertyValue_Read.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_PropertyValue_Read.ind primitive.

The application process shall respond to the A_PropertyValue_Read.ind primitive with an A_PropertyValue_Read.res primitive containing the requested number of elements of the Property value of the Property of the associated Interface Object. If the remote application process has a problem, e.g., object or Property does not exist or the data does not fit in a PDU or the requester has not the required access rights, then the nr_of_elem of the A_PropertyValue_Response-PDU shall be zero and shall contain no data.

octet 6						octet 7						octet 8						octet 9						octet 10						octet 11											
						APCI						object_index						property_id						nr_of_elem		start_index															
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0		
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																										
						1	1	1	1	0	1	0	1	0	1																										

Figure 46 - A_PropertyValue_Read-PDU (example)

octet 6						octet 7						octet 8						octet 9						octet 10						octet 11						octet 12 ... octet n												
						APCI						object_index						property_id						nr_of - elem		start_index						data																
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																																	
						1	1	1	1	0	1	0	1	1	0																																	

Figure 47 - A_PropertyValue_Response-PDU (example)

The remote Application Layer shall accept the service response and pass it with a T_Data_Individual.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters TSAP and priority of the T_Data_Individual.req primitive, the TSDU is an A_PropertyValue_Response-PDU.

The local Application Layer shall map a T_Data_Individual.ind primitive with TSDU = A_PropertyValue_Response-PDU to an A_PropertyValue_Read.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_PropertyValue_Read.Acon primitive.

Error and exception handling

If the remote application process receives an A_PropertyValue_Read-PDU with start_index = 0, this is, reading the current number of elements of the Property Value array, but with a nr_of_elem greater than 1, then it shall respond with an A_PropertyValue_Response-PDU with start_index = 0 and nr_of_elem = 1 and the field data shall contain the current number of elements of the Property array.

A_PropertyValue_Read.req(ack_request, priority, hop_count_type, ASAP, object_index, property_id, nr_of_elem, start_index)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_index:	the object_index of the object addressed
property_id:	the Property_id of the Property of the object
nr_of_elem:	the number of array elements to be read in the Property value
start_index:	the array index of the first array element to be read

A_PropertyValue_Read.Lcon(ack_request, priority, hop_count_type, ASAP, object_index, property_id, nr_of_elem, start_index, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_index:	the object_index of the object addressed
property_id:	the property_id of the Property of the object addressed
nr_of_elem:	the number of array elements to be read in the Property value
start_index:	the array index of the first array element to be read
a_status:	ok: A_PropertyValue_Read sent successfully with T_Data_Individual service
	not_ok: transmission of the associated T_Data_Individual request frame did not succeed

A_PropertyValue_Read.ind(priority, hop_count_type, ASAP, object_index, property_id, nr_of_elem, start_index)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_index:	the object_index of the object addressed
property_id:	the property_id of the Property of the object addressed
nr_of_elem:	the number of array elements to be read in the Property value
start_index:	the array index of the first array element to be read

A_PropertyValue_Read.res(ack_request, priority, hop_count_type, ASAP, object_index, property_id, nr_of_elem, start_index, data)

ack_request: Data Link Layer Acknowledge requested or don't care

priority: system, urgent, normal or low priority

hop_count_type: standard NL hop count value

ASAP: local reference of the Service Access Point or Individual Address

object_index: the object_index of the object addressed

property_id: the property_id of the Property of the object addressed

nr_of_elem: the number of array elements read in the Property value or zero if problem occurred

start_index: the array index of the first array element read

data: the value of the array elements read, or no data, if a problem occurred

A_PropertyValue_Read.Acon(priority, hop_count_type, ASAP, object_index, property_id, nr_of_elem, start_index, data)

priority: system, urgent, normal or low priority

hop_count_type: standard NL hop count value

ASAP: local reference of the Service Access Point or Individual Address

object_index: the object_index of the object addressed

property_id: the property_id of the Property of the object addressed

nr_of_elem: the number of array elements read in the Property value or zero if problem occurred

start_index: the array index of the first array element read

data: the value of the array elements read, or no data, if a problem occurred

3.4.4.2 A_PropertyValue_Write-service

The A_PropertyValue_Write.req primitive shall be applied by the user of Application Layer to modify the value of a Property of an Interface Object. The communication partner shall be addressed with a local ASAP that shall be mapped to an Individual Address by Transport Layer. The Interface Object of the partner shall be addressed with the object_index and the Property of the Interface Object shall be addressed with the property_id. The nr_of_elem and start_index shall indicate the number of array elements starting with the given start_index in the Property value that the user wants to write to.

The local Application Layer shall accept the service request and pass it with a T_Data_Individual.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters TSAP and priority of the T_Data_Individual.req primitive, the TSDU shall be an A_PropertyValue_Write-PDU.

The remote Application Layer shall map a T_Data_Individual.ind primitive with TSDU = A_PropertyValue_Write-PDU to an A_PropertyValue_Write.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_PropertyValue_Write.ind primitive.

[illegible]

`A_PropertyValue_Write.Lcon(ack_request, priority, hop_count_type, ASAP, object_index, property_id, nr_of_elem, start_index, data, a_status)`

`ack_request`: Data Link Layer Acknowledge requested or don't care

`priority`: system, urgent, normal or low priority

`hop_count_type`: standard NL hop count value

`ASAP`: local reference of the Service Access Point or Individual Address

`object_index`: the `object_index` of the object addressed

`property_id`: the `property_id` of the Property of the object addressed

`nr_of_elem`: the number of array elements to be written in the Property value

`start_index`: the array index of the first array element to be written

`data`: the data to write to the array elements

`a_status`: `ok`: `A_PropertyValue_Write` sent successfully with `T_Data_Individual` service
 `not_ok`: transmission of the associated `T_Data_Individual` request frame did not succeed

`A_PropertyValue_Write.ind(priority, hop_count_type, ASAP, object_index, property_id, nr_of_elem, start_index, data)`

`priority`: system, urgent, normal or low priority

`hop_count_type`: standard NL hop count value

`ASAP`: local reference of the Service Access Point or Individual Address

`object_index`: the `object_index` of the object addressed

`property_id`: the `property_id` of the Property of the object addressed

`nr_of_elem`: the number of array elements to be written in the Property value

`start_index`: the array index of the first array element to be written

`data`: the data to write to the array elements

3.4.5 Data Property Extended services

3.4.5.1 A_PropertyExtValue_Read

The `A_PropertyExtValue_Read.req` primitive shall be applied by the user of Application Layer to read the value of a Property of an Interface Object. The communication partner shall be addressed with a local ASAP that shall be mapped to an Individual Address by the Transport Layer. The Interface Object of the partner shall be addressed with an Interface Object Type and the Object Instance and the Property of the Interface Object shall be addressed with a Property Identifier. The `nr_of_elem` and `start_index` shall indicate the number of array elements starting with the given `start_index` in the Property value that the user wants to read. The user of Application Layer in the partner device shall respond with an `A_PropertyExtValue_Read.res`, this is, the service shall be confirmed by the remote application process.

The local Application Layer shall accept the service request and pass it with a `T_Data_Individual.req` to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters ASAP and priority of the `T_Data_Individual.req` or the `T_Data_Connected.req` primitive, the TSDU shall be an `A_PropertyExtValue_Read-PDU`.

The remote Application Layer shall map a `T_Data_Individual.ind` primitive with TSDU = `A_PropertyExtValue_Read-PDU` to an `A_PropertyExtValue_Read.ind` primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the `A_PropertyExtValue_Read.ind` primitive.

The application process shall respond to the A_PropertyExtValue_Read.ind primitive with an A_PropertyExtValue_Read.res primitive containing the requested number of elements of the Property value of the Property of the associated Interface Object.

NOTE 11 For the Object Instance more than one octet is required.

[illegible]

octet 13								octet 14								octet 15							
nr_of_elem								start_index															
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0

Figure 49 - A PropertyExtValue Read-PDU (example)

[illegible]

octet 13								octet 14								octet 15								octet 16 ... octet n							
nr_of_elem								start_index								data															
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0

Figure 50 - A PropertyExtValue Response-PDU (example)

The remote Application Layer shall accept the service response and pass it with a T_Data_Individual.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters TSAP and priority of the T_Data_Individual.req or the T_Data_Connected.req primitive, the TSDU shall be an A_PropertyExtValue Response-PDU.

The local Application Layer shall map a T_Data_Individual.ind primitive with TSDU = A_PropertyExt-Value_Response-PDU to an A_PropertyExtValue_Read.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_PropertyExtValue_Read.Acon primitive.

Error and exception handling

- If the MaS has a problem, e.g., Interface Object or Property does not exist, then the nr_of_elem shall be set to zero and the start_index of the response shall be set to same value as received with the request. The data field of a negative response shall contain error information. The error information of a negative confirmation shall be a one octet long enumerated data field. For more details concerning the error handling and the Error Code Set please refer to clause 3.4.5.5 “Return Codes”.

- If the remote application process receives an A_PropertyExtValue_Read-PDU with start_index = 0, this is, reading the current number of elements of the Property Value array, but with a nr_of_elem greater than 1, then it shall respond with an A_PropertyExtValue_Response-PDU with start_index = 0 and nr_of_elem = 1 and the field data shall contain the current number of elements of the Property array (2 octet value).

A_PropertyExtValue_Read.req(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, nr_of_elem, start_index)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
nr_of_elem:	the number of array elements to be read in the Property value
start_index:	the array index of the first array element to be read

A_PropertyExtValue_Read.Lcon(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, nr_of_elem, start_index, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
nr_of_elem:	the number of array elements to be read in the Property value
start_index:	the array index of the first array element to be read
a_status:	ok: A_PropertyExtValue_Read sent successfully with T_Data_Individual service not_ok: transmission of the associated T_Data_Individual request frame did not succeed

A_PropertyExtValue_Read.ind(priority, hop_count_type, ASAP, object_type, object_instance, property_id, nr_of_elem, start_index)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
nr_of_elem:	the number of array elements to be read in the Property value
start_index:	the array index of the first array element to be read

`A_PropertyExtValue_Read.res(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, nr_of_elem, start_index, data)`

<code>ack_request:</code>	Data Link Layer Acknowledge requested or don't care
<code>priority:</code>	system, urgent, normal or low priority
<code>hop_count_type:</code>	standard NL hop count value
<code>ASAP:</code>	local reference of the Service Access Point or Individual Address
<code>object_type:</code>	the Interface Object Type of the addressed Interface Object
<code>object_instance:</code>	the Object Instance of the addressed Interface Object
<code>property_id:</code>	the Property Identifier of the Property of the addressed Interface Object
<code>nr_of_elem:</code>	the number of array elements to be read in the Property value
<code>start_index:</code>	the array index of the first array element to be read
<code>data:</code>	the value of the array elements read, or an error code if an error occurred

`A_PropertyExtValue_Read.Acon(priority, hop_count_type, ASAP, object_type, object_instance, property_id, nr_of_elem, start_index, data)`

<code>priority:</code>	system, urgent, normal or low priority
<code>hop_count_type:</code>	standard NL hop count value
<code>ASAP:</code>	local reference of the Service Access Point or Individual Address
<code>object_type:</code>	the Interface Object Type of the addressed Interface Object
<code>object_instance:</code>	the Object Instance of the addressed Interface Object
<code>property_id:</code>	the Property Identifier of the Property of the addressed Interface Object
<code>nr_of_elem:</code>	the number of array elements to be read in the Property value
<code>start_index:</code>	the array index of the first array element to be read
<code>data:</code>	the value of the array elements read, or an error code

3.4.5.2 A_PropertyExtValue_WriteCon-service

The `A_PropertyExtValue_WriteCon.req` primitive shall be applied by the user of Application Layer to modify the value of a Property of an Interface Object. The communication partner shall be addressed with a local ASAP that shall be mapped to an Individual Address by the Transport Layer. The Interface Object of the partner shall be addressed with the Interface Object Type and the Object Instance and the Property of the Interface Object shall be addressed with the Property Identifier. The `nr_of_elem` and `start_index` shall indicate the number of array elements starting with the given `start_index` in the Property value that the user wants to write to.

The local Application Layer shall accept the service request and pass it with a `T_Data_Individual.req` to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters TSAP and priority of the `T_Data_Individual.req` or the `T_Data_Connected.req` primitive, the TSDU shall be an `A_PropertyExtValue_WriteCon-PDU`.

The remote Application Layer shall map a `T_Data_Individual.ind` primitive with TSDU = `A_PropertyExtValue_WriteCon-PDU` to an `A_PropertyExtValue_WriteCon.ind` primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the `A_PropertyExtValue_WriteCon.ind` primitive.

octet 6								octet 7								octet 8								octet 9								octet 10								octet 11								octet 12								
								APCI								interface_object_type																object_instance								property_id																
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0									
								APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																																								
						0	1	1	1	0	0	1	1	1	0																																									

octet 13								octet 14								octet 15								octet 16 ... octet n																					
nr_of_elem								start_index								data																													
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0														

Figure 51 - A PropertyExtValue_WriteCon-PDU (example)

octet 6								octet 7								octet 8								octet 9								octet 10								octet 11								octet 12							
		APCI						interface_object_type																object_instance																property_id															
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0								
		APCI APCI APCI APCI APCI APCI APCI APCI																																																					
						0	1	1	1	0	0	1	1	1	1																																								

octet 13								octet 14								octet 15								octet 16							
nr_of_elem								start_index																return_code							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0

Figure 52 - A_PropertyExtValue_WriteConRes-PDU (example)

The remote application process shall respond to the A_PropertyExtValue_WriteCon.ind primitive with an A_PropertyExtValue_WriteCon.res primitive containing the requested number of elements of the Property value of the Property of the associated Interface Object. The return_code shall contain the appropriate Return Code on success or failure as specified in clause 3.4.5.5 “Return Codes”.

Error and exception handling

- If the MaS has a problem, e.g., Interface Object or Property does not exist, then the nr_of_elem shall be set to zero and the start_index of the response shall be set to same value as received with the request. The return_code field of a negative response shall contain error information. The error information of a negative confirmation shall be a one octet long enumerated data field. For more details concerning the error handling and the Error Code Set please refer to clause 3.4.5.5 “Return Codes”.

The remote Application Layer shall accept the service response and shall pass it with a T_Data_Individual.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters TSAP and priority of the T_Data_Individual.req or the T_Data_Connected.req primitive, the TSDU shall be an A_PropertyExtValue WriteConRes-PDU.

The local Application Layer shall map a T_Data_Individual.ind primitive with TSDU = A_PropertyExtValue_WriteConRes-PDU to an A_PropertyExtValue_WriteCon.Acon primitive if an A_PropertyExtValue_WriteCon-PDU is sent before to this communication partner to this Interface Object and Property. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_PropertyExtValue_WriteCon.Acon primitive.

A_PropertyExtValue_WriteCon.req(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, nr_of_elem, start_index, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
nr_of_elem:	the number of array elements to be written in the Property value
start_index:	the array index of the first array element to be written
data:	the data to write to the array elements

A_PropertyExtValue_WriteCon.Lcon(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, nr_of_elem, start_index, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
nr_of_elem:	the number of array elements to be written in the Property value
start_index:	the array index of the first array element to be written
data:	the data to write to the array elements
a_status:	ok: A_PropertyExtValue_WriteCon sent successfully with T_Data_Individual service
	not_ok: transmission of the associated T_Data_Individual request frame did not succeed

A_PropertyExtValue_WriteCon.ind(priority, hop_count_type, ASAP, object_type, object_instance, property_id, nr_of_elem, start_index, data)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
nr_of_elem:	the number of array elements to be written in the Property value
start_index:	the array index of the first array element to be written
data:	the data to write to the array elements

`A_PropertyExtValue_WriteCon.res(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, nr_of_elem, start_index, a_result)`

<code>ack_request:</code>	Data Link Layer Acknowledge requested or don't care
<code>priority:</code>	system, urgent, normal or low priority
<code>hop_count_type:</code>	standard NL hop count value
<code>ASAP:</code>	local reference of the Service Access Point or Individual Address
<code>object_type:</code>	the Interface Object Type of the addressed Interface Object
<code>object_instance:</code>	the Object Instance of the addressed Interface Object
<code>property_id:</code>	the Property Identifier of the Property of the addressed Interface Object
<code>nr_of_elem:</code>	the number of array elements to be written in the Property value
<code>start_index:</code>	the array index of the first array element to be written
<code>return_code:</code>	the return code of the operation

`A_PropertyExtValue_WriteCon.Acon(priority, hop_count_type, ASAP, object_type, object_instance, property_id, nr_of_elem, start_index, a_result)`

<code>priority:</code>	system, urgent, normal or low priority
<code>hop_count_type:</code>	standard NL hop count value
<code>ASAP:</code>	local reference of the Service Access Point or Individual Address
<code>object_type:</code>	the Interface Object Type of the addressed Interface Object
<code>object_instance:</code>	the Object Instance of the addressed Interface Object
<code>property_id:</code>	the Property Identifier of the Property of the addressed Interface Object
<code>nr_of_elem:</code>	the number of array elements to be written in the Property value
<code>start_index:</code>	the array index of the first array element to be written
<code>return_code:</code>	the return code of the operation

3.4.5.3 A_PropertyExtValue_WriteUnCon-service

The `A_PropertyExtValue_WriteUnCon.req` primitive shall be applied by the user of Application Layer to modify the value of a Property of an Interface Object. The communication partner shall be addressed with a local ASAP that shall be mapped to an Individual Address by the Transport Layer. The Interface Object of the partner shall be addressed with the Interface Object Type and the Object Instance and the Property of the Interface Object shall be addressed with the Property Identifier. The `nr_of_elem` and `start_index` shall indicate the number of array elements starting with the given `start_index` in the Property value that the user wants to write to.

The local Application Layer shall accept the service request and pass it with a `T_Data_Individual.req` to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters TSAP and priority of the `T_Data_Individual.req` or the `T_Data_Connected.req` primitive, the TSDU shall be an `A_PropertyExtValue_WriteUnCon-PDU`.

The remote Application Layer shall map a `T_Data_Individual.ind` primitive with TSDU = `A_PropertyExtValue_WriteUnCon-PDU` to an `A_PropertyExtValue_WriteUnCon.ind` primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the `A_PropertyExtValue_WriteUnCon.ind` primitive.

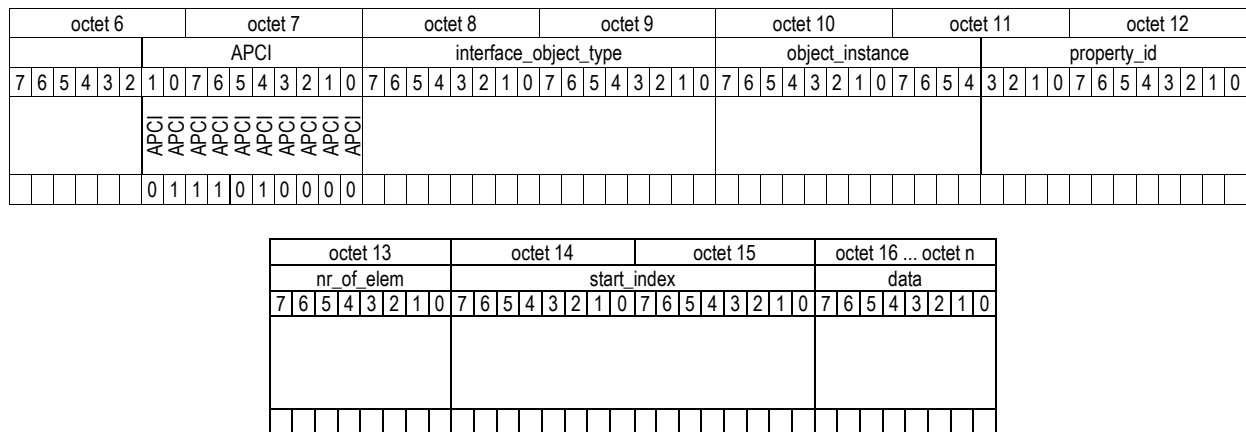


Figure 53 - A_PropertyExtValue_WriteUnCon-PDU (example)

The remote application process shall not respond to the A_PropertyExtValue_WriteUnCon.ind primitive.

Error and exception handling

- If the MaS has a problem - e.g., Interface Object or Property does not exist - then the request shall be ignored.

A_PropertyExtValue_WriteUnCon.req(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, nr_of_elem, start_index, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
nr_of_elem:	the number of array elements to be written in the Property value
start_index:	the array index of the first array element to be written
data:	the data to write to the array elements

A_PropertyExtValue_WriteUnCon.Lcon(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, nr_of_elem, start_index, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
nr_of_elem:	the number of array elements to be written in the Property value
start_index:	the array index of the first array element to be written
data:	the data to write to the array elements
a_status:	ok: A_PropertyExtValue_WriteUnCon sent successfully with T_Data_Individual service not_ok: transmission of the associated T_Data_Individual request frame did not succeed

A_PropertyExtValue_WriteUnCon.ind(priority, hop_count_type, ASAP, object_type, object_instance, property_id, nr_of_elem, start_index, data)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
nr_of_elem:	the number of array elements to be written in the Property value
start_index:	the array index of the first array element to be written
data:	the data to write to the array elements

3.4.5.4 A_PropertyExtValue_InfoReport-service

The A_PropertyExtValue_InfoReport.req primitive shall be applied by the user of Application Layer to report the value of a Property of an Interface Object. The communication partner shall be addressed with a local ASAP that shall be mapped to an Individual Address by the Transport Layer. The value reported shall be addressed with its Interface Object Type and its Object Instance, and the Property of the Interface Object shall be addressed with the Property Identifier. The nr_of_elem and start_index shall indicate the number of array elements starting with the given start_index in the Property value that the user wants to report.

The local Application Layer shall accept the service request and pass it with a T_Data_Individual.req or T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters TSAP and priority of the T_Data_Individual.req or the T_Data_Connected.req primitive, the TSDU shall be an A_PropertyExtValue_InfoReport-PDU.

The remote Application Layer shall map a T_Data_Individual.ind primitive with TSDU = A_PropertyExtValue_InfoReport-PDU to an A_PropertyExtValue_InfoReport.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_PropertyExtValue_InfoReport.ind primitive.

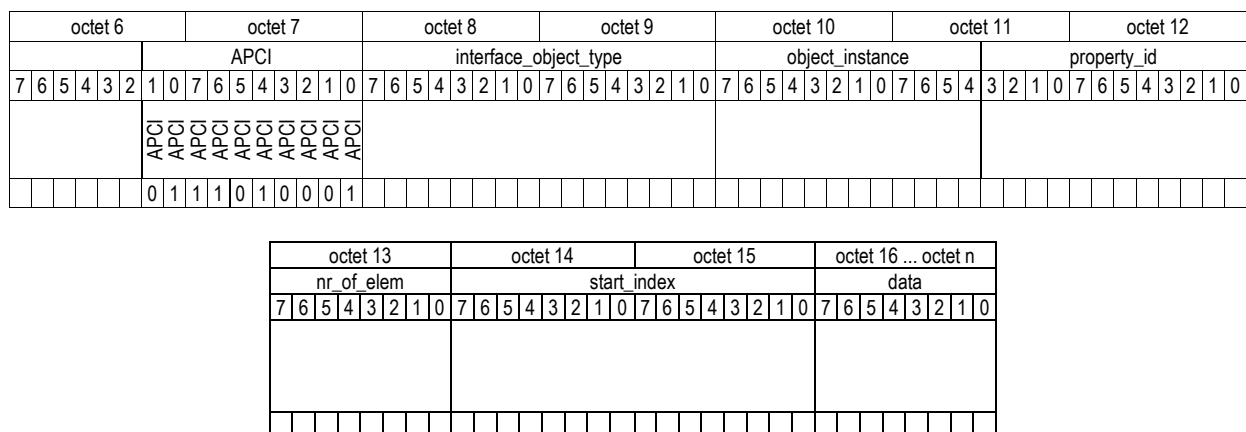


Figure 54 - A_PropertyExtValue_InfoReport-PDU (example)

The remote application process shall not respond to the A_PropertyExtValue_InfoReport.ind primitive.

Error and exception handling

- The application program ignores the data if the device is not a consumer of the Property.

A_PropertyExtValue_InfoReport.req(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, nr_of_elem, start_index, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
nr_of_elem:	the number of array elements to be reported in the Property value
start_index:	the array index of the first array element to be reported
data:	the data to report

A_PropertyExtValue_InfoReport.Lcon(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, nr_of_elem, start_index, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
nr_of_elem:	the number of array elements to be reported in the Property value
start_index:	the array index of the first array element to be reported
data:	the data to report
a_status:	ok: A_PropertyExtValue_InfoReport sent successfully with T_Data_Individual service
	not_ok: transmission of the associated T_Data_Individual request frame did not succeed

A_PropertyExtValue_InfoReport.ind(priority, hop_count_type, ASAP, object_type, object_instance, property_id, nr_of_elem, start_index, data)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
nr_of_elem:	the number of array elements to be reported in the Property value
start_index:	the array index of the first array element to be reported
data:	the data to report

3.4.5.5 Return Codes

Return Code	Error/Success Type	Name
Basic positive Return Code		
00h	Success	E_SUCCESS
	The service, function or command is executed successfully, without additional information.	
Generic positive Return Codes 01h-1Fh		
None proposed		
Specific positive Return Codes 20h-5Fh		
None proposed		
Specific negative Return Codes A0h-DFh		
None proposed		
Generic negative Return Codes E0h-FEh		
F1h	Memory Error	E_MEMORY_ERROR
	memory cannot be accessed or only with fault(s)	
F4h	Device Capabilities	E_LENGTH_EXCEEDS_MAX_APDU_LENGTH
	Requested data will not fit into a Frame supported by this server. This shall be used for Device limitations of the maximum supported Frame length by accessing resources (Properties, Function Properties, memory...) of the device.	
F5h	Data overflow	E_DATA_OVERFLOW
	This means that one wants to write data beyond what is reserved for the addressed Resource.	
F6h	Out of MinRange	E_DATA_MIN
	Write value too low. If the value is lower than the lowest supported value, then preferably this value shall be given Instead, of "Value not supported".	
F7h	Out of MaxRange	E_DATA_MAX
	Write value too high. If the value is higher than the highest supported value, then preferably this value shall be given Instead, of "Value not supported".	
F8h	void data	E_DATA_VOID
	This shall mean that the service or the function (Property) is supported, but the request data is not valid for this receiver. This value shall also be given if the requested data contains an enumeration value that is not supported, in between the supported ranges.	
F9h	Temporarily not available	E_TEMPORARILY_NOT_AVAILABLE
	This shall mean that the data could in generally be written, but that it is not possible at this time because • another MaC is accessing the data, or • the data is currently being processed by the MaS (e.g., being flashed, or being renewed...).	

Return Code	Error/Success Type	Name
FAh	Write Only	E_ACCESS_WRITE_ONLY
	This shall mean that a read access is attempted to a “write only” service or Resource. This shall be for Resources (Properties, Function Properties, memory...) etc. that can be written, but that cannot be read.	
FBh	Read Only	E_ACCESS_READ_ONLY
	This shall means that a write access is attempted to a “read only” service or Resource. This shall be for Resources (Properties, Function Properties, memory...) etc. that can be read, but that cannot be written.	
FCh	Access denied	E_ACCESS_DENIED
	This shall mean that the access to the data or function is denied because of authorisation reasons, A_Authorize as well as KNX Security.	
FDh	void address	E_ADDRESS_VOID
	The Interface Object or the Property is not present, or the index is out of range.	
FEh	Type Conflict	E_DATA_TYPE_CONFLICT
	Write access with a wrong datatype (Datapoint length)	
Basic Negative Return Code		
FFh	Error	E_ERROR
	The service, function or command has failed without a closer indication of the problem.	

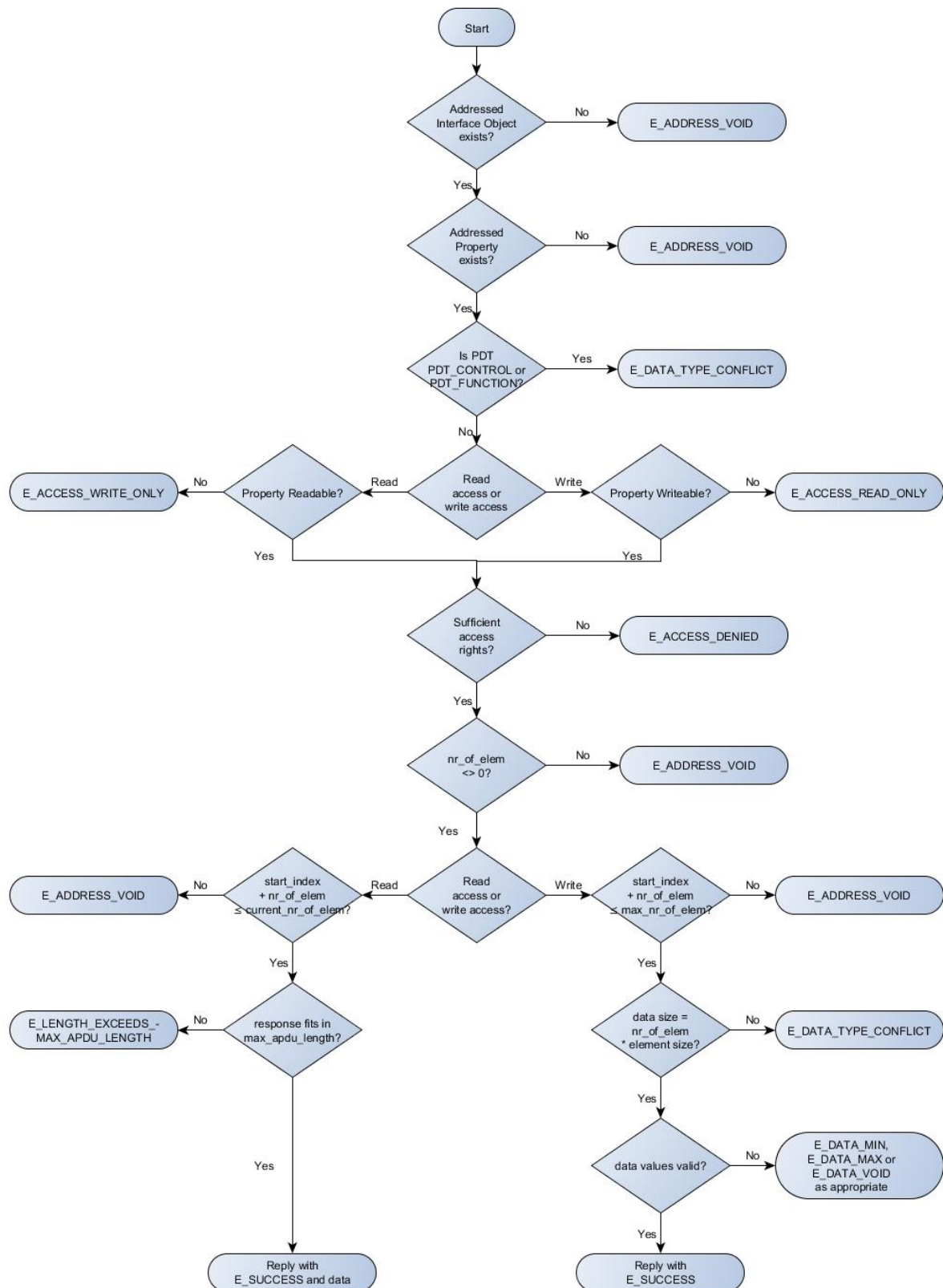


Figure 55 - Flow chart for Extended Data Property services

3.4.6.1 A_Link_Read-service

The A_Link_Read.req primitive shall be applied by the local Application Layer user to read the links to a given Group Object in a communication partner. The communication partner shall be addressed with a local ASAP that shall be mapped to an Individual Address by the Transport Layer. The Group Object of the communication partner shall be addressed with a Group Object Number. The index in the list of links from which the reading shall start shall be indicated with the start index. The remote Application Layer user in the communication partner shall respond with an A_Link_Read.res containing the requested link information; this is, the service shall be confirmed by the remote Application Layer.

The local Application Layer shall accept the service request and pass it with a T_Data_Individual.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Individual.req primitive, the TSDU shall be an A_Link_Read-PDU.

octet 6								octet 7								octet 8								octet 9							
								APCI								group_object_number								start_index							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
								APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI									-- -- --							
						1	1	1	1	1	0	0	1	0	1																

Figure 56 - A_Link_Read-PDU (example)

The remote Application Layer shall map a T_Data_Individual.ind primitive with TSDU = A_Link_Read-PDU to an A_Link_Read.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_Link_Read.ind primitive.

The remote management process shall respond to the A_Link_Read.ind primitive with an A_Link_Read.res primitive containing the current list of links on the addressed Group Object and the sending Group Address if any. The TSDU shall be an A_Link_Response-PDU.

- If more Group Addresses are assigned to the Group Object than what can be transported in the maximal frame length of the device, then the group_address_list may be limited to the first n Group Addresses association to this Group Object to build a maximal length A_Link_Response-PDU. For devices not supporting the L_Data_Extended frame format, this value n shall be 6.
- If less Group Addresses are assigned to the Group Object than what can be transported in the maximal frame length of the device, then the group_address_list shall only contain the assigned Group Addresses and the A_Link_Response-PDU shall be stopped after the last linked Group Address. The A_Link_Response-PDU may thus have a variable length.
- If the remote management process has a problem, e.g., addressed Group Object does not exist or no Group Addresses are assigned to the Group Object from the request start index, then it shall respond with a negative response.

Positive response

sending_address:	1 to 15, 0 means none.
start_index:	1 to 15
group_address_list:	The A_Link_Response-PDU can contain 0 ⁸⁾ to 6 Group Addresses in the ASDU; each Group Address shall occupy 2 octets.

⁸⁾ “0” is the typical response in case no association exists for the considered Group Object.

sending_address:	1 to 15, 0 means none
start_index:	0 ⁹⁾
group_address_list:	Group Addresses (each on 16 bit) : none

octet 6						octet 7						octet 8						octet 9				octet 10						...				octet n ≤ 21																	
						APCI						group_object_number						sending_address		start_address		group_address_list																											
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0		
						APCI APCI APCI APCI APCI APCI APCI APCI												s s s		- - - -																													
						1	1	1	1	1	1	0	0	1	1	0																																	

Figure 57 - A Link Response-PDU (example)

The remote Application Layer shall accept the service response and shall pass it with a T_Data_Individual.req to the remote Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters TSAP and priority of the T_Data_Individual.req primitive; the TSDU shall be an A_Link_Response-PDU.

The local Application Layer shall map a T_Data_Individual.ind primitive with a TSDU = A_Link_Response-PDU to an A_Link_Read.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_Link_Read.res primitive.

A Link_Read.req (ASAP, group_object number, priority, start_index)

ASAP:	local reference of the Service Access Point or Individual Address.
group_object_number:	local index of the intended Group Object. The counting shall start with 1.
priority:	system, urgent, normal or low priority
start_index	index of first GA from which the GAs assigned to the address GO in the communication partner shall be read. The counting shall start with 1. The range shall be 1 to 15.

A Link Read.res (ASAP, group address list, group object number, priority, sending address, start index)

ASAP:	local reference of the Service Access Point or Individual Address.
group_address_list:	list of Group Addresses connected to the Group Object; 0 to 6 ¹⁰⁾ Group Addresses can be contained, each shall occupy 2 octets.
group_object_number:	local index of the addressed Group Object. The counting shall start with 1.
priority:	system, urgent, normal or low priority
sending_address:	index of the sending address in the list of GAs attached to the Group Object. The counting shall start with 1.
start_index:	index of first transmitted GA in the list of GAs attached to the Group Object. The counting shall start with 1.

9) Sufficient to match negative response and request, as long as response is awaited before sending any other request to same device.

¹⁰⁾ This list is limited to the 6 first Group Addresses (for a given Group Object) starting at the start_index. It is expected however that devices with more than 6 links to a single Group Object will use more powerful mechanisms.

3.4.6.2 A_Link_Write-service

The A_Link_Write.req primitive shall be applied by the user of Application Layer to add or remove a single Group Address to or from a given Group Object in a communication partner. The communication partner shall be addressed with a local ASAP that shall be mapped to an Individual Address by the Transport Layer. The Group Object of the communication partner shall be addressed with a Group Object number. It shall indicate the required action through the argument flags. In case a Group Address is added to a Group Object, the argument flags shall also indicate whether the added Group Address shall be the sending Group Address for the referred Group Object or not.

The local Application Layer shall accept the service request and pass it with a T_Data_Individual.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Individual.req primitive, the TSDU shall be an A_Write_Link-PDU.

octet 6						octet 7						octet 8						octet 9						octet 10						octet 11									
						APCI						group_object_number						flags						group_address															
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI							d flag s flag																	
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																										
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																										
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																										
						1	1	1	1	1	1	0	0	1	1	1								0	0	0	0	0	0										

Figure 58 - A_Link_Write-PDU (example)

Flags

d

- 0: add: Add the contained Group Address to the list of Group Addresses assigned to the referred Group Object.
- 1: delete: Remove the contained Group Address from the list of Group Addresses assigned to the referred Group Object.

s

The flag s shall only be interpreted in case the flag d equals 0; in case the flag d equals 1, the value of the flag s shall be don't care.

- 0: not sending: The contained Group Address shall not be the sending Group Address for the referred Group Object.
- 1: sending: The contained Group Address shall be the sending Group Address for the referred Group Object.

The reception of an A_Link_Write-PDU on an existing association with a different value of the s-flag shall result in an update of the table entry with the new s-flag value.

A Group Address set as "sending" shall take precedence on other previous sending Group Address.

The remote Application Layer shall map a T_Data_Individual.ind primitive with TSDU = A_Link_Write-PDU to an A_Link_Write.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_Link_Write.ind primitive.

The remote management process shall update the link information of the addressed Group Object according to the contained parameters. It shall respond to the A_Link_Write.ind primitive with an A_Link_Write.res primitive containing the resulting current list of links on the addressed Group Object and the indication of the sending address if any. The TSDU shall be an A_Link_Response-PDU.

Error and exception handling

- If the remote management process has a problem, e.g., addressed Group Object does not exist or the contained Group Address is not assigned to the addressed Group Object, then it shall respond with a negative response in the A_Link_Response-PDU.
- If after the evaluation of the request more than six Group Addresses are assigned to the addressed Group Object, the remote management process shall only send one single A_Link_Response-PDU containing the six first Group Addresses, thus possibly not containing the Group Address contained in the request.

A_Link_Write.req (ASAP, flags, group_address, group_object_number, priority)

ASAP:	local reference of the Service Access Point
flags:	action to be done (delete/add); sending group address flag
group_address:	Group Address corresponding to the link to add or delete
group_object_number:	local index of the intended Group Object
priority:	system, urgent, normal or low priority

3.4.7 Function Property services

3.4.7.1 A_FunctionPropertyCommand-Service

The A_FunctionPropertyCommand.req primitive shall be applied by the user of Application Layer to call a Function Property of an Interface Object in a communication partner. The communication partner shall be addressed with a local ASAP that shall be mapped to an Individual Address by the local Transport Layer. The Interface Object of the partner shall be addressed with the object_index and the Function Property of the Interface Object shall be addressed with the property_id.

The local Application Layer shall accept the service request and pass it with a T_Data_Individual.req or a T_Data_Connected.req (depending on whether the service is called connection-oriented or connectionless) to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters TSAP and priority of the T_Data_Individual.req or the T_Data_Connected.req primitive; the TSDU shall be an A_FunctionPropertyCommand-PDU.

The remote Application Layer shall map a T_Data_Individual.ind or a T_Data_Connected.ind primitive with TSDU = A_FunctionPropertyCommand-PDU to an A_FunctionPropertyCommand.ind primitive. The parameter comm_mode shall be set to “connection_oriented” in case of a T_Data_Connected.ind and to “connectionless” in case of a T_Data_Individual.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_FunctionPropertyCommand.ind primitive.

The remote application shall check whether the Property Datatype of the Property addressed by this A_FunctionPropertyCommand.ind is PDT_Function. If this is the case it shall call the function and pass the input parameters to it, otherwise the error handling shall apply; see clause 3.4.7.3.

The remote application process shall respond to the A_FunctionPropertyCommand.ind primitive with an A_FunctionPropertyCommand.res primitive containing the return_code and the function specific output data.

Figure 59 – A_FunctionPropertyCommand-PDU (example)

A_FunctionPropertyCommand.ind(ASAP, comm_mode, data, hop_count_type, object_index, priority, property_id)

ASAP:	local reference of the Service Access Point or Individual Address
comm_mode:	connection-oriented or connectionless
data:	input data to the function
hop_count_type:	standard NL hop count value
object_index:	the object_index of the addressed Interface Object
priority:	system, urgent, normal or low priority
property_id:	the property_id of the Property of the addressed Interface Object

A_FunctionPropertyCommand.res(ack_request, ASAP, comm_mode, data, hop_count_type, object_index, priority, property_id, return_code)

ack_request:	Data Link Layer Acknowledge requested or don't care
ASAP:	local reference of the Service Access Point or Individual Address
comm_mode:	connection-oriented or connectionless
data:	output data from the function
hop_count_type:	standard NL hop count value
object_index:	the object_index of the addressed Interface Object
priority:	system, urgent, normal or low priority
property_id:	the property_id of the Property of the addressed Interface Object
return_code	error code returned by the function

A_FunctionPropertyCommand.Acon(ASAP, comm_mode, data, hop_count_type, object_index, priority, property_id, return_code)

ASAP:	local reference of the Service Access Point or Individual Address
comm_mode:	connection-oriented or connectionless
data:	output data from the function
hop_count_type:	standard NL hop count value
object_index:	the object_index of the addressed Interface Object
priority:	system, urgent, normal or low priority
property_id:	the property_id of the Property of the addressed Interface Object
return_code	error code returned by the function

3.4.7.2 A_FunctionPropertyState_Read-service

The A_FunctionPropertyState_Read.req primitive shall be applied by the user of Application Layer to read the state of a Function Property of an Interface Object in a remote device. The communication partner shall be addressed with a local ASAP that shall be mapped to an Individual Address by the local Transport Layer. The Interface Object of the remote device shall be addressed with an object_index and the Property of the Interface Object shall be addressed with a property_id. The user of the Application Layer in the remote device shall respond with an A_FunctionPropertyState_Read.res, this is, the service shall be confirmed by the remote application process.

The local Application Layer shall accept the service request and pass it with a T_Data_Individual.req or a T_Data_Connected.req (depending on whether the service is sent connection-oriented or connectionless) to the local Transport Layer. The TSDU shall be an A_FunctionPropertyState_Read-PDU.

The remote Application Layer shall map a T_Data_Individual.ind primitive or a T_Data_Connected.ind primitive with TSDU = A_FunctionPropertyState_Read-PDU to an A_FunctionPropertyState_Read.ind primitive.

The remote application shall check whether the Property Datatype of the Property addressed by this A_FunctionPropertyState_Read.ind is PDT_Function. If this is the case it shall call the function and pass the input parameters to it, otherwise the error handling shall apply; see clause 3.4.7.3.

The application process shall respond to the A_FunctionPropertyState_Read.ind primitive with an A_FunctionPropertyState_Read.res primitive containing the function specific return_code and the function specific output data.

octet 6								octet 7								octet 8								octet 9								octet 10 ...								
						APCI						object_index								property_id								data ...												
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																									
						1	0	1	1	0	0	1	0	0	0																									

Figure 60 - A_FunctionPropertyState_Read-PDU (example)

octet 6								octet 7								octet 8								octet 9								octet 10								octet 11 ...									
								APCI								object_index								property_id								return_code								data ...									
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0		
						APCI	APCI																																										
						1	0	1	1	0	0	1	0	0	1																																		

Figure 61 - A_FunctionPropertyState_Response-PDU (example)

The remote Application Layer shall accept the service response and pass it with a T_Data_Individual.req or a T_Data_Connected.req (depending whether the service shall be sent connection-oriented or connectionless) to the local Transport Layer. The TSDU shall be an A_FunctionPropertyState_Response-PDU.

The local Application Layer shall map a T_Data_Individual.ind or T_Data_Connected.ind primitive with TSDU = A_FunctionPropertyState_Response-PDU to an A_FunctionPropertyState_Read.Acon primitive.

A_FunctionPropertyState_Read.req(ASAP, ack_request, comm_mode, data, hop_count_type, object_index, priority, property_id)

ASAP: local reference of the Service Access Point or Individual Address

ack_request: Data Link Layer Acknowledge requested or don't care

comm_mode: connection_oriented or connectionless

data: input data to the function for reading

hop_count_type: standard NL hop count value

object_index: the object_index of the addressed Interface Object

priority: system, urgent, normal or low priority

property_id: the property_id of the Property of the Interface Object

A_FunctionPropertyState_Read.Lcon(ack_request, ASAP, comm_mode, data, hop_count_type, object_index, priority, property_id, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
ASAP:	local reference of the Service Access Point or Individual Address
comm_mode:	connection_oriented or connectionless
data	input data to the function for reading
hop_count_type:	standard NL hop count value
object_index:	the object_index of the addressed Interface Object
priority:	system, urgent, normal or low priority
property_id:	the property_id of the Property of the addressed Interface Object
a_status:	ok: A_FunctionPropertyState_Read-PDU sent successfully with T_Data_Individual or T_Data_Connected service
	not_ok: transmission of the associated T_Data_Individual or T_Data_Connected request frame did not succeed

A_FunctionPropertyState_Read.ind(ASAP, comm_mode, data, hop_count_type, object_index, priority, property_id)

ASAP:	local reference of the Service Access Point or Individual Address
comm_mode:	connection_oriented or connectionless
data	input data to the function for reading
hop_count_type:	standard NL hop count value
object_index:	the object_index of the addressed Interface Object
priority:	system, urgent, normal or low priority
property_id:	the property_id of the Property of the addressed Interface Object

A_FunctionPropertyState_Read.res(ack_request, ASAP, comm_mode, data, hop_count_type, object_index, priority, property_id, return_code)

ack_request:	Data Link Layer Acknowledge requested or don't care
ASAP:	local reference of the Service Access Point or Individual Address
comm_mode:	connection_oriented or connectionless
data:	output data from the function
hop_count_type:	standard NL hop count value
object_index:	the object_index of the addressed Interface Object
priority:	system, urgent, normal or low priority
property_id:	the property_id of the Property of the addressed Interface Object
return_code	error code returned by the function

A_FunctionPropertyState_Read.Acon(ASAP, comm_mode, data, hop_count_type, object_index, priority, property_id, return_code)

ASAP:	local reference of the Service Access Point or Individual Address
comm_mode:	connection_oriented or connectionless
data:	output data from the function
hop_count_type:	standard NL hop count value
object_index:	the object_index of the addressed Interface Object
priority:	system, urgent, normal or low priority
property_id:	the property_id of the Property of the addressed Interface Object
return_code	error code returned by the function

3.4.7.3 Error and exception handling for Function Property services

If the Interface Object Property accessed by A_FunctionPropertyCommand-PDU or by A_FunctionPropertyState_Read-PDU is not of the Property Datatype PDT_Function, the remote application shall respond with a A_FunctionPropertyState_Response-PDU without the field return_code (this is, the returned PDU shall not contain the field return_code) and without the field data (this is, the returned PDU shall not contain the field data).

In case the remote application is able to successfully call a Function Property, then the Function Property shall deliver a return_code in the field return_code of the PDU. The following rules shall apply for all functions:

- Return_code = 00h: function successfully executed; this is the return code 00h shall be the positive result of the function;
- Return_code ≠ 00h: error.

Error codes are defined in a function specific way.

In case an Interface Object Property of Property Datatype PDT_Function is accessed via the A_PropertyValue_Read-service or the A_PropertyValue_Write-service, the Application Layer shall respond with an A_PropertyValue_Response-PDU with the standard error handling for Data Properties (this is the field nr_of_elem shall be zero and there shall be no data field).

In case an Interface Object Property of Property Datatype PDT_Function is accessed via the A_PropertyDescription_Read, the Application Layer shall respond with an A_PropertyDescription_Response-PDU with type = PDT_Function and max_nr_of_elem = 1; read_level and write_level can have any value.

3.4.8 Function Property Extended services

3.4.8.1 A_FunctionPropertyExtCommand-Service

The A_FunctionPropertyExtCommand.req primitive shall be applied by the user of Application Layer to call a Function Property of an Interface Object in a communication partner. The communication partner shall be addressed with a local ASAP that shall be mapped to an Individual Address by the local Transport Layer. The Interface Object of the partner shall be addressed with the Interface Object Type and the Object Instance, and the Function Property of the Interface Object shall be addressed with the Property Identifier.

The local Application Layer shall accept the service request and pass it with a T_Data_Individual.req (depending on whether the service is called connectionless or connection-oriented) to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters TSAP and priority of the T_Data_Individual.req or the T_Data_Connected.req primitive; the TSDU shall be an A_FunctionPropertyExtCommand-PDU.

The remote Application Layer shall map a T_Data_Individual.ind primitive with TSDU = A_FunctionPropertyExtCommand-PDU to an A_FunctionPropertyExtCommand.ind primitive. The parameter comm_mode shall be set to “connection_oriented” in case of a T_Data_Connected.ind and to “connectionless” in case of a T_Data_Individual.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_FunctionPropertyExtCommand.ind primitive.

The remote application shall check whether the Property Datatype of the Property addressed by this A_FunctionPropertyExtCommand.ind is PDT_FUNCTION or PDT_CONTROL. If the property datatype is PDT_FUNCTION, it shall call the function and pass the input parameters to it, if the property datatype is PDT_CONTROL, it shall execute the control as specified in clause 3.4.8.4, otherwise the error handling shall apply; see clause 3.4.8.3.

The remote application process shall respond to the A_FunctionPropertyExtCommand.ind primitive with an A_FunctionPropertyExtCommand.res primitive containing the return_code and the function specific output data.

octet 6								octet 7								octet 8								octet 9								octet 10								octet 11								octet 12								
								APCI								interface_object_type																object_instance								property_id																
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0									
								APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																																								
						0	1	1	1	0	1	0	1	0	0																																									

octet 13 to octet n							
data							
7	6	5	4	3	2	1	0

Figure 62 - A_FunctionPropertyExtCommand-PDU (example)

The remote Application Layer shall accept the service response and pass it with a T_Data_Individual.req (depending on whether the service shall be sent connection-oriented or connectionless) to the local Transport Layer. The TSDU shall be an A_FunctionPropertyExtState_Response-PDU.

The local Application Layer shall map a T_Data_Individual.ind or T_Data_Connected.ind primitive with TSDU = A_FunctionPropertyExtState_Response-PDU to an A_FunctionPropertyExtCommand.Acon primitive.

A_FunctionPropertyExtCommand.req(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
data:	input data to the function

A_FunctionPropertyExtCommand.Lcon(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, data, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
data:	input data to the function
a_status:	ok: A_FunctionPropertyExtCommand sent successfully with T_Data_Individual- or T_Data_Connected-service
	not_ok: transmission of the associated T_Data_Individual – or T_Data_Connected - request Frame did not succeed

A_FunctionPropertyExtCommand.ind(priority, hop_count_type, ASAP, object_type, object_instance, property_id, data)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
data:	input data to the function

A_FunctionPropertyExtCommand.res(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, return_code, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
return_code	the return code of the operation
data:	Output data from the function (function-specific)

A_FunctionPropertyExtCommand.Acon(priority, hop_count_type, ASAP, object_type, object_instance, property_id, return_code, data)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
return_code	the result code of the operation
data:	Output data from the function (function-specific)

3.4.8.2 A_FunctionPropertyExtState_Read-service

The A_FunctionPropertyExtState_Read.req primitive shall be applied by the user of Application Layer to read the state of a Function Property of an Interface Object in a remote device. The communication partner shall be addressed with a local ASAP that shall be mapped to an Individual Address by the local Transport Layer. The Interface Object of the remote device shall be addressed with an Interface Object Type and the Object Instance, and the Property of the Interface Object shall be addressed with a Property Identifier. The user of the Application Layer in the remote device shall respond with an A_FunctionPropertyExtState_Read.res; this is, the remote application process service shall confirm the service.

The local Application Layer shall accept the service request and pass it with a T_Data_Individual.req (depending on whether the service is sent connectionless or connection-oriented) to the local Transport Layer. The TSDU shall be an A_FunctionPropertyExtState_Read-PDU.

The remote Application Layer shall map a T_Data_Individual.ind primitive with TSDU = A_FunctionPropertyExtState_Read-PDU to an A_FunctionPropertyExtState_Read.ind primitive.

The remote application shall check whether the Property Datatype of the Property addressed by this A_FunctionPropertyExtState_Read.ind is PDT_FUNCTION or PDT_CONTROL. If the property datatype is PDT_FUNCTION, it shall call the function and pass the input parameters to it, if the property datatype is PDT_CONTROL, it shall return the state of the control as specified in clause 3.4.8.4, otherwise the error handling shall apply; see clause 3.4.8.3.

The application process shall respond to the A_FunctionPropertyExtState_Read.ind primitive with an A_FunctionPropertyExtState_Read.res primitive containing the function specific Return Code and the function specific output data.

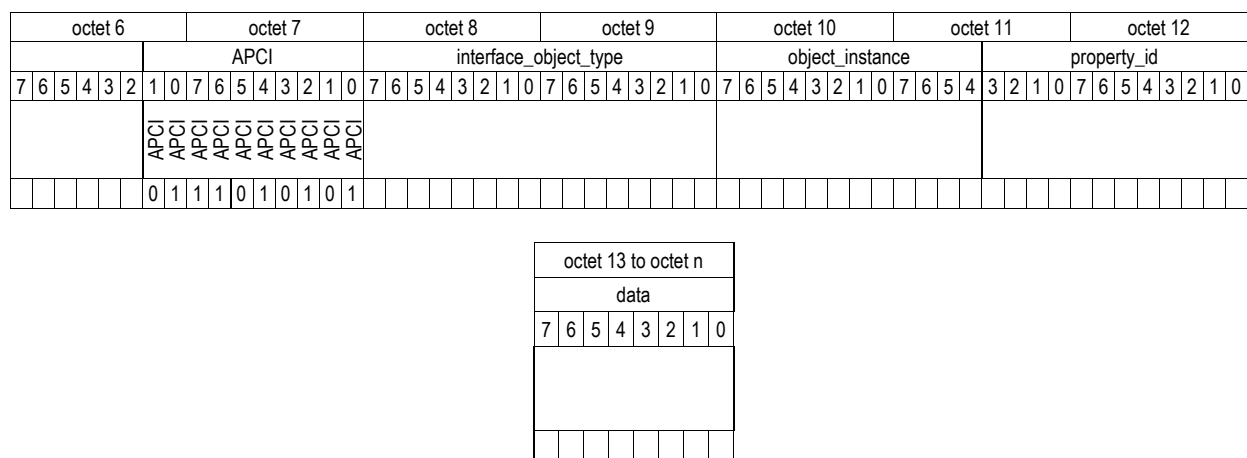


Figure 63 - A_FunctionPropertyExtState_Read-PDU (example)

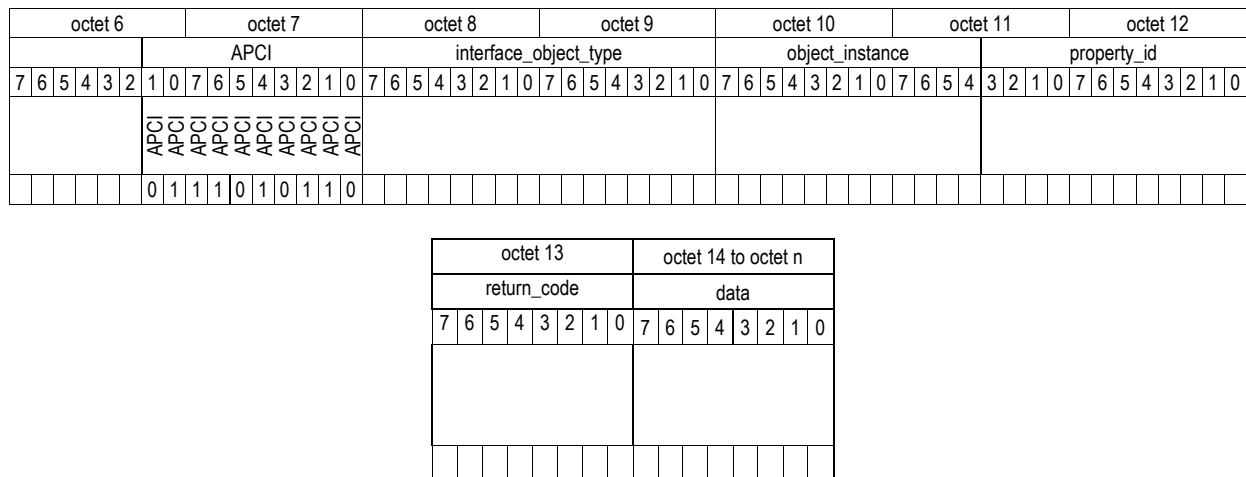


Figure 64 - A_FunctionPropertyExtState_Response-PDU (example)

The remote Application Layer shall accept the service response and pass it with a T_Data_Individual.req (depending whether the service shall be sent connectionless or connection-oriented) to the local Transport Layer. The TSDU shall be an A_FunctionPropertyExtState_Response-PDU.

The local Application Layer shall map a T_Data_Individual.ind or T_Data_Connected.ind primitive with TSDU = A_FunctionPropertyExtState_Response-PDU to an A_FunctionPropertyExtState_Read.Acon primitive.

A_FunctionPropertyExtState_Read.req(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
data:	input data to the function for reading

A_FunctionPropertyExtState_Read.Lcon(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, data, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
data:	input data to the function for reading
a_status:	ok: A_FunctionPropertyExtState_Read-PDU sent successfully with T_Data_Individual - or T_Data_Connected service not_ok: transmission of the associated T_Data_Individual – or T_Data_Connected request Frame did not succeed

A_FunctionPropertyExtState_Read.ind(priority, hop_count_type, ASAP, object_type, object_instance, property_id, data)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
data:	input data to the function for reading

A_FunctionPropertyExtState_Read.res(ack_request, priority, hop_count_type, ASAP, object_type, object_instance, property_id, return_code, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
return_code:	the Return Code of the operation
data:	output data from the function (function-specific)

A_FunctionPropertyExtState_Read.Acon(priority, hop_count_type, ASAP, object_type, object_instance, property_id, return_code, data)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point or Individual Address
object_type:	the Interface Object Type of the addressed Interface Object
object_instance:	the Object Instance of the addressed Interface Object
property_id:	the Property Identifier of the Property of the addressed Interface Object
return_code:	the Return Code of the operation
data:	Output data from the function (function-specific)

3.4.8.3 Error and exception handling for Function Property services

- If the Interface Object Property accessed by A_FunctionPropertyExtCommand-PDU or by A_FunctionPropertyExtState_Read-PDU is not present, not accessible, or not of the Property Datatype PDT_FUNCTION or PDT_CONTROL then the remote application shall respond with an A_FunctionPropertyExtState_Response-PDU with the appropriate return Code (see table below) in the field return_code. The returned PDU shall not contain the field data (this is, the returned PDU shall not contain the octets 14...n).
- In case the remote application is able to successfully call a Function Property, then the Function Property shall deliver a function result in the field return_code and data of the PDU. The function result is defined in a function-specific way
- In case an Interface Object Property of Property Datatype PDT_FUNCTION or PDT_CONTROL is accessed via the A_PropertyExtDescription_Read, the Application Layer shall respond with an A_PropertyExtDescription_Response-PDU with type = PDT_FUNCTION or PDT_CONTROL and max_nr_of_elem = 1; read_level and write_level can have any value.

NOTE 12 All newly defined function properties shall return a return code compatible with the unified return code schema. Several existing functions definitions are not compatible, e.g., using a return code from the range 00h-7Fh for error results. It is the responsibility of the client to handle these exceptional cases.

Return Code	Error Type	Error Name
Basic positive Return Code		
00h	Success	E_SUCCESS
	The service, function or command is executed successfully, without additional information.	
Generic positive Return Codes 01h-1Fh		
None proposed		
Specific positive Return Codes 20h-5Fh		
None proposed		
Specific negative Return Codes A0h-DFh		
None proposed		
Generic negative Return Codes E0h-FEh		
F1h	Memory Error	E_MEMORY_ERROR
	Memory cannot be accessed or only with fault(s).	
F2h	Command invalid	E_COMMAND_INVALID
	This means that the command is not supported by this server.	
F3h	Command impossible	E_COMMAND_IMPOSSIBLE
	This means that the command is supported and well formatted, but that it is not possible to be executed right now. There is a dependency that is not fulfilled.	
F4h	Device Capabilities	E_LENGTH_EXCEEDS_MAX_APDU_LENGTH
	Result data will not fit into a Frame supported by this server. This shall be used for Device limitations of the maximum supported Frame length by accessing resources (Properties, Function Properties, memory...) of the device.	
F8h	void data	E_DATA_VOID
	The service or the function is supported, but the request data is not valid for this receiver.	
F9h	Temporarily not available	E_TEMPORARILY_NOT_AVAILABLE
	This shall mean that the function is not available at this time because - another MaC is accessing the data, or - the data is currently being processed by the MaS (e.g., being flashed, or being renewed...)	
FCh	Access denied	E_ACCESS_DENIED
	The access is not allowed because of authorisation reasons, A_Authorize as well as KNX Security.	
FDh	void address	E_ADDRESS_VOID
	The Interface Object or the property is not present	

Return Code	Error Type	Error Name
FEh	Type Conflict	E_DATA_TYPE_CONFLICT
	Property is not of type PDT_FUNCTION or PDT_CONTROL	
Basic Negative Return Code		
FFh	Error	E_ERROR
	The service, function or command has failed without a closer indication of the problem.	

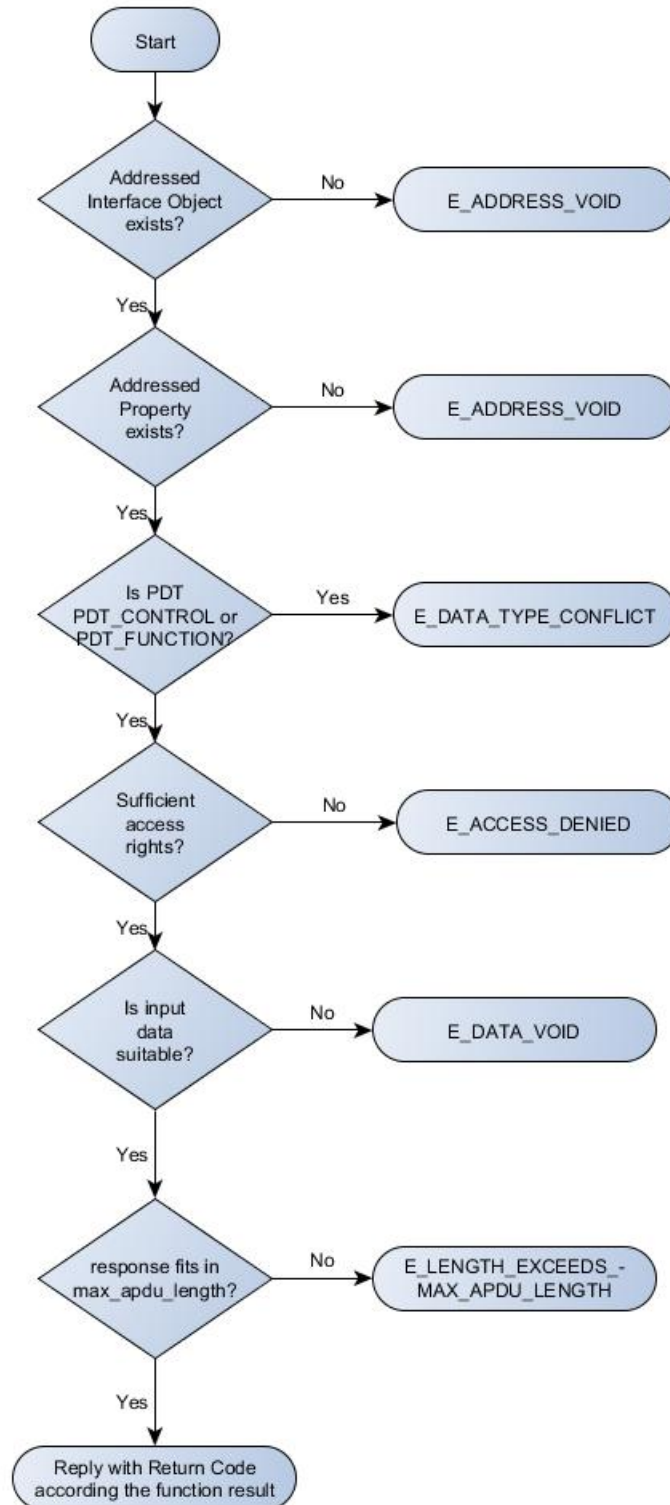


Figure 65 - Flow chart for Extended Function Property services

3.4.8.4 Function Property services for PDT_CONTROL

The mapping of the input and output data of Function Property services for properties of datatype PDT_CONTROL to the corresponding input and output data of the traditional data property services shall be as follows.

- For A_FunctionPropertyExtCommand and A_FunctionPropertyCommand, the data field shall contain exactly the 10 octet data as would be written when using data services.

- For A_FunctionPropertyExtState_Read and A_FunctionPropertyState_Read the data field shall be empty.
- For A_FunctionPropertyExtState_Response and A_FunctionPropertyState_Response, the return_code shall either
 - be a positive return code and the data field shall be a single octet containing the new state as returned when using data services, or
 - be a negative return code, in which case the data field shall be empty.

See also in [05] for the use of Property services and Extended Property services and the use of Return Codes, in the specification of the Management Procedures DM_LoadStateMachineWrite, DM_LoadStateMachineVerify, DM_LoadStateMachineRead, DM_RunStateMachineWrite, DM_RunStateMachineVerify and DM_RunStateMachineRead.

3.4.9 Memory services

3.4.9.1 A_MemoryExtended_Read-service

The A_MemoryExtended_Read.req primitive shall be applied by the user of Application Layer, to read between 1 and 250 octets in the address space of the remote communication partner. The parameter “address” shall specify the 24 bit start address and “number of data octets” shall contain the number of octets to be read beginning with the start address to increasing addresses. The service shall be confirmed by the remote application process with the contents of the address space.

The local Application Layer shall accept the service request and pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_MemoryExtended_Read - PDU.

The remote Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_MemoryExtended_Read - PDU to an A_MemoryExtended_Read.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_MemoryExtended_Read.ind primitive.

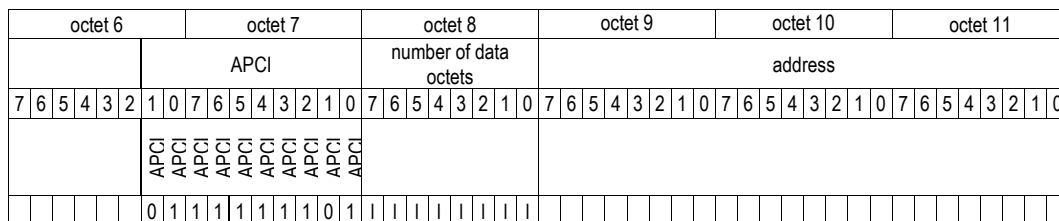


Figure 66 - A_MemoryExtended_Read-PDU

The remote application process shall respond to the A_MemoryExtended_Read.ind primitive with an A_MemoryExtended_Read.res primitive containing the number of octets read beginning with the start address to increasing addresses. If the remote application process has a problem, e.g., address space unreachable or protected or an illegal number of octets are requested, then the parameter *Return Code* of the A_MemoryExtended_Read_Response-PDU shall be filled with the corresponding error code.

Details are specified in 3.4.9.1.1.

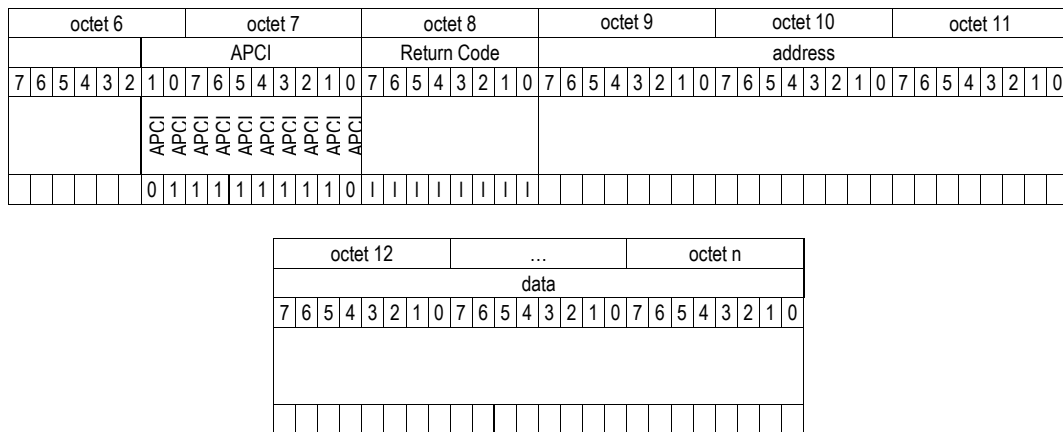


Figure 67 - A_MemoryExtended_Read_Response-PDU (example)

The remote Application Layer shall accept the service response and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters TSAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_MemoryExtended_Read_Response-PDU.

The Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_MemoryExtended_Read_Response-PDU to an A_MemoryExtended_Read.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_MemoryExtended_Read.Acon primitive.

A_MemoryExtended_Read.req(ack_request, priority, hop_count_type, ASAP, number, memory_address)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
par_auth:	This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
par_conf:	This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
ASAP:	local reference of the Service Access Point
number:	number of octets to be read beginning with the start address to increasing addresses
address:	specifies the 24 bit start address

A_MemoryExtended_Read.Lcon(ack_request, priority, hop_count_type, ASAP, number, memory_address, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
par_auth:	This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
par_conf:	This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
ASAP:	local reference of the Service Access Point
number:	number of octets to be read beginning with the start address to increasing addresses
address:	specifies the 24 bit start address
a_status: ok:	ok: A_Memory_Read sent successfully with T_Data_Connected service
	not_ok: transmission of the associated T_Data_Connected request Frame did not succeed

A_MemoryExtended_Read.ind(priority, hop_count_type, ASAP, number, memory_address)

priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 par_auth: This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
 par_conf: This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
 ASAP: local reference of the Service Access Point
 number: number of octets to be read beginning with the start address to increasing addresses
 address: specifies the 24 bit start address

A_MemoryExtended_Read.res(ack_request, priority, hop_count_type, ASAP, Return Code, memory_address, data)

ack_request: Data Link Layer Acknowledge requested or don't care
 priority: system, urgent, normal or low priority
 hop_count_type: standard NL hop count value
 par_auth: This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
 par_conf: This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
 ASAP: local reference of the Service Access Point
 Return Code: Return Code of the executed command
 address: specifies the 24 bit start address
 data: the octet(s) read

Error handling

If data are to be read from a protected area or from any logical address that is not associated to physical memory the Return Code gives details to the indicated an error. The same shall apply if only part of the memory to be read is protected or physically not existing.

In addition, the Return Code gives details to the indicated an error if the value of the parameter “number of data octets” is greater than Maximum APDU Length – 4 ¹¹⁾.

3.4.9.1.1 A_MemoryExtended_Read_Response-PDU

Table 3 - Read Return Codes

Return Code	Error Type	Error Name
Basic positive Return Code		
0h	Success	E_SUCCESS
	The service, function or command is executed successfully, without additional information.	

¹¹⁾ The maximal APDU length that shall be supported is specified in the specification of PID_MAX_APDULENGTH in [04].

Return Code	Error Type	Error Name
Generic positive Return Codes 01h-1Fh		
None proposed		
Specific positive Return Codes 20h-5Fh		
None proposed		
Specific negative Return Codes A0h-DFh		
None proposed		
Generic negative Return Codes E0h-FEh		
F1h	Memory Error	E_MEMORY_ERROR
	memory cannot be accessed or only with fault(s)	
F4h	Device Capabilities	E_LENGTH_EXCEEDS_MAX_APDU_LENGTH
	Requested data will not fit into a Frame supported by this server.	
F9h	Temporarily not available	E_TEMPORARILY_NOT_AVAILABLE
	This shall mean that the data could in generally be written, but that it is not possible at this time because • another MaC is accessing the data • the data is currently being processed by the MaS (e.g., being flashed, or being renewed...)	
FAh	Write Only	E_ACCESS_WRITE_ONLY
	read access to a “write only” memory location	
FCh	Access denied	E_ACCESS_DENIED
	The access is not allowed because of authorisation reasons, A_Authorize as well as KNX Security.	
FDh	void address	E_ADDRESS_VOID
	The “address” does not exist.	
Basic Negative Return Code		
FFh	Error	E_ERROR
	The service, function or command has failed without a closer indication of the problem.	

The A_MemoryExtended_Read_Response-PDU shall have a variable length that shall depend on the Return Code. This is shown in Figure 68.

8	9	10	11	12	..	n	Name
Return Code	address			data			
00h	address			data			E_SUCCESS
F1h	address			(This field shall not be present. The APDU shall end with the address.)			E_MEMORY_ERROR
F4h	address			(This field shall not be present. The APDU shall end with the address.)			E_LENGTH_EXCEEDS_MAX_APDU_LENGTH
F9h	address			(This field shall not be present. The APDU shall end with the address.)			E_TEMPORARILY_NOT_AVAILABLE
FAh	address			(This field shall not be present. The APDU shall end with the address.)			E_ACCESS_WRITE_ONLY
FCh	address			(This field shall not be present. The APDU shall end with the address.)			E_ACCESS_DENIED
FDh	address			(This field shall not be present. The APDU shall end with the address.)			E_ADDRESS_VOID
FFh	address			(This field shall not be present. The APDU shall end with the address.)			E_ERROR
all other values	reserved						Reserved for future positive confirmations

Figure 68 - A MemoryExtended Read Response-PDU (detailed)

3.4.9.2 A MemoryExtended Write-service

The user of the Application Layer shall apply the A_MemoryExtended_Write.req primitive to write between 1 octet and 250 octets in the address space of the remote communication partner. The parameter “address” shall specify the 24-bit start address and the parameter “number of data octets” shall contain the number of octets to be written beginning with the start address to increasing addresses.

The service shall be a confirmed service. Confirmation is received via A_MemoryExtended_Write_Response. The setting of the Verify Mode has no influence on this service and will not change the behaviour of the Management Server.

The APCI shall be the APCI_MemoryExtended_Write. The communication partner shall be addressed with a local ASAP that shall be mapped to an Individual Address by the Transport Layer.

The local Application Layer shall accept the service request and shall pass it with a T_Data_Individual.req to the local Transport Layer. The parameter ASAP and priority shall be mapped to the corresponding ASAP of the T_Data_Individual.req primitive. The TSDU shall be an A_MemoryExtended_Write-PDU.

[illegible]

Figure 69 - A MemoryExtended Write-PDU (example)

The remote Application Layer shall map a T_Data_Individual.ind primitive with TSDU = A_MemoryExtended_Write-PDU to an A_MemoryExtended_Write.ind primitive. The argument TSAP and priority shall be mapped to the corresponding argument ASAP of the A_MemoryExtended_Write.ind primitive.

The remote Application Process shall respond to the A_MemoryExtended_Write.ind primitive with an A_MemoryExtended_Write.res primitive containing the Return Code and the same address as used in the A_MemoryExtended_Write.ind primitive. If the remote application process has a problem, e.g., memory area unreachable or protected, the appropriate Return Code shall be filled the corresponding error code.

Details are specified in 3.4.9.2.1.

If the remote Application Layer Process applies an A_MemoryExtended_Write.res, then the remote Application Layer shall accept the service response and pass it with a T_Data_Individual.req (T_Data_Connect.req) to the remote Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters TSAP and priority of the T_Data_Individual.req primitive, the TSDU shall be an A_MemoryExtended_Write_Response-PDU.

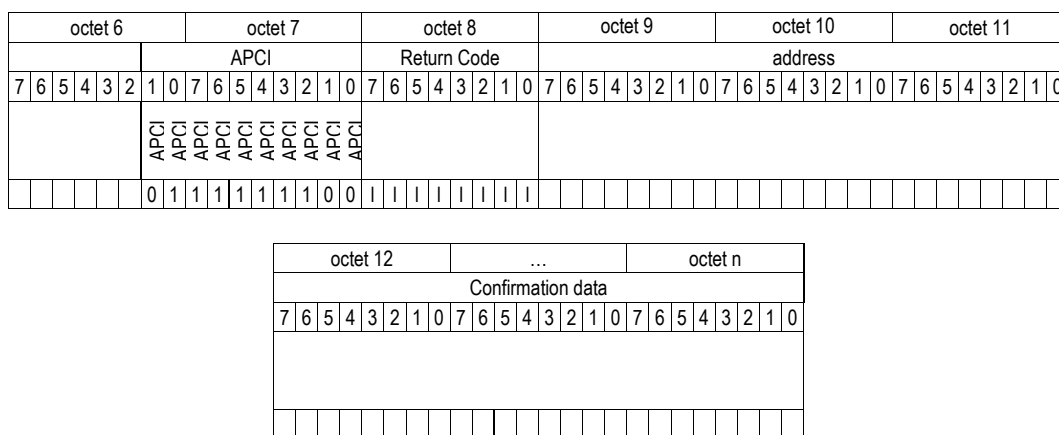


Figure 70 - A MemoryExtended Write Response-PDU (example)

The local Application Layer shall map a T_Data_Individual.ind (T_Data_Connected.ind) primitive with TSDU = A_MemoryExtended_Write_Response-PDU to an A_MemoryExtended_Write.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_MemoryExtended_Write.Acon primitive.

A MemoryExtended Write.req(priority, ASAP, number of octets, address, sequence number, data)

ASAP:	local reference of the Service Access Point
priority:	system, urgent, normal or low priority
par_auth:	This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
par_conf:	This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
number:	number of octets to be written beginning with the start address to increasing addresses
address:	specifies the 24 bit start address from which the data shall be written in the MaS
data	the octet(s) to be written

A_MemoryExtended_Write.Lcon(priority, ASAP, number of octets, address, sequence number, data)

ASAP:	local reference of the Service Access Point
priority:	system, urgent, normal or low priority
par_auth:	This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
par_conf:	This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
number:	number of octets to be written beginning with the start address to increasing addresses
address:	specifies the 24 bit start address from which the data shall be written in the MaS
a_status:	ok: A_MemoryExtended_Write sent successfully with T_Data_Individual-service not_ok: transmission of the associated T_Data_Individual request Frame did not succeed
data	the octet(s) to be written

A_MemoryExtended_Write.ind(priority, ASAP, number of octets, address, sequence number, data)

ASAP:	local reference of the Service Access Point
priority:	system, urgent, normal or low priority
par_auth:	This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
par_conf:	This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
number:	number of octets to be written beginning with the start address to increasing addresses
address:	specifies the 24 bit start address from which the data shall be written in the MaS
data	the octet(s) to be written

A_MemoryExtended_Write.res(priority, ASAP, number of octets, address, sequence number, return_code, confirmation_data)

ASAP:	local reference of the Service Access Point
priority:	system, urgent, normal or low priority
par_auth:	This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
par_conf:	This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
number:	number of octets to be written beginning with the start address to increasing addresses
address:	specifies the 24 bit start address from which the data shall be written in the MaS
return_code:	Return Code and format of confirmation_data
confirmation_data:	additional data for the Return Code

A_MemoryExtended_Write.Rcon(priority, ASAP, number of octets, address, sequence number, return_code, confirmation_data)

ASAP:	local reference of the Service Access Point
priority:	system, urgent, normal or low priority
par_auth:	This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
par_conf:	This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
number:	number of octets to be written beginning with the start address to increasing addresses
address:	specifies the 24 bit start address from which the data shall be written in the MaS
return_code:	Return Code and format of confirmation_data
confirmation_data:	additional data for the Return Code
a_status:	ok: A_MemoryExtended_Write.res sent successfully with T_Data_Individual service
	not_ok: transmission of the associated T_Data_Individual request Frame did not succeed

A_MemoryExtended_Write.Acon(priority, ASAP, number of octets, address, sequence number, return_code, confirmation_data)

ASAP:	local reference of the Service Access Point
priority:	system, urgent, normal or low priority
par_auth:	This parameter shall indicate whether the service is communicated using secure communication with authentication or not.
par_conf:	This parameter shall indicate whether the service is communicated using secure communication with confidentiality or not.
number:	number of octets to be written beginning with the start address to increasing addresses
address:	specifies the 24 bit start address from which the data shall be written in the MaS
return_code	Return Code and format of confirmation_data
confirmation_data:	additional data for Return Code

3.4.9.2.1 A_MemoryExtended_Write_Response-PDU

Table 4 - Write Return Codes

Return Code	Error Type	Error Name
Basic positive Return Code		
00h	Success	E_SUCCESS
	The service, function or command is executed successfully, without additional information ¹²⁾ .	

Return Code	Error Type	Error Name
Generic positive Return Codes 01h-1Fh		
01h	CRC over original data ¹²⁾	E_SUCCESS_WITH_CRC
	Positive confirmation of message with CRC16-CCITT ¹³⁾ beginning after the APCL Octet, over received data including address and number of data octets.	
Specific positive Return Codes 20h-5Fh		
None proposed		
Specific negative Return Codes A0h-DFh		
None proposed		
Generic negative Return Codes E0h-FEh		
F1h	Memory Error	E_MEMORY_ERROR
	memory cannot be accessed or only with fault(s)	
F4h	Device Capabilities	E_LENGTH_EXCEEDS_MAX_APDU_LENGTH
	Received data cannot be evaluated due to reception capabilities This shall be used for Device limitations of the maximum supported Frame length by accessing resources (Properties, Function Properties, memory...) of the device ¹⁴⁾ .	
F9h	Temporarily not available	E_TEMPORARILY_NOT_AVAILABLE
	This shall mean that the data could in generally be written, but that it is not possible at this time because • another MaC is accessing the data • the data is currently being processed by the MaS (e.g., being flashed, or being renewed...)	
FBh	Read Only	E_ACCESS_READ_ONLY
	write access to a “read only” memory location	
FCh	Access denied	E_ACCESS_DENIED
	The access is not allowed because of authorisation reasons, A_Authorize as well as KNX Security.	
FDh	void address	E_ADDRESS_VOID
	The “address” does not exist.	

¹²⁾ The MaC cannot require from the MaS in the A_MemoryExtended_Write.req to reply with a CRC. Whether or not the MaS replies with a CRC is implementation dependent or may be required in a Profile definition. If the MaS does not use the CRC, the MaC is not required to verify the memory after a Configuration Procedure (e.g., by reading a CRC over a memory segment back from the MaS.)

¹³⁾ Width = 16 bit, Truncated polynomial = 1021h, Initial value = FFFFh, Input date is NOT reflected, Output CRC is NOT reflected, No XOR is performed on the output CRC. Same CRC as in System B

¹⁴⁾ This answer might not be sent by the device, because of the received Frame cannot be evaluated by the device and it is dropped in a lower layer (e.g., Link Layer) for the Memory Write service.

Return Code	Error Type	Error Name
Basic Negative Return Code		
FFh	Error	E_ERROR
	The service, function or command has failed without a closer indication of the problem.	

The A_MemoryExtended_Write_Response-PDU shall have a variable length that shall depend on the Return Code. This is shown in Figure 71. The used Return Code is dependent on the MaS implementation (see [06]).

EXAMPLE 2 If a MaS e.g., does not support the CRC16-CCITT algorithm, it may use the Return Code "E_SUCCESS" Instead, of "E_SUCCESS with CRC".

8	9	10	11	12	..	n	Name
Return Code	address			Confirmation data			
00h	address			(This field shall not be present. The APDU shall end with the address.)			E_SUCCESS
01h	address			CRC16-CCITT ¹⁵⁾		n/a	E_SUCCESS with CRC
F1h	address			(This field shall not be present. The APDU shall end with the address.)			E_MEMORY_ERROR
F4h	address			(This field shall not be present. The APDU shall end with the address.)			E_LENGTH_EXCEEDS_MAX_APDU_LENGTH
F9h	address			(This field shall not be present. The APDU shall end with the address.)			E_TEMPORARILY_NOT_AVAILABLE
FBh	address			(This field shall not be present. The APDU shall end with the address.)			E_ACCESS_READ_ONLY
FCh	address			(This field shall not be present. The APDU shall end with the address.)			E_ACCESS_DENIED
FDh	address			(This field shall not be present. The APDU shall end with the address.)			E_ADDRESS_VOID
FFh	address			(This field shall not be present. The APDU shall end with the address.)			E_ERROR
all other values	reserved						Reserved for future positive confirmations

Figure 71 - A_MemoryExtended_Write_Response-PDU (detailed)

3.5 Application Layer Services on Point-to-point Connection-Oriented Communication Mode

3.5.1 Introduction

A Point-to-point connection-oriented communication mode shall connect one device with another device. The following services can be applied on point-to-point connection-oriented communication modes if the connection is established (see Transport Layer state machine). Due to the behaviour of the Transport Layer state machine, the user of the Application Layer has to take into account that the connection may be released by the remote communication partner or by an error detected in the communication protocol. Therefore a T_Disconnect.ind primitive may occur at any time, i.e. also if the user of Application Layer is waiting for a confirmation from the Application Layer. The transport layer services T_Connect.ind and T_Disconnect.ind are mapped transparently to A_Connect.ind and A_Disconnect.ind service and passed to the user of Application Layer.

¹⁵⁾ Width = 16 bit, Truncated polynomial = 1021h (CCITT-16), Initial value = FFFFh, Input date is NOT reflected, Output CRC is NOT reflected, No XOR is performed on the output CRC. Same CRC as in System B

The Application Layer also provides an access protection mechanism on the point-to-point connection-oriented communication mode by an authorization procedure. This procedure is described in clause 3.5.7 "A_Authorize_Request-service".

3.5.2 A_ADC_Read-service

The A_ADC_Read.req primitive shall be applied by the user of Application Layer to read the value of the AD-converter of the communication partner. The service shall be confirmed by the remote application process containing the value of the converter.

The local Application Layer shall accept the service request and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_ADC_Read-PDU.

The remote Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_ADC_Read-PDU to an A_ADC_Read.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_ADC_Read.ind primitive.

The A_ADC_Read-PDU shall contain the channel number of the AD-converter and the number of consecutive read operations to the AD-converter.

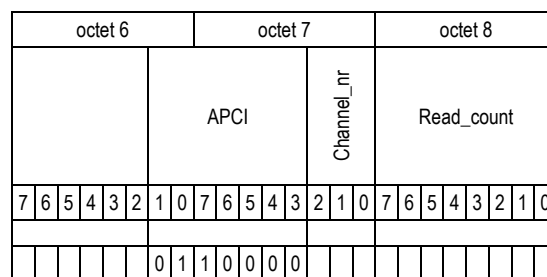


Figure 72 - A_ADC_Read-PDU (example)

The application process shall respond to the A_ADC_Read.ind primitive with an A_ADC_Read.res primitive containing the value of the AD-converter computed by the summation of the consecutive CPU accesses. If the remote application process has a problem, e.g., overflow when computing the summation, or wrong channel number, then the read_count of the A_ADC_Response-PDU shall be zero.

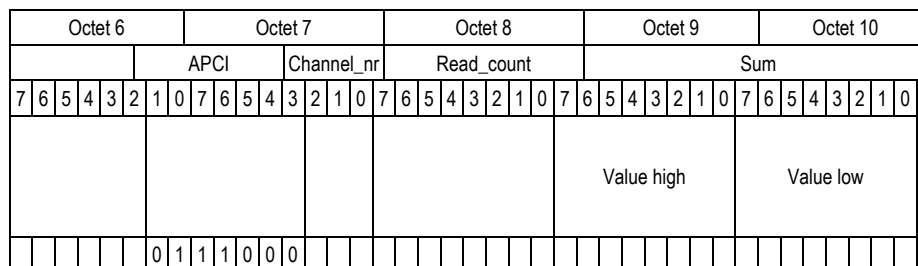


Figure 73 - A_ADC_Response-PDU (example)

The remote Application Layer shall accept the service response and pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_ADC_Response-PDU.

The Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_ADC_Response-PDU to an A_ADC_Read.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_ADC_Read.Acon primitive.

A_ADC_Read.req(ack_request, priority, hop_count_type, ASAP, channel_nr, read_count)

ack_request: Data Link Layer Acknowledge requested or don't care
priority: system, urgent, normal or low priority
hop_count_type: standard NL hop count value
ASAP: local reference of the Service Access Point
channel_nr: The channel_nr of the AD-converter
read_count: number of desired consecutive CPU accesses to the AD-converter

A_ADC_Read.Lcon(ack_request, priority, hop_count_type, ASAP, channel_nr, read_count, a_status)

ack_request: Data Link Layer Acknowledge requested or don't care
priority: system, urgent, normal or low priority
hop_count_type: standard NL hop count value
ASAP: local reference of the Service Access Point
channel_nr: The channel_nr of the AD-converter
read_count: number of desired consecutive CPU accesses to the AD-converter
a_status: ok: A_ADC_Read sent successfully with T_Data_Connected service
not_ok: transmission of the associated T_Data_Connected request frame did not succeed

A_ADC_Read.ind(priority, hop_count_type, ASAP, channel_nr, read_count)

priority: system, urgent, normal or low priority
hop_count_type: standard NL hop count value
ASAP: local reference of the Service Access Point
channel_nr: The channel_nr of the AD-converter
read_count: number of desired consecutive CPU accesses to the AD-converter

A_ADC_Read.res(ack_request, priority, hop_count_type, ASAP, channel_nr, read_count, sum)

ack_request: Data Link Layer Acknowledge requested or don't care
priority: system, urgent, normal or low priority
hop_count_type: standard NL hop count value
ASAP: local reference of the Service Access Point
channel_nr: The channel_nr of the AD-converter
read_count: number of CPU accesses executed to the AD-converter or zero to indicate a problem
sum: sum of AD-converter values

A_ADC_Read.Acon(priority, hop_count_type, ASAP, channel_nr, read_count, sum)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
channel_nr:	The channel_nr of the AD-converter
read_count:	number of CPU accesses executed to the AD-converter or zero to indicate a problem
sum:	sum of AD-converter values

3.5.3 A_Memory_Read-service

The A_Memory_Read.req primitive shall be applied by the user of Application Layer, to read between 1 and 63 octets in the address space of the remote communication controller. The parameter memory_address shall specify the 16 bit start address and number shall contain the number of octets to be read beginning with the start address to increasing addresses. The service shall be confirmed by the remote application process with the contents of the address space.

The local Application Layer shall accept the service request and pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_Memory_Read-PDU.

The remote Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_Memory_Read-PDU to an A_Memory_Read.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_Memory_Read.ind primitive.

octet 6						octet 7						octet 8						octet 9							
						APCI		number				address (high)						address (low)							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0		
						APCI																			
						APCI																			
						APCI																			
						APCI																			
						1	0	0	0																

Figure 74 - A_Memory_Read-PDU (example)

The remote application process shall respond to the A_Memory_Read.ind primitive with an A_Memory_Read.res primitive containing the number of octets read beginning with the start address to increasing addresses. If the remote application process has a problem, e.g., address space unreachable or protected or an illegal number of octets are requested, then the parameter number of the A_Memory_Response-PDU shall be zero and shall contain no data.

octet 6						octet 7						octet 8						octet 9						octet 10 ... n							
						APCI			number			address (high)						address (low)						data							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
						APCI																									
						APCI																									
						APCI																									
						APCI																									
						1	0	0	1																						

Figure 75 - A_Memory_Response-PDU (example)

The remote Application Layer shall accept the service response and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters TSAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_Memory_Response-PDU.

The Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_Memory_Response-PDU to an A_Memory_Read.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_Memory_Read.Acon primitive.

A_Memory_Read.req(ack_request, priority, hop_count_type, ASAP, number, memory_address)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be read beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address

A_Memory_Read.Lcon(ack_request, priority, hop_count_type, ASAP, number, memory_address, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be read beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address
a_status:	ok: A_Memory_Read sent successfully with T_Data_Connected service
	not_ok: transmission of the associated T_Data_Connected request frame did not succeed

A_Memory_Read.ind(priority, hop_count_type, ASAP, number, memory_address)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be read beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address

A_Memory_Read.res(ack_request, priority, hop_count_type, ASAP, number, memory_address, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets read beginning with the start address to increasing addresses, or zero to indicate a problem
memory_address:	specifies the 16 bit start address
data:	the octet(s) read

Error handling

If data are to be read from a protected area or from any logical address that is not associated to physical memory then in the A_Memory_Response-PDU the field number shall be zero and there shall be no field data to indicate an error ¹⁶⁾. The same shall apply if only part of the memory to be read is protected or physically existing.

In addition, the same shall apply if the value of the parameter number is greater than Maximum APDU Length – 3 ¹⁷⁾.

The A_Memory_Write.req primitive shall be applied by the user of Application Layer, to write between 1 octet and 63 octets in the address space of the remote communication controller. The parameter memory_address shall specify the 16 bit start address and the parameter number shall contain the number of octets to be written beginning with the start address to increasing addresses.

The service shall be a confirmed service if Verify Mode is active, otherwise it shall be an acknowledged service.

The local Application Layer shall accept the service request and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_Memory_Write-PDU.

The remote Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_Memory_Write-PDU to an A_Memory_Write.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments TSAP and priority of the A_Memory_Write.ind primitive.

octet 6						octet 7						octet 8						octet 9						octet 10 ... n							
						APCI		number				address (high)				address (low)				data											
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
						APCI	APCI	APCI	APCI																						
						1	0	1	0																						

Figure 76 - A_Memory_Write-PDU (example)

¹⁶⁾ Existing devices may have a different error handling.

¹⁷⁾ The maximal APDU length that shall be supported is specified in the specification of PID_MAX_APDULENGTH in [04].

With inactive Verify Mode the remote application process shall not respond. Instead, the local Application Layer shall map a T_Data_Connected.con primitive to an A_Memory_Write.Lcon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_Memory_Write.Lcon primitive; number, memory_address and data shall be don't care.

With active Verify Mode the remote application process shall respond to the A_Memory_Write.ind primitive with an A_Memory_Write.res primitive containing the requested number of octets of the associated memory area. The value of the associated memory area shall be explicitly read back after writing to it. If the remote application process has a problem, e.g., memory area unreachable or protected or an illegal number of octets are requested, then the parameter number shall be zero and shall contain no data.

The remote Application Layer shall accept the service response and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_Memory_Response-PDU.

The Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_Memory_Response-PDU to an A_Memory_Write.Acon primitive if an A_Memory_Write-PDU has been sent before over this connection. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_Memory_Write.Acon primitive.

A_Memory_Write.req(ack_request, priority, hop_count_type, TSAP, number, memory_address, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be written beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address
data:	the octet(s) to be written

A_Memory_Write.Lcon(ack_request, priority, hop_count_type, ASAP, number, memory_address, data, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be written beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address
data:	the octet(s) to be written or no data
a_status:	ok: A_Memory_Write sent successfully with T_Data_Connected service
	not_ok: transmission of the associated T_Data_Connected request frame did not succeed

A_Memory_Write.ind(priority, hop_count_type, TSAP, number, memory_address, data)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be written beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address
data:	the octet(s) to be written

A_Memory_Write.res(ack_request, priority, hop_count_type, ASAP, number, memory_address, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets written beginning with the start address to increasing addresses, or zero to indicate a problem
memory_address:	specifies the 16 bit start address
data:	the octet(s) read back or no data

A_Memory_Write.Acon(priority, hop_count_type, ASAP, number, memory_address, data)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets written beginning with the start address to increasing addresses, or zero to indicate a problem
memory_address:	specifies the 16 bit start address
data:	the octet(s) read back or no data

Error Handling

If data are to be written to a protected area from any logical address that is not associated to physical memory then the service indication shall be ignored. In any case physical memory shall not be addressable via different logical addresses. If only a part of the addressed memory is protected or does not exist, then the complete write operation shall fail.

In addition, the remote Application Layer shall ignore the A_Memory_Write.ind if the value of the parameter “number” is greater than Maximum APDU Length – 3 ²⁰⁾.

In addition, the remote Application Layer shall ignore the A_Memory_Write.ind if the parameter *number* is not equal to the number of received data octets.

If Verify Mode is active, then in case of a failed write operation the field *number* of the A_Memory_Response-PDU shall be zero and there shall be no field data to indicate an error ²¹⁾.

²⁰⁾ The maximal APDU length that shall be supported is specified in the specification of PID_MAX_APDULENGTH in [04].

²¹⁾ Existing devices may have a different error-handling.

3.5.5 A_MemoryBit_Write-service

The A_MemoryBit_Write.req primitive shall be applied by the user of Application Layer, to modify between 1 bit and 40 bits in a contiguous block of up to 5 octets in the address space of the remote communication controller. The parameter memory_address shall specify the 16 bit start address and number shall contain the number of octets to be modified beginning with the start address to increasing addresses. The A_MemoryBit_Write shall allow to

- set individual bits of the contiguous block to zero, and
- set individual bits of the contiguous block to one, and
- leave individual bits of the contiguous block unmodified, and
- invert individual bits of the contiguous block,

using the parameters and_data and xor_data. Both parameters shall have the same number of octets as the contiguous block indicated in the parameter number. The resulting value for each individual bit in the contiguous block shall be computed using the two associated bits of and_data and xor_data with the following function (see Figure 77):

$$\text{result_bit}(i) = (\text{and_data_bit}(i) \text{ AND } \text{block_bit}(i)) \text{ XOR } \text{xor_data_bit}(i)$$

and_data_bit(i)	xor_data_bit(i)	result_bit(i)
0	0	0
0	1	1
1	0	block_bit(i)
1	1	NOT block_bit(i)

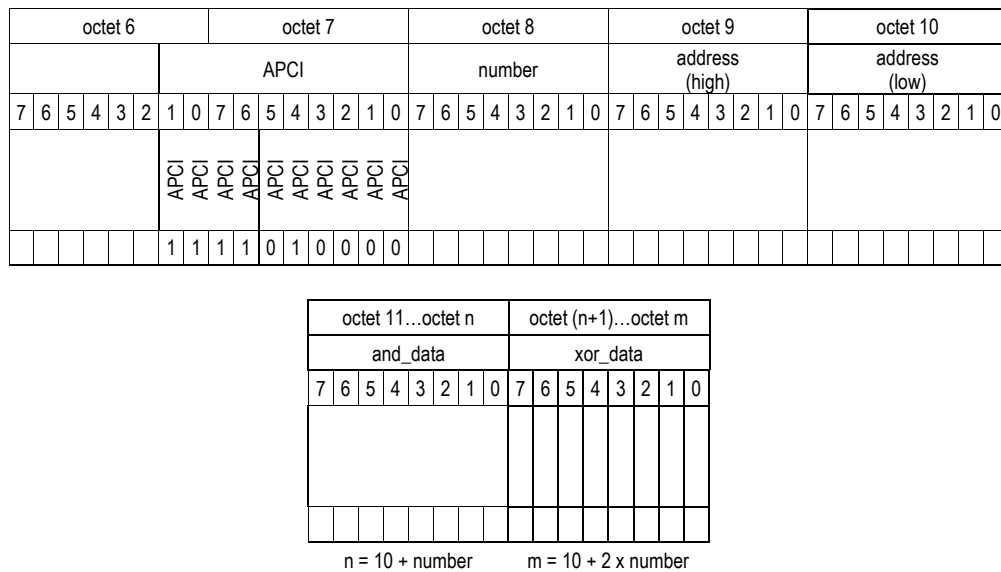
Figure 77 - Function Table for A_MemoryBit_Write-services (example)

The service shall be a confirmed service if Verify Mode is active, otherwise it shall be an acknowledged service.

The local Application Layer shall accept the service request and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_MemoryBit_Write-PDU.

With inactive Verify Mode the remote application process shall not respond. Instead, the local Application Layer shall map a T_Data_Connected.con primitive to an A_MemoryBit_Write.Lcon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_MemoryBit_Write.Lcon primitive; number, memory_address and data shall be don't care.

With active Verify Mode the remote application process shall respond to the A_MemoryBit_Write.ind primitive with an A_MemoryBit_Write.res primitive containing the requested number of octets of the associated memory area. The value of the associated memory area shall be explicitly read back after writing to it. If the remote application process has a problem, e.g., memory area unreachable or protected or an illegal number of octets are requested, then the parameter number shall be zero and shall contain no data.

**Figure 78 - A_MemoryBit_Write-PDU (example)**

The remote Application Layer shall accept the service response and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters TSAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU is an A_Memory_Response-PDU.

The Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_Memory_Response-PDU to an A_MemoryBit_Write.Acon primitive if an A_MemoryBit_Write-PDU is sent before over this connection. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_MemoryBit_Write.Acon primitive.

A_MemoryBit_Write.req(ack_request, priority, hop_count_type, ASAP, number, memory_address, and_data, xor_data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be modified beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address
and_data:	see Figure 77
xor_data:	see Figure 77

A_MemoryBit_Write.Lcon(ack_request, priority, hop_count_type, ASAP, number, memory_address, and_data, xor_data, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be modified beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address
and_data:	see Figure 77
xor_data:	see Figure 77
a_status: ok:	A_MemoryBit_Write sent successfully with T_Data_Connected service
not_ok:	transmission of the associated T_Data_Connected request frame did not succeed

A_MemoryBit_Write.ind(priority, hop_count_type, ASAP, number, memory_address, and_data, xor_data)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be modified beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address
and_data:	see Figure 77
xor_data:	see Figure 77

A_MemoryBit_Write.res(ASAP, priority, number, memory_address, data)

ASAP:	local reference of the Service Access Point
priority:	system, urgent, normal or low priority
number:	number of octets modified beginning with the start address to increasing addresses, or zero to indicate a problem
memory_address:	specifies the 16 bit start address
data:	the octet(s) read back or no data

Error handling

If data are to be written to a protected area or from any logical address that is not associated to physical memory then the service indication shall be ignored. In any case physical memory shall not be addressable via different logical addresses. If only a part of the addressed memory is protected or does not exist, then the complete write operation shall fail. If Verify Mode is active, then in case of a failed write operation no data shall be returned to indicate an error ²²⁾.

²²⁾ Existing devices may have a different error-handling.

3.5.6 A_UserData

3.5.6.1 Definition

The A_UserData-service shall be used for Application Device Management. The Application Device Management is that part of the Device Management that is implemented in the application.

The Application Device Management uses a logical address-space of 1 Mb. The mapping from the logical address space to the physical address and vice versa is task of the Application Device Management.

The Application Device Management may also influence directly or indirectly the application program. But this has to be defined individually for each device.

All services provided by the Application Device Management are based on the A_USER_DATA-messages and the corresponding services.

3.5.6.2 A_UserMemory_Read-service

The A_UserMemory_Read.req primitive shall be applied by the user of Application Layer, to read between 1 octet and 15 octets in the address space of the remote application controller. The parameter memory_address shall specify the 20 bit start address (4 bit address extension + 8 bit address high + 8 bit address low) and the parameter number shall contain the number of octets to be read beginning with the start address to increasing addresses. The service shall be confirmed by the remote application process with the contents of the address space.

The local Application Layer shall accept the service request and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_UserMemory_Read-PDU.

The remote Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_UserMemory_Read-PDU to an A_UserMemory_Read.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_UserMemory_Read.ind primitive.

octet 6										octet 7										octet 8										octet 9										octet 10									
										APCI										address extension					number					address (high)										address (low)									
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0										
										APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																															
							1	0	1	1	0	0	0	0	0																																		

The remote Application Layer shall accept the service response and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_UserMemory_Response-PDU.

The Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_UserMemory_Response-PDU to an A_UserMemory_Read.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_UserMemory_Read.Acon primitive.

A_UserMemory_Read.req(ack_request, priority, hop_count_type, ASAP, number, memory_address)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be read beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address

A_UserMemory_Read.Lcon(ack_request, priority, hop_count_type, ASAP, number, memory_address, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be read beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address
a_status:	ok: A_UserMemory_Read sent successfully with T_Data_Connected service
	not_ok: transmission of the associated T_Data_Connected request frame did not succeed

A_UserMemory_Read.ind(priority, hop_count_type, ASAP, number, memory_address)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be read beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address

A_UserMemory_Read.res(ack_request, priority, hop_count_type, ASAP, number, memory_address, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets read beginning with the start address to increasing addresses, or zero to indicate a problem
memory_address:	specifies the 16 bit start address
data:	the octet(s) read

A_UserMemory_Read.Acon(priority, hop_count_type, ASAP, number, memory_address, data)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets read beginning with the start address to increasing addresses, or zero to indicate a problem
memory_address:	specifies the 16 bit start address
data:	the octet(s) read

Error handling

If data are to be read from a protected area or from any logical address that is not associated to physical memory then in the A_UserMemory_Response-PDU the field *number* shall be zero and there shall be no field *data* to indicate an error. The same shall apply if only part of the memory to be read is protected or physically existing.

In addition, the same shall apply if the value of the parameter *number* is greater than Maximum APDU Length – 4 ²³⁾.

3.5.6.3 A_UserMemory_Write-service

The A_UserMemory_Write.req primitive shall be applied by the user of Application Layer, to write between 1 octet and 15 octets in the address space of the remote application controller. The parameter *memory_address* shall specify the 20 bit start address (4 bit address extension + 8 bit address high + 8 bit address low) and the parameter *number* shall contain the number of octets to be written beginning with the start address to increasing addresses.

The service shall be a confirmed service if Verify Mode is active, otherwise it shall be an acknowledged service.

The local Application Layer shall accept the service request and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_UserMemory_Write-PDU.

The remote Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_UserMemory_Write-PDU to an A_UserMemory_Write.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_UserMemory_Write.ind primitive.

²³⁾ The maximal APDU length that shall be supported is specified in the specification of PID_MAX_APDULENGTH in [04].

octet 6								octet 7								octet 8								octet 9								octet 10								octet 11 ... n							
																address extension				number				address (high)				address (low)				data															
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	8	7	6	5	4	3	2	1								
							APC	APC	APC	APC	APC	APC	APC	APC	APC																																
							1	0	1	1	0	0	0	0	1	0																															

Figure 81 - A_UserMemory_Write-PDU (example)

With inactive Verify Mode the remote application process shall not respond. Instead, the local Application Layer shall map a T_Data_Connected.con primitive to an A_UserMemory_Write.Lcon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_UserMemory_Write.Lcon primitive; number, memory_address and data shall be don't care.

With active Verify Mode the remote application process shall respond to the A_UserMemory_Write.ind primitive with an A_UserMemory_Write.res primitive containing the requested number of octets of the associated memory area. The value of the associated memory area shall be explicitly read back after writing to it. If the remote application process has a problem, e.g., memory area unreachable or protected or an illegal number of octets are requested, then the parameter number shall be zero and shall contain no data.

The remote Application Layer shall accept the service response and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters TSAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_UserMemory_Response-PDU.

The Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_UserMemory_Response-PDU to an A_UserMemory_Write.Acon primitive if an A_UserMemory_Write-PDU has been sent before over this connection. The arguments TSAP and priority shall be mapped to the corresponding arguments TSAP and priority of the A_UserMemory_Write.Acon primitive.

A_UserMemory_Write.req(ack_request, priority, hop_count_type, ASAP, number, memory_address, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be written beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address
data:	the octet(s) to be written

A_UserMemory_Write.Lcon(ack_request, priority, hop_count_type, ASAP, number, memory_address, data, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be written beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address
data:	the octet(s) to be read back or no data
a_status:	ok: A_UserMemory_Write sent successfully with T_Data_Connected service
	not_ok: transmission of the associated T_Data_Connected request frame did not succeed

A_UserMemory_Write.ind(priority, hop_count_type, ASAP, number, memory_address, data)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be written beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address
data:	the octet(s) to be written

A_UserMemory_Write.res(ack_request, priority, hop_count_type, TSAP, number, memory_address, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets written beginning with the start address to increasing addresses, or zero to indicate a problem
memory_address:	specifies the 16 bit start address
data:	the octet(s) read back or no data

A_UserMemory_Write.Acon(priority, hop_count_type, TSAP, number, memory_address, data)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets written beginning with the start address to increasing addresses, or zero to indicate a problem
memory_address:	specifies the 16 bit start address
data:	the octet(s) read back or no data

Error handling

If data are to be written to a protected area or from any logical address that is not associated to physical memory then the service indication shall be ignored. In any case physical memory shall not be addressable via different logical addresses. If only a part of the addressed memory is protected or does not exist, then the complete write operation shall fail.

In addition, the remote Application Layer shall ignore the A_Memory_Write.ind if the value of the parameter *number* is greater than Maximum APDU Length – 4 ²⁴⁾.

In addition, the remote Application Layer shall ignore the A_Memory_Write.ind if the value of the parameter *number* is not equal to the number of received data octets.

If Verify Mode is active, then in case of a failed write operation the field *number* of the A_UserMemory_Response-PDU shall be zero and there shall be no field *data* to indicate an error.

3.5.6.4 A_UserMemoryBit_Write-service

The A_UserMemoryBit_Write.req primitive shall be applied by the user of Application Layer, to modify between 1 bit and 40 bits in a contiguous block of up to 5 octets in the address space of the remote application controller. The parameter *memory_address* shall specify the 16 bit start address and *number* shall contain the number of octets to be modified beginning with the start address to increasing addresses. The A_UserMemoryBit_Write shall allow to

- set individual bits of the contiguous block to zero, and
- set individual bits of the contiguous block to one, and
- leave individual bits of the contiguous block unmodified, and
- invert individual bits of the contiguous block

using the parameters *and_data* and *xor_data*. Both parameters shall have the same number of octets as the contiguous block indicated in the parameter *number*. The resulting value for each individual bit in the contiguous block shall be computed using the two associated bits of *and_data* and *xor_data* with the following function (Figure 82):

$$\text{result_bit}(i) = (\text{and_data_bit}(i) \text{ AND } \text{block_bit}(i)) \text{ XOR } \text{xor_data_bit}(i)$$

and_data_bit(i)	xor_data_bit(i)	result_bit(i)
0	0	0
0	1	1
1	0	block_bit(i)
1	1	NOT block_bit(i)

Figure 82 - Function Table for A_UserMemoryBit_Write-service

The service shall be a confirmed service if Verify Mode is active, otherwise it shall be an acknowledged service.

The local Application Layer shall accept the service request and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_UserMemoryBit_Write-PDU.

The remote Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_UserMemoryBit_Write-PDU to an A_UserMemoryBit_Write.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_UserMemoryBit_Write.ind primitive.

²⁴⁾ The maximal APDU length that shall be supported is specified in the specification of PID_MAX_APDULENGTH in [04].

octet 6								octet 7								octet 8								octet 9								octet 10								octet 11 ... octet n								oct. (n+1) ... oct. (m)																																																																							
																number								address (high)								address (low)								and_data								xor_data																																																																							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0																																																																
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															
								APCI																																																																																																															

Figure 83 - A_UserMemoryBit_Write-PDU (example)

With inactive Verify Mode the remote application process shall not respond. Instead, the local Application Layer shall map a T_Data_Connected.con primitive to an A_UserMemoryBit_Write.Lcon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_UserMemoryBit_Write.Lcon primitive; number, memory_address and data shall be don't care.

With active Verify Mode the remote application process shall respond to the A_UserMemoryBit_Write.ind primitive with an A_UserMemoryBit_Write.res primitive containing the requested number of octets of the associated memory area. The value of the associated memory area shall be explicitly read back after writing to it. If the remote application process has a problem, e.g., memory area unreachable or protected or an illegal number of octets are requested, then the parameter number shall be zero and shall contain no data.

The remote Application Layer shall accept the service response and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters TSAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_User-Memory Response-PDU.

The Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_UserMemory_Response-PDU to an A_UserMemoryBit_Write.Acon primitive if an A_UserMemoryBit_Write-PDU is sent before over this connection. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_UserMemoryBit_Write.Acon primitive.

```
A_UserMemoryBit_Write.req(ack_request, priority, hop_count_type, ASAP, number, memory_address,
                           and data, xor data)
```

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be modified beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address
and_data:	see Figure 82
xor_data:	see Figure 82

A_UserMemoryBit_Write.Lcon(ack_request, priority, hop_count_type, ASAP, number, memory_address, and_data, xor_data, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be modified beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address
and_data:	see Figure 82
xor_data:	see Figure 82
a_status: ok:	A_UserMemoryBit_Write sent successfully with T_Data_Connected service
not_ok:	transmission of the associated T_Data_Connected request frame did not succeed

A_UserMemoryBit_Write.ind(priority, hop_count_type, ASAP, number, memory_address, and_data, xor_data)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets to be modified beginning with the start address to increasing addresses
memory_address:	specifies the 16 bit start address
and_data:	see Figure 82
xor_data:	see Figure 82

A_UserMemoryBit_Write.res(ack_request, priority, hop_count_type, ASAP, number, memory_address, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets modified beginning with the start address to increasing addresses, or zero to indicate a problem
memory_address:	specifies the 16 bit start address
data:	the octet(s) read back or no data

A_UserMemoryBit_Write.Acon(priority, hop_count_type, ASAP, number, memory_address, data)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
number:	number of octets modified beginning with the start address to increasing addresses, or zero to indicate a problem
memory_address:	specifies the 16 bit start address
data:	the octet(s) read back or no data

Error handling

If data are to be written to a protected area or from any logical address that is not associated to physical memory then the service indication shall be ignored. In any case physical memory shall not be addressable via different logical addresses. If only a part of the addressed memory is protected or does not exist, then the complete write operation shall fail. If Verify Mode is active, then in case of a failed write operation no data shall be returned to indicate an error.

3.5.6.5 A_UserManufacturerInfo_Read-service

The A_UserManufacturerInfo_Read.req primitive shall be applied by the user of Application Layer, to read manufacturer information in a communication partner. The manufacturer information shall consist of three octets. Octet zero shall indicate the manufacturer identification of the device. Octets one and two shall be manufacturer specific. The service shall be confirmed by the remote application process.

The local Application Layer shall accept the service request and pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_UserManufacturer-Info_Read-PDU.

The remote Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_UserManufacturerInfo_Read-PDU to an A_UserManufacturerInfo_Read.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_UserManufacturerInfo_Read.ind primitive.

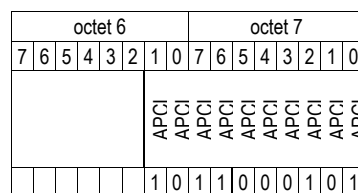


Figure 84 - A_UserManufacturerInfo_Read-PDU (example)

The remote application process shall respond to the A_UserManufacturerInfo_Read.ind primitive with an A_UserManufacturerInfo_Read.res primitive containing the manufacturer information.

NOTE 13 Because of the encoding of manufacturer_id in a single octet, the encoding space is limited to 255. Unless noted differently, this service is thus not recommended for use in new implementations.

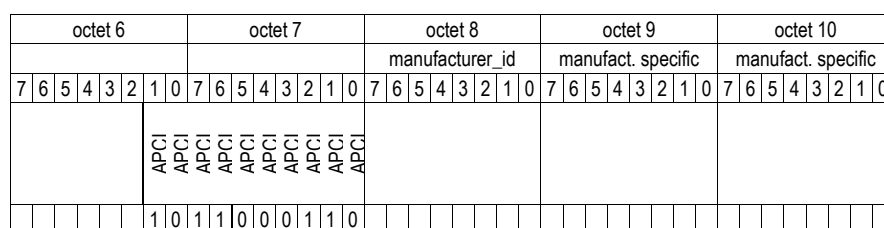


Figure 85 - A_UserManufacturerInfo_Response-PDU (example)

The remote Application Layer shall accept the service response and pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_UserManufacturerInfo_Response-PDU.

The Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_UserManufacturerInfo_Response-PDU to an A_UserManufacturerInfo_Read.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_UserManufacturerInfo_Read.Acon primitive.

Limitations

In the response-PDU of this service only one single octet is specified for the KNX manufacturer code. This does not allow transmitting a KNX manufacturer code with value above 255. Therefore, use of this service is not recommended for new stack implementations.

A_UserManufacturerInfo_Read.req(ack_request, priority, hop_count_type, ASAP)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point

A_UserManufacturerInfo_Read.Lcon(ack_request, priority, hop_count_type, ASAP, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
a_status:	ok: A_UserManufacturerInfo_Read sent successfully with T_Data_Connected service
	not_ok: transmission of the associated T_Data_Connected request frame did not succeed

A_UserManufacturerInfo_Read.ind(priority, hop_count_type, ASAP)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point

A_UserManufacturerInfo_Read.res(ack_request, priority, hop_count_type, ASAP, mfact_info)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
mfact_info:	three octets manufacturer information

A_UserManufacturerInfo_Read.Acon(priority, hop_count_type, ASAP, mfact_info)

priority: system, urgent, normal or low priority

hop_count_type: standard NL hop count value

ASAP: local reference of the Service Access Point

mfact_info: three octets manufacturer information

3.5.7 A_Authorize_Request-service

The A_Authorize_Request.req primitive shall be applied by the user of Application Layer, to inform the communication partner about the key that shall be four octets long and of data type unsigned32. The remote partner shall know a number of valid keys and shall be able to associate a valid key to an access level. This access level shall be stored as the current access level of this partner and shall be sent back in an A_Authorize_Response-PDU. Access levels (unsigned8) between 0 (maximum level, i.e., maximum access rights) and 3 (minimum level, i.e. minimum access rights) or 0 (maximum level, i.e. maximum access rights) and 15 (minimum level, i.e. minimum access rights) are allowed. If the communication partner doesn't authorize himself or if the key is not a valid key, then the current access level for this communication partner shall be set as specified under "Error and exception handling" below.

The current access level may be used by the remote application process to decide whether or not a communication partner is allowed to request a certain read or write operation.

The remote user may associate different keys to different access levels and handle following services on this connection depending on the current access level.

A current access level shall be valid until the connection is released or a new key is indicated with the A_Authorize_Request service.

The local Application Layer shall accept the service request and pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_Authorize_Request-PDU.

The remote Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_Authorize_Request-PDU to an A_Authorize_Request.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_Authorize_Request.ind primitive.

octet 6								octet 7								octet 8								octet 9								---								octet 12							
																must be 0																key															
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0									7	6	5	4	3	2	1	0
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																																
						1	1	1	1	0	1	0	0	0	1	0	0	0	0	0	0	0	0																								

Figure 86 - A_Authorize_Request-PDU (example)

The remote application process shall respond to the A_Authorize_Request.ind primitive with an A_Authorize_Request.res primitive containing the associated access level.

octet 6								octet 7								octet 8							
																level							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
								APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI								
						1	1	1	1	0	1	0	0	1	0								

Figure 87 - A_Authorize_Response-PDU (example)

The remote Application Layer shall accept the service response and pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_Authorize_Response-PDU.

The Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_Authorize_Response-PDU to an A_Authorize_Request.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_Authorize_Request.Acon primitive.

Error and exception handling

- If the Remote Management supports authorization and if the communication partner does not authorize itself, the Remote Management shall select the maximum access level protected with FFFFFFFFh as the current access level.
- If the Remote Management supports authorization and if the communication partner authorizes itself with an invalid key, the Remote Management shall select the minimal access level (this is level 3 or level 15) as the current access level.

A_Authorize_Request.req(ack_request, priority, hop_count_type, ASAP, key)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
key:	the key of the requester

A_Authorize_Request.ind(priority, hop_count_type, ASAP, key)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
key:	the key of the requester

A_Authorize_Request.res(ack_request, priority, hop_count_type, ASAP, level)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
level:	the granted access level to the requester

A_Authorize_Request.Acon(priority, hop_count_type, ASAP, level)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
level:	the granted access level to the requester

3.5.8 A_Key_Write-service

The A_Key_Write.req primitive shall be applied by the user of Application Layer, to modify or delete the key associated to a certain access level in the communication partner. The parameter level of the A_Key_Write.req primitive shall indicate the access level that shall be modified; the parameter key shall indicate the new key value.

Every device shall be able to handle exactly one key per access level. The number of access levels supported by a device is Profile dependent.

Key	Access Level
Key for level 0	0
Key for level 1	1
...	...
Key for Level N-1	N-1
None	N (free access)

Figure 88 - Association Table of Keys to Access Levels

If the key indicated in the A_Key_Write.ind primitive is FFFFFFFFh, then the corresponding key entry in the association table of keys to access levels shall be set to invalid, this is, then there shall be no key associated to the corresponding level any more. The current access level shall be less or equal to the access level indicated in the A_Key_Write.ind primitive, otherwise the remote application process shall return FFh in the A_Key_Response-PDU. In all other cases the remote application process shall store the indicated key in the corresponding entry of the association table of keys to access levels and shall respond to the A_Key_Write.ind primitive with an A_Key_Write.res primitive containing the access level set for the corresponding key.

The local Application Layer shall accept the service request and pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_Key_Write-PDU.

The remote Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_Key_Write-PDU to an A_Key_Write.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_Key_Write.ind primitive.

octet 6								octet 7								octet 8								octet 9								---								octet 12							
																level								key																							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0									7	6	5	4	3	2	1	0
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																																
						1	1	1	1	0	1	0	0	0	1	1	0	0	0	0	0	0	0																								

Figure 89 - A_Key_Write-PDU (example)

The remote user shall process the key as described above and shall respond with the A_Key_Write.res primitive.

octet 6								octet 7								octet 8							
												number				level							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
								APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI								
								1	1	1	1	0	1	0	1	0							

Figure 90 - A_Key_Response-PDU (example)

The remote Application Layer shall accept the service response and pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_Key_Response-PDU.

The Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_Key_Response-PDU to an A_Key_Write.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_Key_Write.Acon primitive.

A_Key_Write.req(ack_request, priority, hop_count_type, ASAP, level, key)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
level:	the access level for which the key shall be modified
key:	the new value of the key or FFFFFFFFh to delete the key

A_Key_Write.ind(priority, hop_count_type, ASAP, level, key)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
level:	the access level for which the key shall be modified
key:	the new value of the key or FFFFFFFFh to delete the key

A_Key_Write.res(ack_request, priority, hop_count_type, ASAP, level)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
level:	the access level for which the associated key has been modified, or the minimum access level if it hasn't been modified

A_Key_Write.Acon(priority, hop_count_type, ASAP, level)

priority:	system, urgent, normal or low priority
hop_count_type:	standard NL hop count value
ASAP:	local reference of the Service Access Point
level:	the access level for which the associated key has been modified, or the minimum access level if it hasn't been modified

3.6 Coupler specific Application Layer Services on Point-to-Point connection oriented Communication Mode

3.6.1 A_FilterTable_Open-service

The A_FilterTable_Open.req primitive shall be applied by the user of Application Layer, to open the access to the Filter Table of the remote communication controller. It shall be used before the services “A_FilterTable_Read” or “A_FilterTable_Write”²⁵⁾ are used. There is no additional parameter passed with this service.

The service shall be an acknowledged service, if Verify Mode is active or not.

The local Application Layer shall accept the service request and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_FilterTable_Open-PDU.

The remote Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_FilterTable_Open-PDU to an A_FilterTable_Open.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments TSAP and priority of the A_FilterTable_Open.ind primitive.

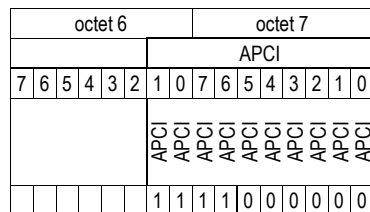


Figure 91 - A_FilterTable_Open-PDU

With active or inactive Verify Mode the remote application process shall not respond. Instead, the local Application Layer shall map a T_Data_Connected.con primitive to an A_FilterTable_Open.Lcon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_FilterTable_Open.Lcon primitive.

A_FilterTable_Open.req(ack_request, priority, hop_count_type, TSAP)

ack_request: Data Link Layer Acknowledge requested or don't care
 priority: system, urgent, normal or low priority
 hop_count_type: hop count 7 or standard
 ASAP: local reference of the Service Access Point

A_FilterTable_Open.Lcon(ack_request, priority, hop_count_type, ASAP, a_status)

ack_request: Data Link Layer Acknowledge requested or don't care
 priority: system, urgent, normal or low priority
 hop_count_type: hop count 7 or standard
 ASAP: local reference of the Service Access Point
 a_status: ok: A_FilterTable_Open sent successfully with T_Data_Connected service
 not_ok: transmission of the associated T_Data_Connected request Frame did not succeed

²⁵⁾ Legacy implementations need this command. Newer implementations however might ignore this command

A_FilterTable_Open.ind(priority, hop_count_type, TSAP)

priority: system, urgent, normal or low priority

hop_count_type: hop count 7 or standard

ASAP: local reference of the Service Access Point

Error handling

There is no error handling defined to this service. If the remote Application Layer cannot execute the command, there will be no dedicated reaction generated in the remote Application Layer. Consequently, the client using this command can only recognize error conditions in the ongoing Management Procedure. For this the general exception behaviour shall be used.

3.6.2 A_FilterTable_Read-service

The A_FilterTable_Read.req primitive shall be applied by the user of Application Layer, to read between 1 octet and 254 octets in the Filter Table of the remote communication controller. The parameter filter_table_address shall specify the 16 bit start address and number shall contain the number of octets to be read beginning with the start address to increasing addresses. The service shall be confirmed by the remote application process with the contents of the address space.

Before this service can be used, the service “A_FilterTable_Open” shall be executed ²⁶⁾.

The local Application Layer shall accept the service request and pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_FilterTable_Read-PDU.

The remote Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_FilterTable_Read-PDU to an A_FilterTable_Read.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_FilterTable_Read.ind primitive.

octet 6						octet 7						octet 8						octet 9						octet 10									
						APCI						number						filter_table_address (high)						filter_table_address (low)									
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0		
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																		
						1	1	1	1	0	0	0	0	0	1																		

Figure 92 - A_FilterTable_Read-PDU (example)

The remote application process shall respond to the A_FilterTable_Read.ind primitive with an A_FilterTable_Read.res primitive containing the number of octets read beginning with the start address to increasing addresses. If the remote application process has a problem, e.g., address space unreachable or protected or an illegal number of octets are requested, then the parameter number of the A_FilterTable_Response-PDU shall be zero and shall contain no data.

octet 6						octet 7						octet 8						octet 9						octet 10						octet 11 ... n									
						APCI						number						filter_table_address (high)						filter_table_address (low)						data									
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																									
						1	1	1	1	0	0	0	0	1	0																								

Figure 93 - A_FilterTable_Response-PDU (example)

²⁶⁾ Legacy implementations need this service. Newer implementations however may ignore this service.

The remote Application Layer shall accept the service response and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters TSAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_FilterTable_Response-PDU.

The Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_FilterTable_Response-PDU to an A_FilterTable_Read.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_FilterTable_Read.Acon primitive.

A_FilterTable_Read.req(ack_request, priority, hop_count_type, ASAP, number, filter_table_address)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	hop count 7 or standard
ASAP:	local reference of the Service Access Point
number:	number of octets to be read beginning with the start address to increasing addresses
filter_table_address:	16 bit start address

A_FilterTable_Read.Lcon(ack_request, priority, hop_count_type, ASAP, number, filter_table_address, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	hop count 7 or standard
ASAP:	local reference of the Service Access Point
number:	number of octets to be read beginning with the start address to increasing addresses
filter_table_address:	specifies the 16 bit start address
a_status:	ok: A_FilterTable_Read sent successfully with T_Data_Connected service
	not_ok: transmission of the associated T_Data_Connected request Frame did not succeed

A_FilterTable_Read.ind(priority, hop_count_type, ASAP, number, filter_table_address)

priority:	system, urgent, normal or low priority
hop_count_type:	hop count 7 or standard
ASAP:	local reference of the Service Access Point
number:	number of octets to be read beginning with the start address to increasing addresses
filter_table_address:	16 bit start address

A_FilterTable_Read.res(ack_request, priority, hop_count_type, ASAP, number, Filter Table address, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	hop count 7 or standard
ASAP:	local reference of the Service Access Point
number:	number of octets read beginning with the start address to increasing addresses, or zero to indicate a problem
filter_table_address:	16 bit start address
data:	the octet(s) read

Error handling

If data are to be read from a protected area or from any logical address that is not associated to physical memory then in the A_FilterTable_Response-PDU the field *number* shall be zero and there shall be no field data to indicate an error²⁷⁾. The same shall apply if only part of the Filter Table to be read is protected or physically existing.

In addition, the same shall apply if the value of the parameter *number* is greater than Maximum APDU Length $- 4^{28}$).

3.6.3 A FilterTable_Write-service

The `A_FilterTable_Write.req` primitive shall be applied by the user of Application Layer, to write between 1 octet and 254 octets in the Filter Table of the remote communication controller. The parameter *filter_table_address* shall specify the 16 bit start address and the parameter *number* shall contain the number of octets to be written beginning with the start address to increasing addresses.

Before this service can be used, the service “A FilterTable Open” shall be executed ²⁹⁾.

The service shall be a confirmed service if Verify Mode is active, otherwise it shall be an acknowledged service.

The local Application Layer shall accept the service request and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_FilterTable_Write-PDU.

The remote Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_Filter-Table_Write-PDU to an A_FilterTable_Write.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments TSAP and priority of the A_FilterTable_Write.ind primitive.

[illegible]

Figure 94 - A FilterTable Write-PDU (example)

27) Existing devices may have a different error handling.

28) The maximal APDU length that shall be supported is specified in the specification of PID_MAX_APDU_LENGTH in [04].

²⁹⁾ Legacy implementations need this service. Newer implementations however may ignore this service.

With inactive Verify Mode the remote application process shall not respond. Instead, the local Application Layer shall map a T_Data_Connected.con primitive to an A_FilterTable_Write.Lcon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_FilterTable_Write.Lcon primitive; number, Filter Table address and data shall be don't care.

With active Verify Mode the remote application process shall respond to the A_FilterTable_Write.ind primitive with an A_FilterTable_Write.res primitive containing the requested number of octets of the associated Filter Table area. The value of the associated Filter Table area shall be explicitly read back after writing to it. If the remote application process has a problem, e.g., Filter Table area unreachable or protected or an illegal number of octets are requested, then the parameter *number* shall be zero and there shall be no field data to indicate an error.

The remote Application Layer shall accept the service response and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_FilterTable_Response-PDU.

The Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_FilterTable_Response-PDU to an A_FilterTable_Write.Acon primitive if an A_FilterTable_Write-PDU has been sent before over this connection. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_FilterTable_Write.Acon primitive.

A_FilterTable_Write.req(ack_request, priority, hop_count_type, TSAP, number, Filter Table address, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	hop count 7 or standard
ASAP:	local reference of the Service Access Point
number:	number of octets to be written beginning with the start address to increasing addresses
filter_table_address:	the 16 bit start address
data:	the octet(s) to be written

A_FilterTable_Write.Lcon(ack_request, priority, hop_count_type, ASAP, number, Filter Table address, data, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	hop count 7 or standard
ASAP:	local reference of the Service Access Point
number:	number of octets to be written beginning with the start address to increasing addresses
filter_table_address:	the 16 bit start address
data:	the octet(s) to be written or no data
a_status:	ok: A_FilterTable_Write sent successfully with T_Data_Connected service
	not_ok: transmission of the associated T_Data_Connected request Frame did not succeed

A_FilterTable_Write.ind(priority, hop_count_type, TSAP, number, Filter Table address, data)

priority: system, urgent, normal or low priority
 hop_count_type: hop count 7 or standard
 ASAP: local reference of the Service Access Point
 number: number of octets to be written beginning with the start address to increasing addresses
 filter_table_address: the 16 bit start address
 data: the octet(s) to be written

A_FilterTable_Write.res(ack_request, priority, hop_count_type, ASAP, number, Filter Table address, data)

ack_request: Data Link Layer Acknowledge requested or don't care
 priority: system, urgent, normal or low priority
 hop_count_type: hop count 7 or standard
 ASAP: local reference of the Service Access Point
 number: number of octets written beginning with the start address to increasing addresses, or zero to indicate a problem
 filter_table_address: the 16 bit start address
 data: the octet(s) read back or no data

A_FilterTable_Write.Acon(priority, hop_count_type, ASAP, number, Filter Table address, data)

priority: system, urgent, normal or low priority
 hop_count_type: hop count 7 or standard
 ASAP: local reference of the Service Access Point
 number: number of octets written beginning with the start address to increasing addresses, or zero to indicate a problem
 filter_table_address: the 16 bit start address
 data: the octet(s) read back or no data

Error handling

If data are to be written to a protected area from any logical address that is not associated to physical memory then the service indication shall be ignored. In any case physical memory shall not be addressable via different logical addresses. If only a part of the addressed Filter Table is protected or does not exist, then the complete write operation shall fail.

In addition, the remote Application Layer shall ignore the A_FilterTable_Write.ind if the value of the parameter *number* is greater than Maximum APDU Length – 4 ³⁰⁾.

In addition, the remote Application Layer shall ignore the A_FilterTable_Write.ind if the parameter *number* is not equal to the number of received data octets.

If Verify Mode is active, then in case of a failed write operation the field *number* of the A_FilterTable_Response-PDU shall be zero and there shall be no field data to indicate an error ³¹⁾.

³⁰⁾ The maximal APDU length that shall be supported is specified in the specification of PID_MAX_APDU_LENGTH in [04].

³¹⁾ Existing devices may have a different error-handling.

3.6.4 A_RouterMemory_Read-service

The A_RouterMemory_Read.req primitive shall be applied by the user of Application Layer, to read between 1 octet and 254 octets in the memory of the second controller of the remote communication controller. The parameter memory_address shall specify the 16 bit start address and number shall contain the number of octets to be read beginning with the start address to increasing addresses. The service shall be confirmed by the remote application process with the contents of the address space.

The local Application Layer shall accept the service request and pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_RouterMemory_Read-PDU.

The remote Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_RouterMemory_Read-PDU to an A_RouterMemory_Read.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_RouterMemory_Read.ind primitive.

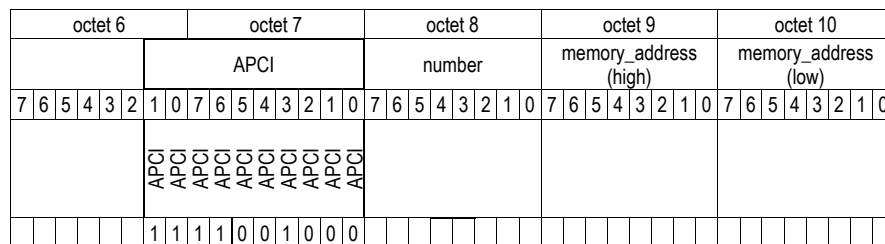


Figure 95 - A_RouterMemory_Read-PDU (example)

The remote application process shall respond to the A_RouterMemory_Read.ind primitive with an A_RouterMemory_Read.res primitive containing the number of octets read beginning with the start address to increasing addresses. If the remote application process has a problem, e.g., address space unreachable or protected or an illegal number of octets are requested, then the parameter number of the A_RouterMemory_Response-PDU shall be zero and shall contain no data.

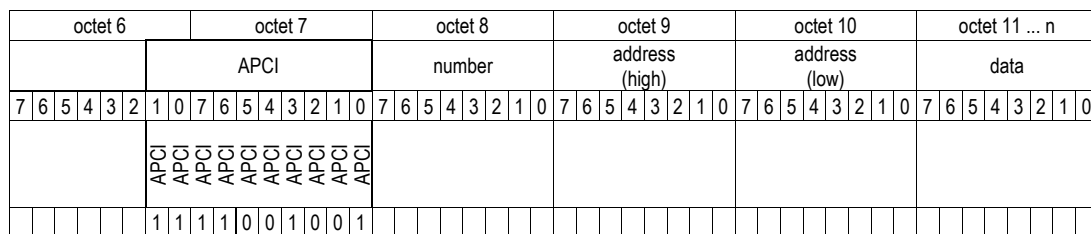


Figure 96 - A_RouterMemory_Response-PDU (example)

The remote Application Layer shall accept the service response and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters TSAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_RouterMemory_Response-PDU.

The Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_RouterMemory_Response-PDU to an A_RouterMemory_Read.Acon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_RouterMemory_Read.Acon primitive.

A_RouterMemory_Read.req(ack_request, priority, hop_count_type, ASAP, number, Memory address)

ack_request: Data Link Layer Acknowledge requested or don't care
priority: system, urgent, normal or low priority
hop_count_type: hop count 7 or standard
ASAP: local reference of the Service Access Point
number: number of octets to be read beginning with the start address to increasing addresses
memory_address: the 16 bit start address

A_RouterMemory_Read.Lcon(ack_request, priority, hop_count_type, ASAP, number, Memory address, a_status)

ack_request: Data Link Layer Acknowledge requested or don't care
priority: system, urgent, normal or low priority
hop_count_type: hop count 7 or standard
ASAP: local reference of the Service Access Point
number: number of octets to be read beginning with the start address to increasing addresses
memory_address: the 16 bit start address
a_status: ok: A_RouterMemory_Read sent successfully with T_Data_Connected service
not_ok: transmission of the associated T_Data_Connected request Frame did not succeed

A_RouterMemory_Read.ind(priority, hop_count_type, ASAP, number, Memory address)

priority: system, urgent, normal or low priority
hop_count_type: hop count 7 or standard
ASAP: local reference of the Service Access Point
number: number of octets to be read beginning with the start address to increasing addresses
memory_address: the 16 bit start address

A_RouterMemory_Read.res(ack_request, priority, hop_count_type, ASAP, number, Memory address, data)

ack_request: Data Link Layer Acknowledge requested or don't care
priority: system, urgent, normal or low priority
hop_count_type: hop count 7 or standard
ASAP: local reference of the Service Access Point
number: number of octets read beginning with the start address to increasing addresses, or zero to indicate a problem
memory_address: the 16 bit start address
data: the octet(s) read

Error handling

If data are to be read from a protected area or from any logical address that is not associated to physical memory then in the A_RouterMemory_Response-PDU the field *number* shall be zero and there shall be no field data to indicate an error. The same shall apply if only part of the Memory to be read is protected or physically existing.

In addition, the same shall apply if the value of the parameter *number* is greater than Maximum APDU Length – 4 ³²⁾.

3.6.5 A_RouterMemory_Write-service

The A_RouterMemory_Write.req primitive shall be applied by the user of Application Layer, to write between 1 octet and 254 octets in the memory of the second controller of the remote communication controller. The parameter *memory_address* shall specify the 16 bit start address and the parameter *number* shall contain the number of octets to be written beginning with the start address to increasing addresses.

The service shall be a confirmed service if Verify Mode is active, otherwise it shall be an acknowledged service.

The local Application Layer shall accept the service request and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_RouterMemory_Write-PDU.

The remote Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_RouterMemory_Write-PDU to an A_RouterMemory_Write.ind primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments TSAP and priority of the A_RouterMemory_Write.ind primitive.

octet 6						octet 7						octet 8						octet 9						octet 10						octet 11 ... n										
						APCI						number						memory_address (high)						memory_address (low)						data										
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	
						APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI	APCI																									
						1	1	1	1	0	0	1	0	1	0																									

Figure 97 - A_RouterMemory_Write-PDU (example)

With inactive Verify Mode the remote application process shall not respond. Instead, the local Application Layer shall map a T_Data_Connected.con primitive to an A_RouterMemory_Write.Lcon primitive. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_RouterMemory_Write.Lcon primitive; number, memory_address and data shall be don't care.

With active Verify Mode the remote application process shall respond to the A_RouterMemory_Write.ind primitive with an A_RouterMemory_Write.res primitive containing the requested number of octets of the associated Memory area. The value of the associated memory area shall be explicitly read back after writing to it. If the remote application process has a problem, e.g., memory area unreachable or protected or an illegal number of octets are requested, then the parameter *number* shall be zero and there shall be no field data to indicate an error.

The remote Application Layer shall accept the service response and shall pass it with a T_Data_Connected.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters of the T_Data_Connected.req primitive, the TSDU shall be an A_RouterMemory_Response-PDU.

³²⁾ The maximal APDU length that shall be supported is specified in the specification of PID_MAX_APDU_LENGTH in [04].

The Application Layer shall map a T_Data_Connected.ind primitive with TSDU = A_RouterMemory_Response-PDU to an A_RouterMemory_Write.Acon primitive if an A_RouterMemory_Write-PDU has been sent before over this connection. The arguments TSAP and priority shall be mapped to the corresponding arguments ASAP and priority of the A_RouterMemory_Write.Acon primitive.

A_RouterMemory_Write.req(ack_request, priority, hop_count_type, TSAP, number, Memory address, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	hop count 7 or standard
ASAP:	local reference of the Service Access Point
number:	number of octets to be written beginning with the start address to increasing addresses
memory_address:	the 16 bit start address
data:	the octet(s) to be written

A_RouterMemory_Write.Lcon(ack_request, priority, hop_count_type, ASAP, number, Memory address, data, a_status)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	hop count 7 or standard
ASAP:	local reference of the Service Access Point
number:	number of octets to be written beginning with the start address to increasing addresses
memory_address:	the 16 bit start address
data:	the octet(s) to be written or no data
a_status:	ok: A_RouterMemory_Write sent successfully with T_Data_Connected service
	not_ok: transmission of the associated T_Data_Connected request Frame did not succeed

A_RouterMemory_Write.ind(priority, hop_count_type, TSAP, number, Memory address, data)

priority:	system, urgent, normal or low priority
hop_count_type:	hop count 7 or standard
ASAP:	local reference of the Service Access Point
number:	number of octets to be written beginning with the start address to increasing addresses
memory_address:	the 16 bit start address
data:	the octet(s) to be written

A_RouterMemory_Write.res(ack_request, priority, hop_count_type, ASAP, number, Memory address, data)

ack_request:	Data Link Layer Acknowledge requested or don't care
priority:	system, urgent, normal or low priority
hop_count_type:	hop count 7 or standard
ASAP:	local reference of the Service Access Point
number:	number of octets written beginning with the start address to increasing addresses, or zero to indicate a problem
memory_address:	the 16 bit start address
data:	the octet(s) read back or no data

A_RouterMemory_Write.Acon(priority, hop_count_type, ASAP, number, Memory address, data)

priority:	system, urgent, normal or low priority
hop_count_type:	hop count 7 or standard
ASAP:	local reference of the Service Access Point
number:	number of octets written beginning with the start address to increasing addresses, or zero to indicate a problem
memory_address:	the 16 bit start address
data:	the octet(s) read back or no data

Error handling

If data are to be written to a protected area from any logical address that is not associated to physical memory then the service indication shall be ignored. In any case physical memory shall not be addressable via different logical addresses. If only a part of the addressed memory is protected or does not exist, then the complete write operation shall fail.

In addition, the remote Application Layer shall ignore the A_RouterMemory_Write.ind if the value of the parameter *number* is greater than Maximum APDU Length – 4 ³³⁾.

In addition, the remote Application Layer shall ignore the A_RouterMemory_Write.ind if the parameter *number* is not equal to the number of received data octets.

If Verify Mode is active, then in case of a failed write operation the field *number* of the A_RouterMemory_Response-PDU shall be zero and there shall be no field data to indicate an error.

³³⁾ The maximal APDU length that shall be supported is specified in the specification of PID_MAX_APDU_LENGTH in [04].

4 Parameters of Application Layer

4.1 Group Object Association Table

The Group Object Association Table shall map TSAPs of multicast communication modes to Application Layer Service Access Points (ASAPs) and vice versa. One TSAP can be mapped to more than one ASAP and one ASAP can have more than one TSAP. The Group Object Association Table may be downloaded using the network management.

Various Realisation Types exist of the Group Object Association Table. Please refer to [04].

4.2 Verify flag

The Verify Flag shall control whether the Verify Mode is enabled or disabled.

The Verify Mode Control is specified in [04].

5 Secure Application Layer

5.1 General requirements and overview

5.1.1 Embedding of the S-AL within the Application Layer and basic functionality

The Secure Application Layer (S-AL) shall take care of the KNX Data Security at the level of the links.

The S-AL shall be part of the Application Layer (AL). This shall allow that the S-A_Data- service be close to the Application Layer services and shall allow security policies to be possibly adapted to each Application Layer service.

The use of the S-AL shall not influence the functionality of Plain Application Layer.

- In reception direction, after accepting the S-A_Data-service, the S-AL shall check the security in function of the communication mode either according to the Point-to-point Key Table or according to the Group Key Table, restore the Plain APDU and if successful forward the contained plain AL-service request internally to the Plain Application Layer.
- In transmission direction, again in function of the communication mode that will be used either according to the Point-to-point Key Table or according to the Group Key Table for the S-A_Data-service, the S-AL shall secure the plain AL-service and shall transmit the secure APDU through the S-A_Data-service.

However, the Plain Application Layer shall transparently forward the security service parameters (par_auth, par_conf and link_index) between the S-AL and the AIL as specified for the Plain Application Layer services in clause 3 “Application Layer services”.

Figure 98 shows the location of the S-AL within the KNX communication stack. It shows the handling of secure messages. This is only a basic scheme. It does not show the handling of plain messages and the possible exceptions. The structure and priorities of the S-AL inside are not complete in this figure.

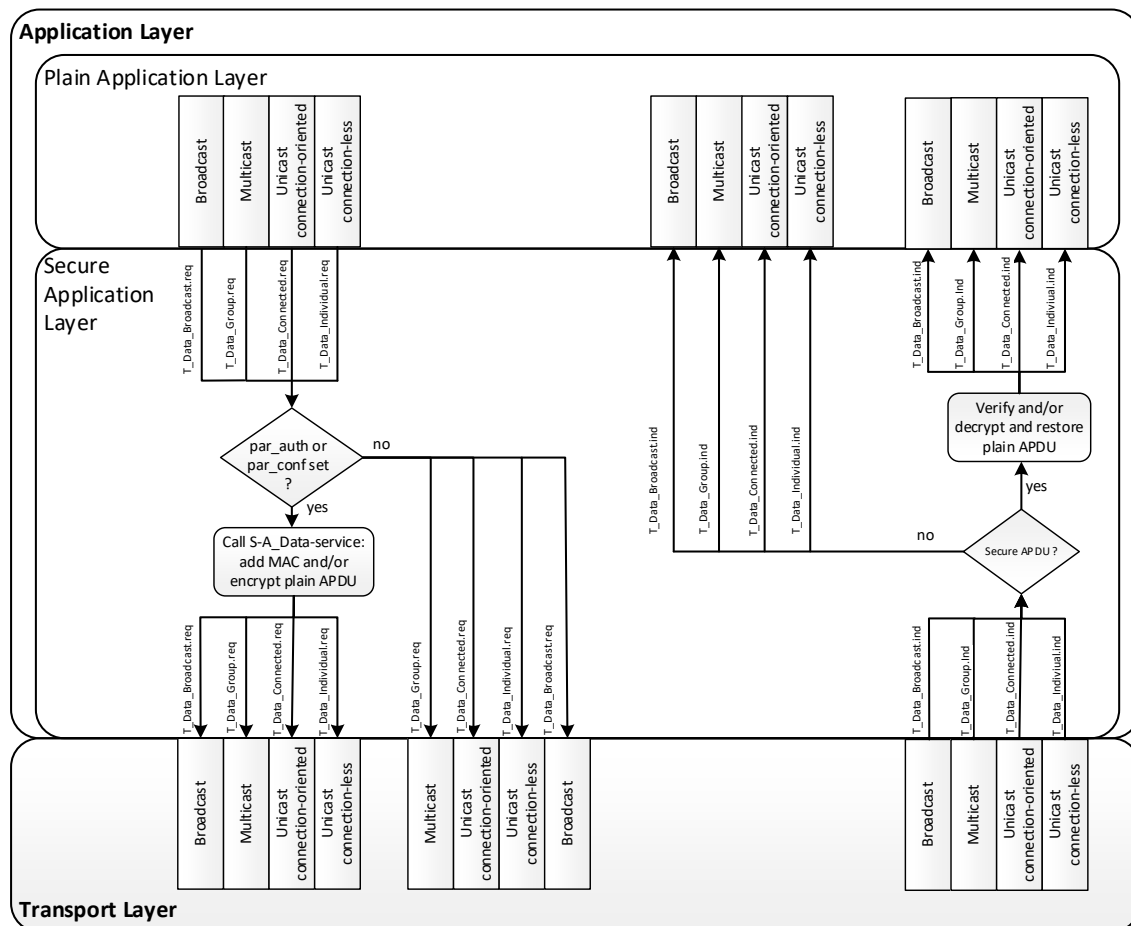


Figure 98 – Location of the S-AL within the Application Layer

Table 1 below gives the overview of the messages of the Secure Application Layer as specified further and the allowed communication modes.

Table 5 – Security Application Layer PDUs

SCF							
b7	b6	b5	b4	b3	b2	b1	b0
							S-AL service
				0	0	0	S-A_Data-PDU
				0	1	0	S-A_Sync_Req-PDU
				0	1	1	S-A_Sync_Res-PDU

Transport Layer services					
T_Data_Group	T_Data_Tag_Group	T_Data_Broadcast	T_Data_SystemBroadcast	T_Data_Individual	T_Data_Connected
M	M	M	M	M	M
X	X	M	M	M	M
X	X	M	M	M	M

5.1.2 S-AL – overview

The S-AL shall have the following functionality.

- Support the S-A_Data-service see clause 5.2
- Handling of the Sequence Number: see clause 5.3
- Handling of security failures see clause 5.4
- Secure handling of the Transport Layer services see clause 5.5

The following clauses specify these and other functions.

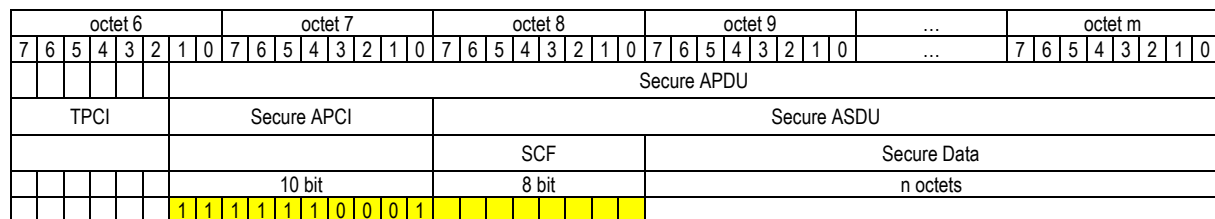
5.1.3 AES-128 with CTR operation mode and AES-CBC-MAC signature (CCM)

5.1.3.1 Secure Data

The common format for the Secure Data with this algorithm shall be as specified in Figure 99. The Secure Data shall contain the following fields. (These are specified in detail below.)

- Sequence Number (SeqNr), and
- Plain APDU (000000b + APCI + ASDU = Data), and
- Message Authentication Code (MAC).

The Secure APCI and the SCF are security algorithm independent and are specified in 5.2.1.



Legend

Plain text specific for the KNX S-A_Data-service.

Figure 99 – Secure APDU (example on TP1)

- **Sequence Number (SeqNr)**

This shall be the value of the *Sequence Number Sending* of the original sender of the message.

See 5.3.

- **000000b**

This constant value shall serve for aligning the APCI on the same bit positions as in the field level media, with the two bits most significant bits in one octet and the two (4 bit APCI) or eight least significant bits in the next octet.

- **Plain APDU (APCI + Data):**

definition: This shall be the APDU of the Plain Application Layer service.

format: The length, possible subfields and their encoding shall comply with the specification as given for the plain AL-service in clause 3 “Application Layer services”.

NOTE 14 This means that also the “optimized format” is used: data of 6 bits and smaller shall be on the lsb in the same octet as the lsb of the APCI, possibly preceded by 0's; with data longer than 6 bit the APCI will also be followed by 000000b and the data starts in the next octet. Please refer to e.g., Figure 5, Figure 6, Figure 7 and Figure 8.

- **Message Authentication Code (MAC)**

See in the following clauses.

5.1.3.2 Common requirements

This clause only gives the KNX specific use of the CCM parameters. It is not the intention or the scope of this paper to define CCM. For the definition of CCM, please refer to [07]. In Annex A, an informative overview is given of the resulting use of CCM for KNX.

The below requirements shall apply both for the use with authentication only as well as with authentication and confidentiality.

NOTE 15 The presence of parameters and state of the link layer and transport layer in B_0 and Ctr_j requires some collaboration between layers in the KNX stack implementation.
 In the “upward” direction this could be implemented by just passing the required information in the “.ind” messages.
 For the “downward” direction, the lower layers have to implement helper methods so the S-AL can e.g., resolve an ASAP to a destination address. The most demanding part is the TL sequence number in the connection-oriented case (which is part of the TPCI field), filling this field may be difficult to ensure in a device implementation.

B_0

The composition of the first block B_0 in the CCM algorithm shall be KNX specific. This is specified in Figure 100. These fields from the KNX Frame are included here, so that they cannot be altered in the communication path between sender and receiver without being detected.

The source address, destination address and TL sequence number are not available when the MaS is accessed via a local TL connection. In this case, these fields shall be set to 0 in the B_0 and Ctr_j blocks.

octet nr												
0 MSB	...	5	6	7	8	9	10	11	12	13	14	15 LSB
SeqNr or Random ^a			SA		DA		00h	AT	TPCI / APCISec		00h	q

- ^a This shall be
- SeqNr for S-A_Data and S-A_Sync_Req-PDU
 - Random for S-A_Sync_Res-PDU

Figure 100 – Format of B_0

AT shall have the 8 bit value A000EEEEb where A shall equal the Address Type of the KNX Frame.

EEEEb shall equal the Extended Frame format (EFF) if L_Data_Extended Frames are used and 0000b otherwise.

NOTE 16 From the Control field (CTRL) or Extended Control Field (CTRLE) (TP1 and PL110) or the KNX Ctrl-field (KNX RF), the fields repeat flag, priority, hop count and ffff are thus not protected. (The value of the hop count changes if the Frame passes a Coupler.)

TPCI / APCISec shall be composed as in Figure 101: the first 6 bits shall be the TPCI of the KNX Frame and the last 10 bits shall be the Secure APCI (3F1h).

bit number	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
field name	TPCI						Secure APCI									
value							1	1	1	1	1	1	0	0	0	1

Figure 101 – Format of the field TPCI/APCISec in B_0

q shall be the length of the payload P: see A.3.

NOTE 17 The composition of P depends on the S-AL-service (S-A_Data with authentication only or with authentication and confidentiality and S-A_Sync). q is however always specified as the length of P.

Ctr_j

Also the format of the Block Counter Ctr_j (see [08]) is KNX specific. Ctr_j shall be composed as specified in Figure 102. The counter [j] for Ctr₀ shall be 00h.

octet nr												
0 MSB	...	5	6	7	8	9	10	11	12	13	14	15 LSB
SeqNr or Random ^a			SA		DA		00h	00h	00h	00h	01h	[j]

^a This shall be

- SeqNr for S-A_Data and S-A_Sync_Req-PDU
- Random for S-A_Sync_Res-PDU

Figure 102 – Format of Ctr_j

NOTE 18 Octet 14 of Ctr_j shall be 01h. This shall guarantee that B₀ and Ctr_j always have different values, regardless of the values of AT and TPCI in B₀.

Each counter value shall be calculated by incrementing the preceding counter value by 1.

$$\text{Ctr}_j = \text{Ctr}_{j-1} + 1 \quad // \text{ For } j = 1 \text{ to } n; n \leq 255$$

5.1.3.3 Confidentiality only

This use case does not exist in the case of “AES-128 with CTR operation mode and AES-CBC-MAC signature (CCM)”, because the field MAC shall always be present.

5.1.3.4 Authentication only

5.1.3.4.1 Format of the Secure Data

The SCF, SeqNr and plain APDU shall not be encrypted. These shall be followed by the MAC.

octet 6								octet 7								octet 8								octet 9-14				...				...					
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0					...				...					
						Secure APDU																															
TPCI						Secure APCI										Secure ASDU																					
																SCF						Secure Data															
																						SeqNr								Plain APDU APCI + Data				MAC			
						10 bit										8 bit						6 octets								n octets				4 octets			
						1	1	1	1	1	1	1	0	0	0	1												000000b									

Legend

- Plain text specific for the KNX S-A_Data-service.
- Plain text from the original AL-service.
- encrypted text

Figure 103 – Secure Data in case only authentication is used (example on TP1)

NOTE 19 The Sequence Number (SeqNr) is transmitted in plain and allows the receiver to quickly check the freshness.

5.1.3.4.2 Message processing (authentication only)

Message generation

The MAC shall be calculated according to the CCM algorithm as specified in [07].

1. In the SCF, set the field *SAI* to denote “CCM with authentication and without confidentiality” according to the requested security parameters (authentication only, no confidentiality) and the requested S-AL service, as specified in Figure 106.
2. The SeqNr shall be the current value of *Sequence Number Sending*, see *PID_SEQUENCE_NUMBER_SENDING* in [04]. *Sequence Number Sending* will then be incremented after the message is passed to the local Transport Layer, see 5.3.
3. For Authentication only
For “Authentication only” P³⁴⁾ shall be empty and so the Plain APDU shall be included in A³⁴⁾.
 $A = SCF \mid 000000b \mid \text{Plain APDU (}=\text{plain APCI} + \text{data)}$
 $P = \{\text{empty}\}$
4. The further calculation shall be according to Annex A.
The Block B₀ shall be as specified in 5.1.3.2.
5. For KNX, the 32 most significant bits of Y_n (MSB₃₂(Y_n)) shall be used as MAC.

Message verification (security check)

The receiver shall verify the message exactly as it is done in the sender and as it is specified in [08]. The following shall be done in the given order.

1. Verify the SCF
In reception direction, if the SCF contains any field of which the value is not supported or any reserved field that is not zero (see Figure 106) then this request shall be ignored; the Frame shall be thrown away. This shall not be logged in the *Security Failures Log*.
2. Verify SeqNr (see 5.3).
The SeqNr shall be verified. If this fails, then the further checks shall not be performed.
3. Compute MAC based on the received fields and check that it equals the received MAC-value.
If the MAC-verification fails, then the message shall not be passed to the AL and AIL; errors on Roles and Permissions will does not be evaluated.

If either one of the three verifications fails, the received secure message shall be ignored and the Security Failure shall be handled. (See 5.4.)

³⁴⁾ For the definition of A and P, please refer to the CCM specification in [07]. ‘A’ is the “associated data” and ‘P’ is the payload.

5.1.3.5 Authentication and Confidentiality

5.1.3.5.1 Format of the Secure Data

The Secure Data shall contain the SeqNr, the cipher APDU and the MAC. The cipher APDU shall be based on the plain APDU encrypted with the AES-128 algorithm with CTR operation mode.

octet 6								octet 7								octet 8								octet 9 to 14								...				...											
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0									...				...											
								Secure APDU																																							
TPCI								Secure APCI								Secure ASDU																															
																SCF								Secure Data																							
																								SeqNr								cipher text of the Plain APDU								MAC							
								10 bit								8 bit								6 octets								n octets								4 octets							
								1	1	1	1	1	1	0	0	0	1																														

Legend

Plain text specific for the KNX S-A_Data-service.

encrypted text

Figure 104 – Secure Data in case Authentication and Confidentiality are used

The cipher text of the plain APDU and the MAC shall be calculated as follows.

5.1.3.5.2 Message processing (authentication and confidentiality)

Message generation

The MAC shall be calculated according to the CCM algorithm and shall then be used in the CCM encryption as specified in [08].

The same key k shall be used for authentication as well as for confidentiality.

1. In the SCF, set the field *SAI* to denote “CCM with authentication and with confidentiality” according to the requested security parameters (authentication and confidentiality) and the requested S-AL service, as specified in Figure 106.
2. The SeqNr shall be the current value of *Sequence Number Sending*, see *PID_SEQUENCE_NUMBER_SENDING* in [04]. *Sequence Number Sending* will then be incremented after the message is passed to the local Transport Layer, see 5.3.
3. For Authentication and Confidentiality

For “Authentication and Confidentiality”, P^{34} shall consist of the Plain APDU and A^{34} shall not contain the Plain APDU since it is already included in P .

$A = \text{SCF}$

$P = 000000b \mid \text{Plain APDU (=plain APCI + data)}$

The further calculation shall be according to Annex A.2.

The Block B_0 shall be as specified in 5.1.3.2.

The blocks Ctr_0 to Ctr_n shall be as specified in 5.1.3.2.
4. Referring to Figure 121, it shall be the result C (cipher text) that shall be transmitted on KNX as cipher text after the SeqNr.

Message decryption and verification (security check)

The message decryption and verification in the receiver shall start exactly as is done in the sender and as it is specified in [08].

1. Verify the SCF

If the SCF contains any field of which the value is not supported or any reserved field that is not zero, the message shall not be evaluated further. This shall not be logged in the Security Failures Log.

2. Verify SeqNr (see 5.3).

The SeqNr shall be verified. If this fails, then the further checks shall not be performed.

3. Decrypt the received message with the security key and reconstruct the payload.

4. Compute MAC based on the received fields and verify that it equals the received MAC-value.

If the MAC-verification or the decryption fails, then the message shall not be passed to the AL and AIL; errors on Roles and Permissions will does not be evaluated.

If either one of the four verifications fails, the received secure message shall be ignored and the Security Failure shall be handled. (See 5.4.)

5.1.4 Other security algorithms

Table 6 lists the specified KNX Security Algorithms.

Values for the SAI that are not defined are reserved. These values shall not be used.

A received secure message with a reserved value for the SAI shall be ignored.

Expectations for possible future algorithms

The algorithm should be selected according to the available hardware performance (processor speed, memory size).

5.2 Support the S-A_Data-service

5.2.1 S-A_Data-service

5.2.1.1 Basic requirements

The S-AL shall support the S-A_Data-service. In transmission direction, this service shall secure the plain AL-service; in reception direction it shall check the security of a received S-A_Data-PDU. The S-AL shall handle the Security Control Field within the secure ASDU.

NOTE 20 This clause solely specifies the S-A_Data-service. It does not specify when this service shall be applied. This depends on the value of the security service parameters `par_auth` and `par_conf` and this is explained in more detail in 5.5 "Secure Handling of the Transport Layer services" below.

The S-A_Data-service shall be an Application Layer internal service. It shall only be possible to call this service in the S-AL through the use of the security service parameters (`par_auth`, `par_conf` and `link_index`) of the plain AL-services; it shall not be possible to call this service explicitly. The S-A_Data-service shall not be available on any External Message Interface.

The S-A_Data-service shall replace the plain AL-service-PDU with the encrypted Secure APDU in transmission direction and vice versa in reception direction.

The local S-AL shall apply the S-A_Data-service to transmit an S-A_Data-PDU on the bus. The local S-AL shall secure the contained plain AL-service-PDU in function of all the following.

- the security service parameters `par_auth` and `par_conf` of the plain AL-service
- the Security Configuration Information in function of the plain AL-service (key, TSAP-type)
- the TSAP or ASAP

EXAMPLE For a different destination GA or a different destination IA, the security key may be different.

- the layer parameters of the S-AL.

This is specified in further detail in the following clauses.

The SCF shall indicate an S-A_Data -PDU as specified in Figure 106.

If the remote S-AL receives an S-A_Data-PDU then it shall do the following.

1. If the Tool Access flag is not set, then it shall check if the Source Address is contained in the Security Individual Address Table. If this is not the case, then the message shall be ignored; no security failure counter shall be incremented to this.
2. If the Source Address is found, then the contained Sequence Number shall be verified as specified in 5.3.
3. If the message is still accepted for further handling, then the security shall be checked according to the appropriate Security Resource related to the used communication mode (e.g., Point-to-point Key Table, Group Key Table, Zone Key Table). Depending on the result the message shall be rejected and the security failure logs possibly updated or the message shall be transmitted internally to AL.

5.2.1.2 Secure APDU

The S-A_Data-PDU shall have the format as given in Figure 105.

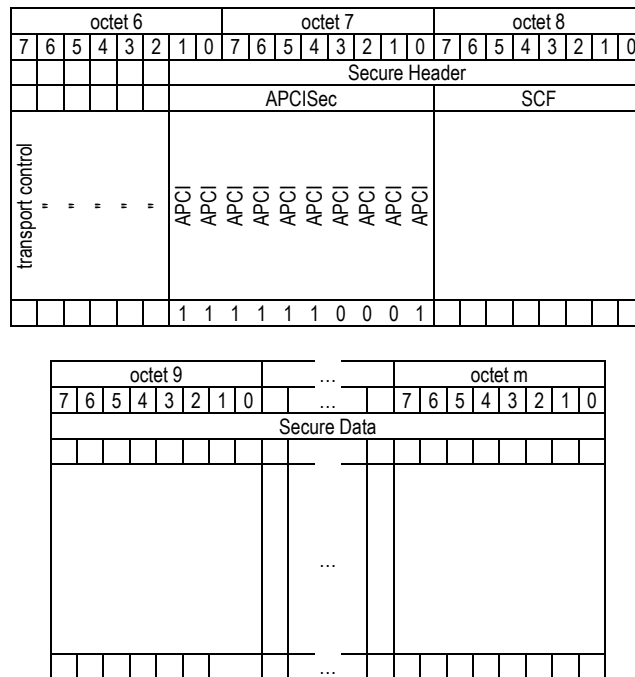


Figure 105 - Format of the encrypted Secure APDU (informative)

The below paragraphs specify the format and use of this message format. For a worked out example, please refer to Annex C

The Secure APDU shall consist of the following fields.

- Secure Header (SHD)

definition: The Secure Header shall always be composed of the APCISec and the SCF.

- **APCISec**

definition: This shall be the dedicated APCI that shall be used to indicate this APDU as a secure APDU.

value: APCISec = F31_{APCI} = 3F1h = 1111110001b

- **Security Control Field (SCF)**

definition: This field shall give the indications about Confidentiality, Authentication and the used security algorithm and its possible operation modes.

encoding: The SCF shall be a 1 octet field formatted as specified in Figure 106.

b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁	b ₀	Algorithm	Service	Authentication	Confidentiality
T	SAI			SBC	S-AL service						
	0	0	0		0	0	0	CCM	S-A_Data-PDU	yes	no
	0	0	1		0	0	0	CCM	S-A_Data-PDU	yes	yes
	0	0	1		0	1	0	CCM	S-A_Sync_Req-PDU	yes	yes
	0	0	1		0	1	1	CCM	S-A_Sync_Res-PDU	yes	yes

Figure 106 - Security Control Field

- **Tool Access (T)**

definition: This bit shall be set if the security (authentication and/or confidentiality) is done using the specific tool entry in the PID_TOOL_KEY or the FDSK.

encoding:

b ₇	Tool Access
0	Not using the specific tool entry
1	Using the specific tool entry

Figure 107 – Tool Access

- **Security Algorithm Identifier (SAI)**

definition: This field shall identify the applied security algorithm and the operation mode if any shall be used together with possible further Security Algorithm related information. For more information about security algorithms, operation modes and Initialisation Vector, please refer to Annex B.

encoding:

Table 6 – Security Algorithm Identifier

b ₆	b ₅	b ₄	Security algorithm and operation mode
0	0	0	CCM (AES CTR for confidentiality and AES CBC for authentication) with authentication only.
0	0	1	CCM with authentication and confidentiality

Other values are reserved for future standard extensions and shall not be used.

- **System Broadcast flag (SBC)**

definition: This field shall indicate whether the resulting Frame shall by a Media Coupler onto an open medium that supports system broadcast communication mode be forwarded on broadcast – respectively on system broadcast communication mode.

NOTE 21 This field is only relevant for the sender to control that the Frame if sent on broadcast communication mode on a closed medium shall be routed by a Media Coupler onto an open medium using system broadcast communication mode, and for the Media Coupler to conclude on the communication mode.

Please also refer to 5.2.1.3.

encoding:

Table 7 – System Broadcast flag

SBC	Name	Definition
0	by service	The communication mode is defined by the used Transport Layer service.
1	system broadcast	The communication mode shall be System Broadcast.

The further fields in the Secure APDU shall depend on the Security Algorithm indicated in the field Algorithm in combination with the use of Authentication and Confidentiality. This is specified in the following clauses.

5.2.1.3 Communication Mode

In transmission direction the communication mode shall equal the communication mode of the requesting plain AL-service or as indicated its service parameter comm_mode. The S-AL shall set the SBC-flag as required in Table 8.

NOTE 22 Devices that do not support system broadcast communication mode (T_Data_SystemBroadcast), like TP1 devices, will thus never set this flag to “system broadcast”.

Table 8 – Setting of the SBC-flag in the S-A_Data-PDU

communication mode of the Plain AL-service	SBC flag
multicast	“by service”
broadcast	“by service”
system broadcast	“system broadcast”
ptp connectionless	“by service”
ptp connection oriented	“by service”
NOTE 1 For the definition of the System Broadcast flag please refer to 5.2.1.2 and Table 7.	

In reception direction, the following shall apply.

- TP1 devices may totally ignore S-A_Data with the SBC flag in the SCF set.
- IP devices may totally ignore ROUTING_INDICATIONS with the SBC flag in the SCF set.
- IP devices shall ignore totally ROUTING_SYSTEM_BROADCAST with the SBC flag cleared.
- RF and PL110 devices shall check the consistency between the SBC flag and the used communication mode and ignore mismatches as follows.

Table 9 – Evaluation of the SBC flag for RF and PL110 devices

Communication mode	SBC flag = 0 (not a system broadcast)	SBC flag = 1 (a system broadcast)
Broadcast	The message shall be accepted.	The message may be ignored totally. NOTE 23 This makes that PID_RF_SBC_CONTROL in the TP1/RF MC does not make sense to be set to “route as broadcast” (1) for S-A_Data with SBC = 1.
System Broadcast	The message shall be ignored totally.	The message shall be accepted.

5.2.1.4 Error – and exception handling

If there is any error – or exception situation in the S-A_Data.req service primitive, then the S-AL shall confirm the plain AL-service negatively (“not_ok”) to the plain AL user.

EXAMPLE 3 The implementation of CCM is not supported with “confidentiality only”; if this would however be required through the combination of the service parameters par_auth and par_conf (see the use par_auth and par_conf in the specification of the AL-services in clause 3 “Application Layer services”), then the plain AL-service shall be confirmed negatively.

In reception direction, if the security of the Secure APDU is verified, the S-AL shall additionally verify if the SeqNr is followed by the field 000000b. If this field does not have this expected value then the message shall not be passed to the P-AL. As the security is verified positively however, the SeqNr of the sender remains handled as normal. This shall not be logged in the Security Failure Log.

5.3 Handling of the Sequence Number

5.3.1 Runtime handling

Goal

The SeqNr shall provide Data Freshness and by this protect the system against replay attacks.

Requirements for sender and receiver

Each time that the S-AL in the sender forwards an S-A_Data-PDU to the TL, it shall increment the *Sequence Number Sending* by 1. This shall be done regardless of the further result of the TL-service, this is, regardless of the t_status of the TL-service confirmation primitive.

NOTE 24 This is special the case on TP1: the Sequence Number Sending shall be incremented even if the transmission of the TP1 frame is on the medium not confirmed or negatively confirmed.

There shall be only one single *Sequence Number Sending* used for all secure communication; there shall be no different Sequence Numbers for sending per secure link or per secure Datapoint.

The receiver shall evaluate the value of the SeqNr for each received secure message from the sender’s IA. On KNX open media (KNX RF, KNX PL110) the sender shall additionally be identified by its KNX Serial Number and Domain Address.

Format

The Sequence Number shall be an unsigned six octet (48 bit) counter as specified in Figure 108.

NOTE 25 In B.2 it is motivated why 6 octets are used.

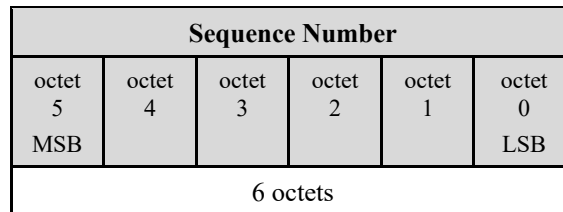


Figure 108 – Format of the Sequence Number

Depending on the Profile, the *Sequence Number Sending* may be stored as PID_SEQUENCE_-NUMBER_SENDING. Please refer to [04] for additional requirements for PID_SEQUENCE_-NUMBER_SENDING.

Basic principle

The Sequence Number shall be a simple counter value.

The 6 octet format makes that it takes over 100 000 year until a counter overrun happens (for calculation of this value see Annex B.2- Figure 122).

If the *Sequence Number Sending* reaches the value FFFFFFFFh then it shall not be incremented anymore; it shall not wrap over to 0; the MaS shall not send any further secure Frames.

For the specific requirements on the handling of the SeqNr for the various communication modes, please refer to the clauses 5.5.3, 5.5.4, 5.5.4.3 and 5.5.6.2.

Transmission

The sender shall maintain one single Sequence Number for all its outgoing communication. This shall be the *Sequence Number Sending*.

EXAMPLE 4 For S-Mode Profiles, this shall be stored in the Property “Sequence Number Sending” as specified in [04].

The sender shall use the value of its *Sequence Number Sending* when transmitting on any Secure Link.

- It shall contain that value in the field Sequence Number (SeqNr) in the Secure APDU (see 5.1.3).
- It shall use the value of that field Sequence Number in the calculation of the Initialisation Vector (see 5.1.3.4.2 and 5.1.3.5.2).

After the S-A_Data-PDU is forwarded to the Transport Layer, the sender shall always increment the Sequence Number Sending.

NOTE 26 This shall always be done regardless of the further handling of the secure message by the lower layers. Even if the transmission fails, for instance because of a negative Layer-2 acknowledge, the Sequence Number Sending shall be incremented.

Reception

If a receiver receives an S-A_Data-PDU then it shall compare the contained Sequence Number with the Sequence Number stored as “Last Valid SeqNr” in the Security Individual Address Table (see [04]) for the IA of the sender of the received S-A_Data-PDU.

If the IA of the sender is not found in the Security Individual Address Key or the Tool Key is not used, then the S-A_Data-PDU shall be ignored and the Frame shall be thrown away; no message shall be passed to the P-AL; else, the processing shall continue.

NOTE 27 For S-A_Data-PDUs with T=1 (using the Tool Key), the Sequence Number for Tool Access (see [04]) is stored outside the *Security Individual Address Table*. There is no Resource definition for this data.

- If the received Sequence Number is **higher than** the “Last Valid SeqNr” then the S-AL shall accept the S-A_Data-PDU.
If the further security checks are positive, then the receiver shall write the contained Sequence Number value for that sender in the Security Individual Address Table (see PID_SECURITY_-INDIVIDUAL_-ADDRESS_TABLE in [04]) as new value for “Last Valid SeqNr”.
- In case the received Sequence Number **equals** the “Last Valid SeqNr” then the S-AL shall ignore the S-A_Data-PDU. It shall not increment the Security Failure Counter.

NOTE 28 This allows that a second or further retransmission of a valid Frame by the Data Link Layer of the sender that would by the receiver not properly be recognised as a retransmission, will not lead to a false recognition as replayed Frame.
- In case the received Sequence Number is **lower** than the “Last Valid SeqNr” then the S-AL shall do the following.
 - The S-AL shall ignore this S-A_Data-PDU.
 - The S-AL shall not block further messages from this sender. Any subsequent message from this sender, with correct SeqNr, shall again be processed as normal.
 - It shall increment by 1 the counter for appropriate communication mode in the Security Failure Log and log this error as “latest failure n” (for access through “ReadServiceID 01h” in the PID_SECURITY_FAILURES_LOG).

Resources

Every device configured to use KNX Data Security shall store

- its own *Sequence Number Sending*
NOTE 29 For S-Mode device, this shall be stored in the Property *Sequence Number Sending* (PID_SEQUENCE_NUMBER_SENDING, see [04]).
- the *Sequence Number for Tool Access*
See [04].
- the *Last Valid Sequence Numbers* of its communication partners from which it receives secure communication.
NOTE 30 For S-Mode device, this shall be stored in the Property *Security Individual Address Table* (PID_SECURITY_INDIVIDUAL_ADDRESS_TABLE, see clause [04]).

Power down behaviour

The *Sequence Number Sending* and the Last Valid Sequence Numbers of the communication partners and the Sequence Number for Tool Access shall be saved in full at power-down and be restored at power-up.

Initial value

This shall be the value ex-factory and prior to any secure communication on a secure link. For the value of the Sequence Number Sending after Master Reset, please see below.

The initial value of the *Sequence Number Sending* for the MaS shall not be zero but shall be any value in the range from 1 to 255.

NOTE 31 The smallest useful value is 1 as a message sent with SeqNr 0 is ignored by the remote S-AL.

NOTE 32 There are no requirements on how this value is calculated. This also does not need to be a random value. The MaS may always assume the same initial value after each Master Reset.

Sequence Number Sending and Master Reset

Erase Code	Effect on the Sequence Number Sending	
	Sequence Number sending before executing ^b Master Reset is	
	< FF0000000000h	≥ FF0000000000h
01h Confirmed Restart	not influenced: no change ^a .	
02h Reset to default state	not influenced: no change ^a .	Re-initialise
05h ResetParam	not influenced: no change ^a .	
06h ResetLinks	not influenced: no change ^a .	
07h Reset to default without IA	not influenced: no change ^a .	
n.a. Local Reset to default state	not influenced: no change ^a .	Re-initialise
^a This shall only concern the <u>execution</u> of the Master Reset. The transmission of the A_Restart_Response-PDU that confirms the request will increment the Sequence Number Sending just like any outgoing message.		
^b This shall be the value of the Sequence Number Sending <i>before</i> executing the Master Reset, so, in case the Master Reset is called via A_Restart, this is <i>after</i> sending the A_Restart_Response-PDU.		

Re-initialisation

The Sequence Number Sending shall be re-initialised with a value that is at least 20 and at maximum FFFFh higher than the preceding initial value and in any case lower than 100000000h.

NOTE 33 The realisation is implementation specific: the MaS may simply store the new initial Sequence Number Sending and add at least 20 at a next re-initialisation, or, it may initialise for instance based on the Download Counter.
Other solutions are possible as well.

NOTE 34 This re-initialisation shall allow that if the MaS has reached this excessive high value of the Sequence Number Sending – because of own malfunction or malfunction of a communication partner or Client – that is stops communicating and can no longer be reprogrammed.

5.3.2 S-A_Sync-service

The purpose of the S-A_Sync-service is to allow one device (local S-AL user) that has no up-to-date knowledge of any of

- the current Last Valid Sequence Number Sending of another communication partner (remote S-AL user) and/or
- the Sequence Number for Tool Access, or
- its own Sequence Number Sending that the remote communication partner expects from itself

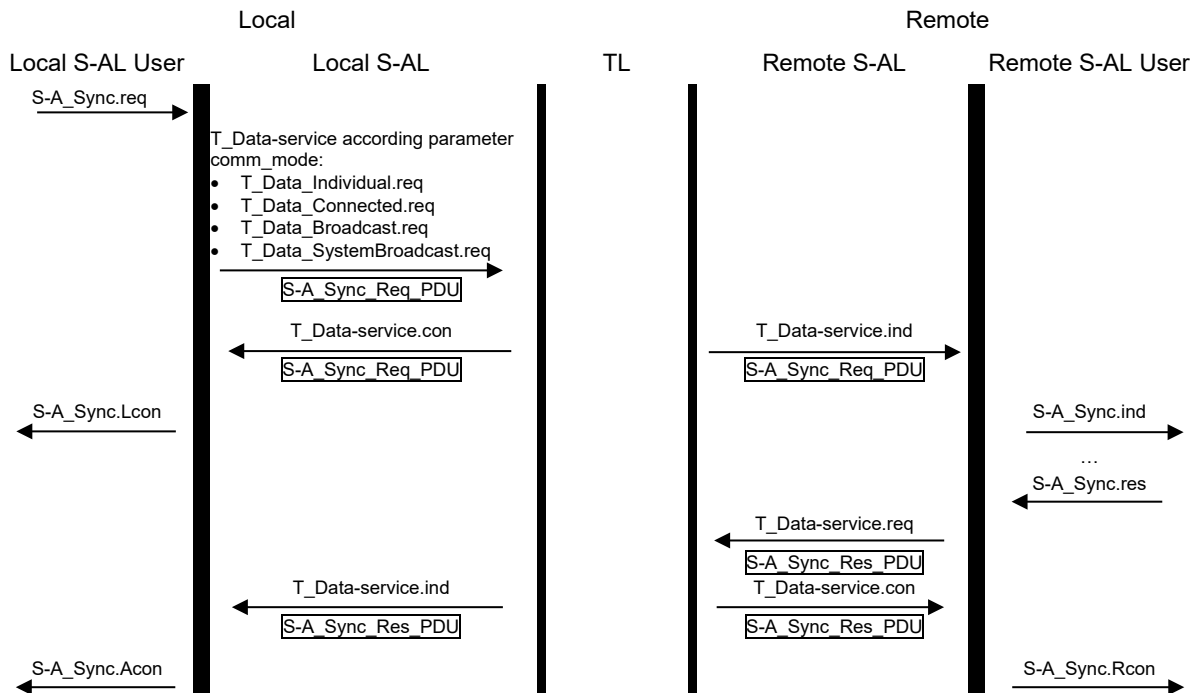
can get these values.

Axioms (informative)

The specification of the S-A_Sync-service and more specifically the values to be transmitted and the values used in the evaluation of the reception of the S-A_Sync_Req-PDU and the S-A_Sync_Res-PDU have some specific details that are caused by the following axioms.

- Axiom 1: The own SeqNr Sending is a “Next Valid SeqNr”; the stored received SeqNrs (Tool Access and other) are “Last Valid SeqNrs”.
- Axiom 2: The S-A_Sync-service contains “next valid SeqNr”, local and remote, and not any “Last Valid SeqNr”.

This means that as indicated below, sometimes a received value has to be compared with a stored value – 1 or a received value – 1 has to be stored.



The local S-AL user shall apply the S-A_Sync.req service primitive to obtain from the remote S-AL the Sequence Number Sending that the remote S-AL will use and the Sequence Number Sending that the remote S-AL expects from the local S-AL.

The S-A_Sync-service will thus consult and possibly update the Resources as listed in 5.3.1 (see [here](#)) both in the local S-AL as well as in the remote S-AL.

- its own *Sequence Number Sending*
- the *Sequence Number for Tool Access*
- the *Last Valid Sequence Numbers* of its communication partners from which it receives secure communication.
- The local S-AL user shall set the **comm_mode parameter** to indicate the communication mode that shall be used for transmitting the request.
 - point-to-point (connectionless or connection-oriented if a TL-connection exists)
The remote S-AL shall be addressed with a local ASAP, which shall be the IA_Index in the Security Individual Address Table.
 - broadcast
 - system broadcast

NOTE 35 Also for (system) broadcast communication mode, the unique key for point-to-point communication with one communication partner will be used to protect the message. This will make that even though all devices within the Domain or within the installation will receive the request, only one single device will be able to properly verify the message and only one device will respond.

- The local S-AL user shall set the **key_select parameter** to indicate whether this synchronisation shall be protected using:
 - the Tool Key, or

NOTE 36 The use case for using the Tool Key is for when the common tool, ETS™, intends to manage a secure device and is uncertain about the Sequence Number for Tool Access that it should use and the Sequence Number Sending that the device will use. The local S-AL user in this case is ETS.

- the key that is stored in the *Point-to-point Key Table* for that ASAP = IA_Index (see the specification of PID_SECURITY_INDIVIDUAL_ADDRESS_TABLE in [04]).

NOTE 37 The typical use case for using the key stored for point-to-point communication in the Point-to-point Key Table, is for synchronisation: if one secure device asks the Sequence Number Sending from another secure device and stores this as its local Last Valid SeqNr for that device. This functionality is at first foreseen for E-Mode devices.

- The local S-AL user shall set the serial_number parameter to indicate whether the field KNX Serial Number in the S-A_Sync_Req-PDUs shall include either:
 - the KNX Serial Number of the single target device, or
 - the value 0.

If the S-A_Sync-service is used on system broadcast – or broadcast communication mode, then the KNX Serial Number shall be contained and the value shall not be zero.

If the S-A_Sync-service is used on point-to-point connectionless or – connection oriented communication mode, then the field KNX Serial Number may contain the assumed KNX Serial Number of the communication partner or may be 0.

The local S-AL shall accept the S-A_Sync.req and shall build the S-A_Sync_Req-PDU as specified in Figure 109. The local S-AL shall set the SBC-flag according to the parameter comm_mode.

Secure APCI	SCF	SeqNr _{local}	KNX Serial Number	Challenge	MAC
10 bit	1 octet	6 octets	6 octets	6 octets	4 octets
3F1h	9Ah			N ₁	

**Figure 109 – Format of the S-A_Sync_Req-PDUs
(EXAMPLE – SCF with Tool Key and SBC set)**

The local S-AL shall challenge the remote S-AL with a challenge N₁ and shall add the Sequence Number Sending in SeqNr_{local} that it assumes that it will use to communicate with the remote S-AL (i.e., the current value of its “Sequence Number Sending”).

NOTE 38 In contrast to S-A_Data, transmitting an S-A_Sync_Req-PDU shall not alter the value of Sequence Number Sending.

The challenge shall be a six octet unsigned integer value. The local S-AL shall do its best not to use the same challenge multiple times, either by incrementing the value each time or by using a sufficiently random value.

The local S-AL shall secure the S-A_Sync_Req-PDU with authentication and confidentiality, as specified for the S-A_Data-service as specified in 5.1.3.5. In function of the parameter key_select, this “key_request” shall be

- the Tool Key ³⁵⁾, in which case the flag T in the SCF shall be set,

or

- the key linked to the IA_Index of the remote S-AL in the Point-to-point Key Table, in which case the flag T in the SCF shall be cleared.

A = SCF | KNX Serial Number

P = Challenge

Ctr₀ and B₀ shall be as in 5.1.3.2; the SeqNr shall be the Sequence Number Sending of the sender, in this case SeqNr_{local}.

The local S-AL shall in function of the service parameter comm_mode forward the S-A_Sync_req-PDU with a T_Data_Individual.req, T_Data_Connected.req, T_Data_Broadcast.req or T_Data_SystemBroadcast.req to the local Transport Layer. The parameters ASAP and priority shall be mapped to the corresponding parameters TSAP and priority of the Transport Layer service primitive, the TSDU shall be an S-A_Sync_Req-PDU.

³⁵⁾ If the Tool Key is not assigned yet, the MaC uses the FDSK.

S-A_Sync.req(ASAP, comm_mode, key_select)

ASAP:	shall either indicate the IA_Index of the IA of the communication partner or be an indication that the Tool Key is used.
comm_mode:	This shall be one of the following. - point-to-domain, connectionless (broadcast) - point-to-all-points, connectionless (system broadcast) - point-to-point, connectionless - point-to-point, connection-oriented
key_select:	shall indicate whether the communication shall be protected using the Tool Key, or whether the key shall be used as stored for the ASAP (IA_Index of the communication partner) from the Point-to-point Key Table.
serial_number	The KNX Serial Number of the communication partner or the value 0.

The local S-AL shall start a time-out timer of 6 s when the message is forwarded to the local TL.

If the remote S-AL receives a T_Data_Individual.ind, T_Data_Connected.ind, T_Data_Broadcast.ind or T_Data_SystemBroadcast.ind primitive with TSDU = S-A_Sync_Req-PDU, then it shall do the following.

1. If the remote S-AL has less than 1 s before already responded to a preceding S-A_Sync_Req-PDU, then it shall ignore this new S-A_Sync_Req-PDU; it shall not evaluate the security or respond.
 2. If the remote S-AL has not responded to an, S-A_Sync_Req-PDU less than 1 s before, then it shall evaluate the field KNX Serial Number.
 - If the S-A_Sync_Req-PDU is received on system broadcast – or broadcast communication mode, then the S-AL shall verify that the field KNX Serial Number differs from 0. If this is not the case, then the S-A_Sync_Req-PDU shall be ignored. This shall not be logged in the Security Failures Log (see PID_SECURITY_FAILURES_LOG in [04]).
 - If the S-A_Sync_Req-PDU is received on point-to-point connectionless or – connection-oriented communication mode and the contained KNX Serial Number differs from 0 but thus not equal the own KNX Serial Number, then the S-A_Sync_Req-PDU shall be ignored. This shall neither be logged in the Security Failures Log (see PID_SECURITY_FAILURES_LOG in [04]).
 - The S-A_Sync_Req-PDU shall be processed further
 - ▲ If the field KNX Serial Number differs from 0 and equals the own KNX Serial Number, or
 - ▲ If the field KNX Serial Number equals 0.
 - The S-A_Sync_Req-PDU shall not be processed further in any other case.
- NOTE 39 The evaluation of the KNX Serial Number shall be done regardless of the communication mode, so, for system- broadcast communication mode as well as for point-to-point communication mode.
- NOTE 40 This preceding evaluation of the KNX Serial Number allows a receiver to quickly check if it is addressed over its KNX Serial Number and save the energy of the further security checks if it is not addressed.
3. If the remote S-AL processes the S-A_Sync_Req-PDU further, then it shall verify the security. If the flag T is set in the S-A_Sync_Req-PDU then it shall use the Tool Key; otherwise, it shall use the key that is in the Point-to-point Key Table with the IA_Index on which the Source Address of the message is found in the *Security Individual Address Table*.

If the remote S-AL has any error, it shall not respond to the S-A_Sync.req.

- If the flag T in the SCF is not set and the remote S-AL does not find the Source Address of the S-A_Sync.req message in its *Security Individual Address Table*, then it shall ignore the message.
- If the remote S-AL encounters an error in the verification of the security, then it shall ignore the message.

4. If the remote S-AL can properly decrypt the S-A_Sync_Req-PDU, then it shall respond with an S-A_Sync.res service primitive as specified in Figure 110.

Secure APCL	SCF	Challenge XOR Random	SeqNr _{remote}	SeqNr _{local}	MAC
10 bit	1 octet	6 octets	6 octets	6 octets	4 octets
3F1h	9Bh				

**Figure 110 – Format of the S-A_Sync_Res-PDU
(EXAMPLE – SCF with Tool Key and SBC set)**

NOTE 41 The field KNX Serial Number of the S-A_Sync_Req-PDU is in the S-A_Sync_Res-PDU replaced by the SeqNr_{local}.

- The remote S-AL shall create a 6 octet Random value and XOR it with the Challenge. The result shall be put in the S-A_Sync_Res-PDU in plain text in the field *Challenge XOR Random*.
- SeqNr_{remote} shall be the SeqNr that the remote S-AL will itself use in further secure communication.
- For the value of SeqNr_{local}, the remote S-AL shall compare the received SeqNr_{local} with the SeqNr that it has stored for that communication partner (named SeqNr_{local,stored})
 - In case the local S-AL uses the Tool Key, SeqNr_{local,stored} shall thus be the Sequence Number for Tool Access (see [04]).
 - In case the local S-AL is any other communication partner, SeqNr_{local,stored} shall thus be the Last Valid SeqNr as stored for that partner in the Security Individual Address Table (see PID_SECURITY_INDIVIDUAL_ADDRESS_TABLE in [04]).

NOTE 43 As SeqNr_{local} is a next SeqNr in subsequent communication, whereas SeqNr_{local,stored} is a last valid SeqNr from the communication partner, SeqNr_{local,stored} has to be compared with SeqNr_{local}-1. As noted above, a SeqNr_{local} of 0 will not be accepted. Within the S-A_Sync_Res-PDU the SeqNr_{local} has to be increased by 1 again to be the sequence number expected from the MaC in subsequent communication.

The remote S-AL shall assume the highest of both values as further SeqNr that it expects from the local S-AL.

IF	THEN
1. (SeqNr _{local} -1) ≤ SeqNr _{local,stored}	SeqNr _{local,stored} remains unchanged
2. (SeqNr _{local} -1) > SeqNr _{local,stored}	SeqNr _{local,stored} shall be set to (SeqNr _{local} -1)

The remote S-AL shall thus set SeqNr_{local,stored} to the maximal value of both (SeqNr_{local} -1) and the current stored value of SeqNr_{local,stored}.

In the S-A_Sync_Res-PDU the remote S-AL shall then include the concluded value of SeqNr_{local,stored} +1.

- The remote S-AL user shall set the comm_mode parameter of the S-A_Sync_Res-PDU equal to the value of the comm_mode parameter of the request that it answers. It can thus be:
 - point-to-point (connectionless or connection-oriented if a TL-connection exists)
The local S-AL (requester) shall be addressed with an ASAP, which shall be the IA_Index of the local (requester) IA in the Security Individual Address Table of the remote S-AL, that shall be mapped to an Individual Address by the remote Transport Layer.
 - broadcast, or
 - system broadcast
- The remote S-AL shall secure the response with authentication and confidentiality using the type of key that is used in the request; this “key_response” shall thus either be
 - the Tool Key or
 - the key that is configured in its Point-to-point Key Table for the IA_Index of the IA of the requester.

The flag T in the SCF shall be set accordingly.

The remote S-AL shall protect the S-A_Sync_Res-PDU with authentication and confidentiality as specified for the S-A_Data-service as specified in 5.1.3.5;

$A = SCF$

$P = SeqNr_{remote} \mid SeqNr_{local}$

Ctr_0 and B_0 shall be as in 5.1.3.2.

The remote S-AL shall in function of the parameter *comm_mode* forward the S-A_Sync.res with a *T_Data_Individual.req*, *T_Data_Connected.req*, *T_Data_Broadcast.req* or *T_Data_SystemBroadcast.req* to the remote Transport Layer. The parameters ASAP (*IA_Index*) and priority shall be mapped to the corresponding parameters TSAP (*IA*) and priority of the Transport Layer service primitive, the TSDU shall be an S-A_Sync_Res-PDU.

NOTE 44 In line with NOTE 38, the transmission of an S-A_Sync_Res-PDU shall not alter the Sequence Number Sending of the remote S-AL.

The principle message sequence shall be as follows. (Precise Management Procedures are specified in [05] as *DM_SecureSync_SBC* and *DM_SecureSync_RCI*).

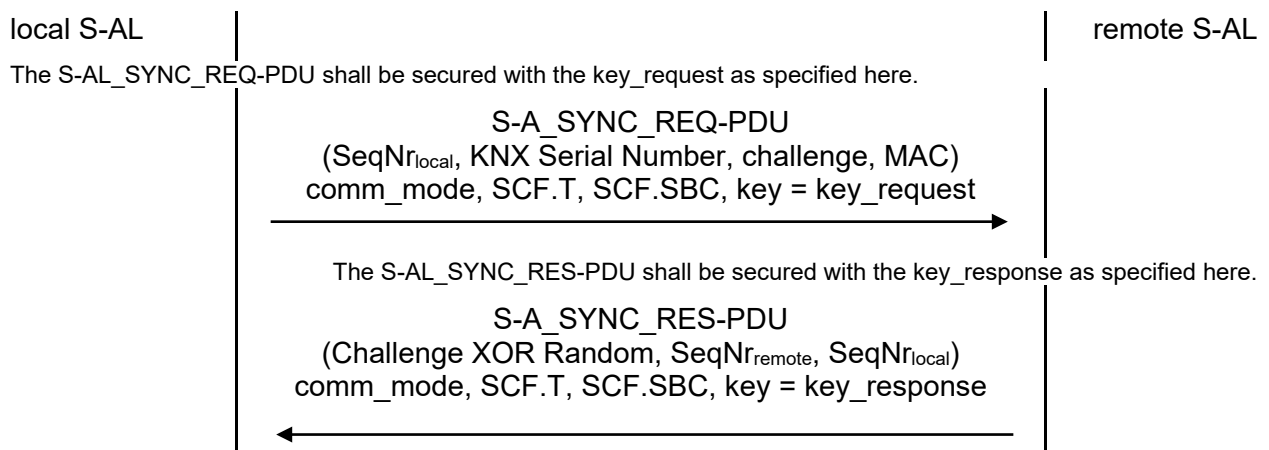


Figure 111 – Principle S-A_Sync-service sequence

If the local S-AL receives a *T_Data_Individual.ind*, *T_Data_Connected.ind*, *T_Data_Broadcast.ind* or *T_Data_SystemBroadcast.ind* primitive with TSDU = S-A_Sync_Res-PDU then it shall do the following.

- It shall calculate the value of the parameter *Random* by performing the XOR operation of the field *Challenge XOR Random* with the original *Challenge* that it created itself in the initiating S-A_Sync_req-PDU.
- Then, it shall verify the security.
 - ▲ For the first six octets of Ctr_j it shall use the above calculated *Random* value; see Figure 102.
 - ▲ If the flag T is set in the S-A_Sync_Res-PDU then it shall use the Tool Key; otherwise, it shall use the key that is through the Security Individual Address Table and the Point-to-point Key Table linked with the Source Address of the message.
- If the local S-AL can properly decrypt the message, then it shall do the following.
 - **Comparison 1:** **Conclusion on $SeqNr_{remote}$**
If the local S-AL stores any value of the $SeqNr$ of the remote S-AL, then it shall compare the stored value ($SeqNr_{remote,stored}$) with the value of $SeqNr_{remote}$ as received in the S-A_Sync_Res-PDU ($SeqNr_{remote,received}$).

NOTE 45 As $SeqNr_{remote,received}$ is a next $SeqNr$ for subsequent communication and $SeqNr_{remote,stored}$ is a last valid $SeqNr$ from the communication partner, it is the value $SeqNr_{remote,received} - 1$ that has to be compared with $SeqNr_{remote,stored}$.

IF	THEN
1. $(\text{SeqNr}_{\text{remote},\text{received}} - 1) \leq \text{SeqNr}_{\text{remote},\text{stored}}$ NOTE 46 This is an exception: the received SeqNr is smaller than the locally stored image. This should normally not happen.	$\text{SeqNr}_{\text{remote},\text{stored}}$ remains unchanged - Option 1: Recommended The local S-AL shall not update its locally stored value of the communication partner's SeqNr. This means that the local S-AL will not be able to communicate further with the remote device. If possible, the local S-AL user – if this is a Configuration Client – should solve the situation by reading out the SeqNr of the remote S-AL in that device's communication partner and by reprogramming the remote S-AL device. - Option 2: Allowed The local S-AL may take the value $(\text{SeqNr}_{\text{remote},\text{received}} - 1)$ as new value of $\text{SeqNr}_{\text{remote},\text{stored}}$ and continue. If the local S-AL user is the Configuration Client then it should ask the MaC user for confirmation to continue.
2. $(\text{SeqNr}_{\text{remote},\text{received}} - 1) > \text{SeqNr}_{\text{remote},\text{stored}}$ NOTE 47 This is the typical case if the local S-AL is out of sync with the remote S-AL.	$\text{SeqNr}_{\text{remote},\text{stored}}$ shall be set to $(\text{SeqNr}_{\text{remote},\text{received}} - 1)$ The local S-AL shall update (increment) its locally stored value of the communication partner's SeqNr with the received value.

If the local S-AL does not store any SeqNr value of the remote S-AL then it shall use the $(\text{SeqNr}_{\text{remote},\text{received}} - 1)$ as value for the $\text{SeqNr}_{\text{remote},\text{stored}}$.

- **Comparison 2:** **Conclusion on $\text{SeqNr}_{\text{local}}$**
The local S-AL shall compare the value of $\text{SeqNr}_{\text{local}}$ ($\text{SeqNr}_{\text{local},\text{received}}$) as received in the S-A_Sync_Res-PDU with its own local SeqNr ($\text{SeqNr}_{\text{local},\text{stored}}$).

IF	THEN
1. $\text{SeqNr}_{\text{local},\text{received}} < \text{SeqNr}_{\text{local},\text{stored}}$ NOTE 48 This is an exception: the remote S-AL should according to the above already have compared the $\text{SeqNr}_{\text{local}}$ that the local S-AL had sent in the S-A_Sync_REQ-PDU with its own value and have returned the highest value. This can however happen if the local S-AL has in between the request and the response sent out S-A_Data-PDUs.	$\text{SeqNr}_{\text{local},\text{stored}}$ remains unchanged The local S-AL shall not update its locally stored value of its own SeqNr.
2. $\text{SeqNr}_{\text{local},\text{received}} > \text{SeqNr}_{\text{local},\text{stored}}$	$\text{SeqNr}_{\text{local},\text{stored}}$ shall be set to $\text{SeqNr}_{\text{local},\text{received}}$ The local S-AL shall update (increment) its own Sequence Number Sending with the received value.

- The local S-AL shall advise the local S-AL-user on the result through an S-A_Sync.Acon service primitive with result = "ok".

- If the local S-AL cannot properly decrypt the message, like if there is a security error or a time-out, then the result shall be “not_ok”.

S-A_Sync.Acon(ASAP, comm_mode, key_select, result)

ASAP:	shall either indicate the IA_Index of the IA of the communication partner or be an indication that the Tool Key is used.	
comm_mode:	This shall be one of the following.	
	- point-to-domain, connectionless (broadcast) - point-to-all-points, connectionless (system broadcast) - point-to-point, connectionless - point-to-point, connection-oriented	
key_select:	shall indicate whether the communication shall be protected using the Tool Key, or whether the key shall be used as stored for the IA (through the IA_Index of the communication partner in the Security Individual Address Table and the Point-to-point Key Table).	
result:	ok:	the Sequence Number of the remote S-AL is properly received
	not_ok:	there is an error in the request of the Sequence Number of the remote S-AL.

Error – and exception handling

- The error handling for decryption failure is given in the specification above.
- If the S-AL receives any A_Secure-PDU with SCF encoding for an S-A_Sync_Req-PDU or S-A_Sync_Res-PDU that is not transferred using point-to-point connectionless, – connection-oriented, broadcast - or system broadcast communication mode, then it shall ignore the message.

5.4 Registration of Security Failures

The **handling** of security failures is mandatory and is contained within the specification of the various parts of the S-AL. This clause only considers the **registration** of security failures.

The Security Failure is specified to only have to do with the failure of the security check in the S-AL service. It is not related to any other exception in the S-AL.

A Security Failure is thus only considered in the operation “Check the Security” in the example flow-charts in Figure 114, Figure 116 and Figure 118.

The following events shall be interpreted as Security Failure.

These events shall lead to an increment of the “cryptographic errors” for (see PID_SECURITY_FAILURES_LOG in [04]) in the Security Failure Logs:

- Decryption failure
- Mismatch between the calculated MAC-value and the MAC-value received.

The following event may be interpreted as Security Failure.

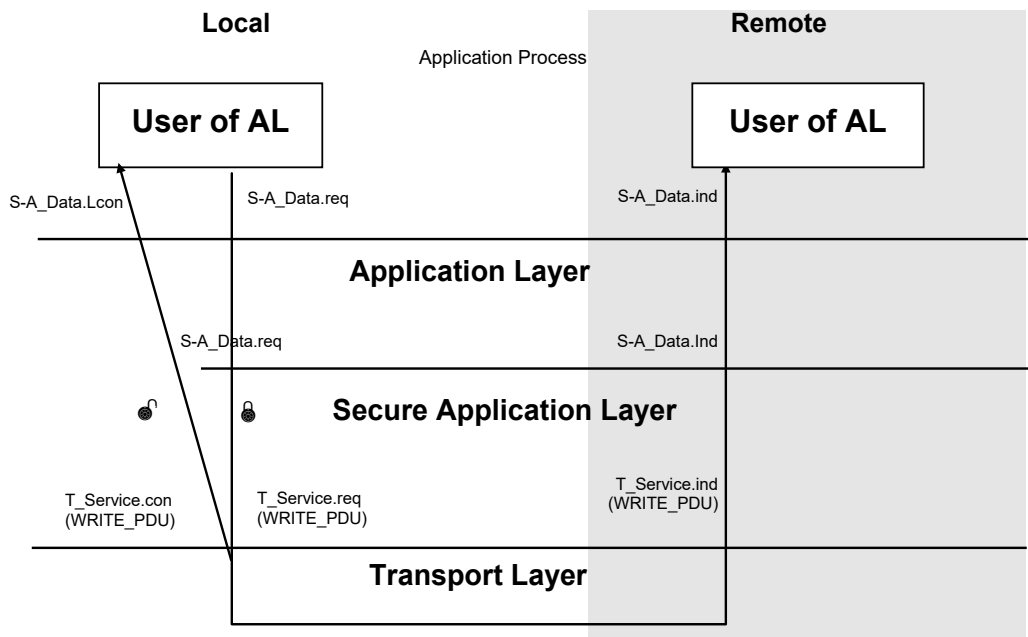
Reception of an S-A_Data-PDU with a value of the field SeqNr that is lower than the value stored for the communication partner in the Security Individual Address Table (see PID_SECURITY_INDIVIDUAL_ADDRESS_TABLE in [04]) if the local S-AL assumes having a valid value for this SeqNr may in the Security Failures Log (see PID_SECURITY_FAILURES_LOG in [04]) increment the “Counter of SeqNr errors”.

5.5 Secure Handling of the Transport Layer services

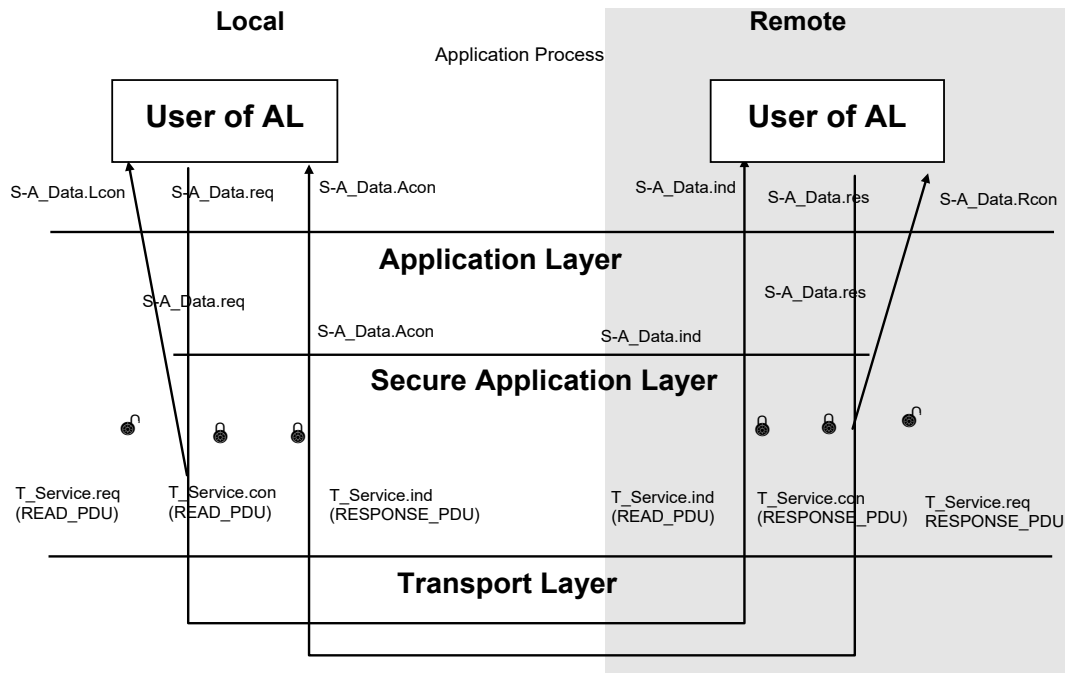
5.5.1 Common requirements

The Secure AL-Service shall be invoked by the TL-service primitives request (req), indication (ind) and confirmation (con).

- In reception direction, for each TL-service, if the received message is secured, then the *S-AL* shall verify the security.
- In transmission direction, the S-AL shall apply the security or not according to the security service parameters *par_auth* and *par_conf*. If security is applied it shall use the Security Resources.



**Figure 112 - Interactivity of the Application Layer
for services that are not remotely confirmed**



**Figure 113 - Interactivity of the Application Layer
for services that are remote confirmed**

Any Transport Layer service indication primitive containing a Secure AL-Service indication shall in any case be handled by the S-AL. (This is not shown in Figure 98.)

5.5.2 Common error handling

- 1 In reception direction, if the security check ³⁶⁾ fails AND only authentication is requested, then it would be possible to forward the Plain APDU to the S-AL, because it is available unencrypted in the Secure APDU. This means that it would be possible to access a secure DP that requires only authentication also via a badly protected APDU. This is not allowed. There are no exceptions to a failed security check.
- 2 In reception direction, if the Load State of the Security Interface Object (PID_LOAD_STATE_CONTROL) has any other Load State than “Loaded” then the evaluation of any of the contained Resources (Point-to-point Key Table, Group Key Table or Zone Key Table) shall also lead to the result “TSAP not found in the key table”.
- 3 In transmission direction, if an AL-service is requested with any of the flags *par_auth* or *par_conf* set and if the *Load State of the Security Interface Object* has any other Load State than the state “Loaded” then this AL-service shall be confirmed negatively.
- 4 In reception direction, if any S-AL service (S-A_Data or S-A_Sync) is requested using TL communication mode for which this is not defined, then this request shall be ignored; the Frame shall be thrown away.

The allowed communication modes are part of the specification of each service (see 5.2.1 and 0); the overview is given in Table 1.

EXAMPLE 5 If the AIL handles a T_Data_Group.ind that contains an S-A_Sync-PDU then it shall ignore this service.

³⁶⁾ The specification of the “security check” contains common requirements (e.g., handling of unsupported values of SCF), but also requirements that depend on the security algorithm (e.g., verification of a sequence number, verification of a MAC or other). Therefore, the “security check” is specified in the security algorithm specific clauses; for CCM, this is specified in 5.1.3.4.2 and 5.1.3.5.2).

5.5.3 Secure AL-service on Multicast Communication Mode - T_Data_Group

5.5.3.1 References

For the specification of the T_Data_Group-service and the handling of TSAPs and ASAPs in the Group Object Association Table, please refer to clause 3.1.1.

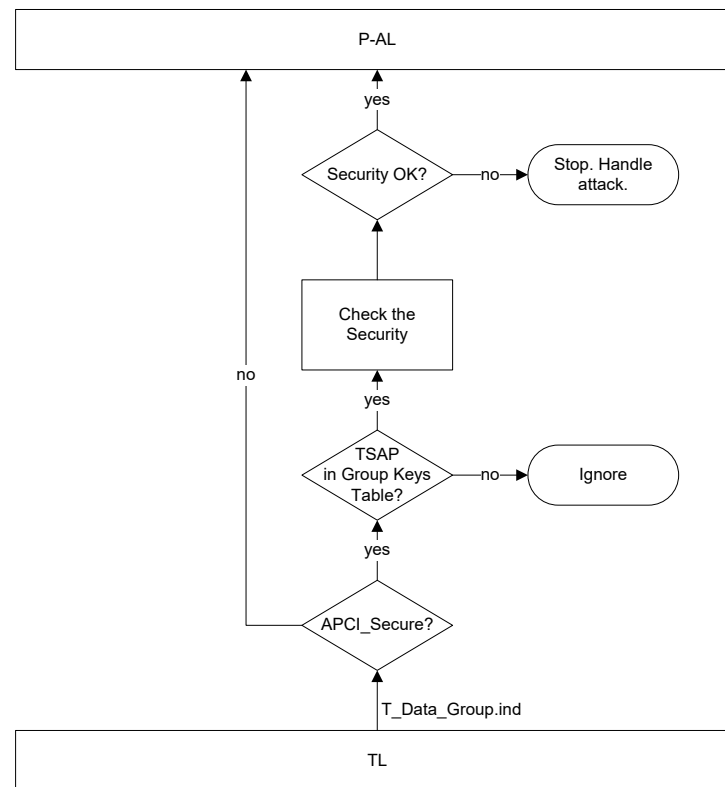
5.5.3.2 Reception - T_Data_Group.ind

The below sequence specifies a decision tree to come to a single conclusion on handling the T_Data_Group.ind.

1. If the T_Data_Group.ind contains an S-A_Data-PDU then the S-AL shall search the GA_Index in the Group Key Table.
 - 1.1 If the GA_Index is found in the Group Key Table, then the S-AL shall check the Tool Access flag in the SCF.
 - 1.1.1 If the Tool Access flag indicates the use of the specific tool entry, then the S-AL shall throw away the S-A_Data-PDU. The message shall not be forwarded to the P-AL. The S-AL shall consider this as an attack and shall handle this in the “Counter of Access or Roles errors” in the Security Failures counter and as Error Type 04h and as “*Error against the Access and Roles*” see PID_SECURITY_FAILURES_LOG in [04].
 - 1.1.2 If the Tool Access flag does not indicate the use of the specific tool entry, then the S-AL shall decrypt the contained S-A_Data-SDU according to the security features indicated in the SCF and check the security of the message.
 - 1.1.2.1 If the security check ³⁶⁾ results positive, then the S-AL shall inform the P-AL according to the contained A_GroupValue_Read.ind(/.Acon) or A_GroupValue_Write.ind(/.Acon) as decoded from S-A_Data-service. The security service parameters par_auth and par_conf shall be set as used in the S-A_Data-PDU and the link_index shall be forwarded as well.
 - 1.1.2.2 If the security check results negative, then the S-AL shall throw away the S-A_Data-PDU. The message shall not be forwarded to the P-AL. (See also error handling 1 above.)
The S-AL shall consider this as an attack and shall handle this. See also PID_SECURITY_FAILURES_LOG in [04].
 - 1.2 If GA_Index is not found in the Group Key Table, then the S-A_Data-PDU shall be ignored. (See also exception 2). The message shall not be forwarded to the P-AL.
2. If the T_Data_Group.ind does not contain an S-A_Data-PDU then the S-AL shall forward the message unchanged to the P-AL. The security service parameters par_auth and par_conf shall both be cleared; the parameter link_index shall be “none”. The T_Data_Group.ind can at this point only contain an APCI_GroupValue_Write, APCI_GroupValue_Read or an APCI_GroupValue_Response.

Error - and exception handling

The common error handling applies, as specified in 5.5.2. There is no further error – and exception handling specific to the service primitives T_Data_Group.ind or T_Data_Group.con.

Flowchart example**Figure 114 – Handling of T_Data_Group.ind by S-AL (informative)****5.5.3.3 Transmission - T_Data_Group.req**

If the S-AL handles an APDU (APCI + Data) that shall be transmitted using a T_Data_Group.req then the S-AL shall conclude on the handling based on the security service parameters *par_auth* and *par_conf* of the AL-service according to the following decision tree.

1. The S-AL shall check the security service parameters *par_auth* and *par_conf* of the AL-service.
 - 1.1 If any of these parameters is set, then the S-AL shall check whether the TSAP (= GA_Index) used for the T_Data_Group.req is contained the *Group Key Table*.
 - 1.1.1 If the TSAP is found in the Group Key Table, then the S-AL shall check if the link_index requires the use of the Tool Key.
 - 1.1.1.1 If the usage of the Tool Key is requested, then the S-AL shall confirm the AL-service negatively to the AIL.
 - 1.1.1.2 If the usage of the Tool Key is not requested, then the S-AL shall apply the security (authentication and/or confidentiality) as requested in the service and build the Secure APDU (see 5.1.3.1), which shall be forwarded with a T_Data_Group.req to the TL. (See common error 3.)
 - 1.1.2 If the TSAP is not found in the *Group Key Table*, then the S-AL shall confirm the AL-service negatively to the AIL.
 - 1.2 If none of the security service parameters is set, then the S-AL shall forward the APDU unchanged with a T_Data_Group.req to the TL.

Error - and exception handling

The common error handling applies, as specified in 5.5.2. There is no further error – and exception handling specific to the service primitive T_Data_Group.req.

Flowchart example

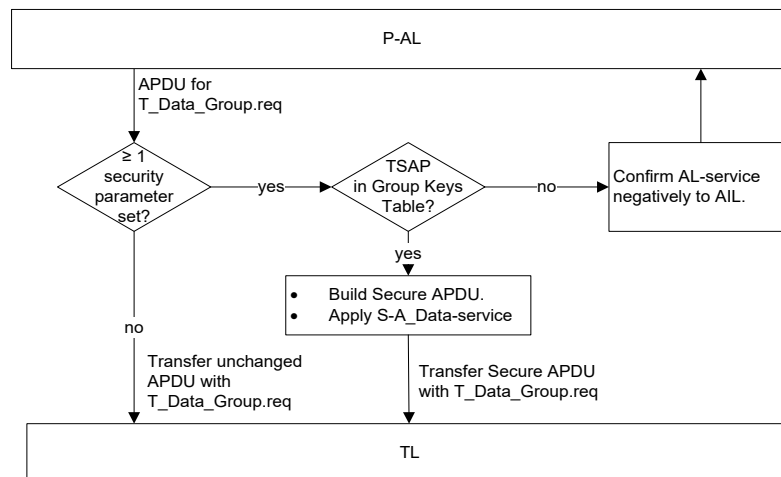


Figure 115 - Handling of T_Data_Group.req by S-AL (informative)

5.5.4 Secure AL-service on Multicast Communication Mode - T_Data_Tag_Group

5.5.4.1 References

For the specification of the T_Data_Tag_Group-service, please refer to [02].

In the below, the *Zone Key Table* is used. The TSAP shall be composed of the “Extended Frame Format” and the Zone Address of the TL-service or AL-service primitive being handled.

5.5.4.2 Reception - T_Data_Tag_Group.ind and T_Data_Tag_Group.con

The below sequence specifies a decision tree to come to a single conclusion on handling the T_Data_Tag_Group.ind.

1. If the T_Data_Tag_Group.ind contains an S-A_Data-PDU then the S-AL shall search the TSAP in the *Zone Key Table*.
 - 1.1 If the TSAP is found in the *Zone Key Table* – possibly with the evaluation of wildcards -, then the S-AL shall check the Tool Access flag in the SCF.
 - 1.1.1 If the Tool Access flag indicates the use of the specific tool entry, then the S-AL shall throw away the S-A_Data-PDU. The message shall not be forwarded to the P-AL. The S-AL shall consider this as an attack and shall handle this in the “Counter of Access or Roles errors” in the Security Failures counter and as Error Type 04h and as “*Error against the Access and Roles*” see PID_SECURITY_FAILURES_LOG in [04].
 - 1.1.2 If the Tool Access flag does not indicate the use of the specific tool entry, then the S-AL shall decrypt the contained S-A_Data-SDU and check the security of the message.
 - 1.1.1.1 If the security check results positive, then the S-AL shall inform the P-AL according to the contained A_GroupPropValue_Read.ind, A_GroupPropValue_Response.ind, A_GroupPropValue_Write.ind or A_GroupPropValue_InfoReport.ind as decoded from the S-A_Data-service.

The security service parameters *par_auth* and *par_conf* shall be set as used in the S-A_Data-PDU and the link_index shall be forwarded as well.

- 1.1.2.1 If the security check results negative, then the S-AL shall throw away the S-A_Data-PDU. The message shall not be forwarded to the P-AL. (See also error handling 1 above.)
The S-AL shall consider this as an attack and shall handle this. See also PID_SECURITY_FAILURES_LOG in [04].
- 1.2 If the zone address is not found in the *Zone Key Table*, then the S-A_Data-PDU shall be ignored. (See also error handling 2 above). The message shall not be forwarded to the P-AL.
2. If the T_Data_Tag_Group.ind does not contain an S-A_Data-PDU then the S-AL shall forward the T_Data_Tag_Group.ind unmodified to the P-AL. The security service parameters *par_auth* and *par_conf* shall both be cleared; the parameter *link_index* shall be “none”.

Error - and exception handling

The common error handling applies, as specified in 5.5.2. There is no further error – and exception handling specific to the service primitives T_Data_Tag_Group.ind and T_Data_Tag_Group.con.

Flowchart example

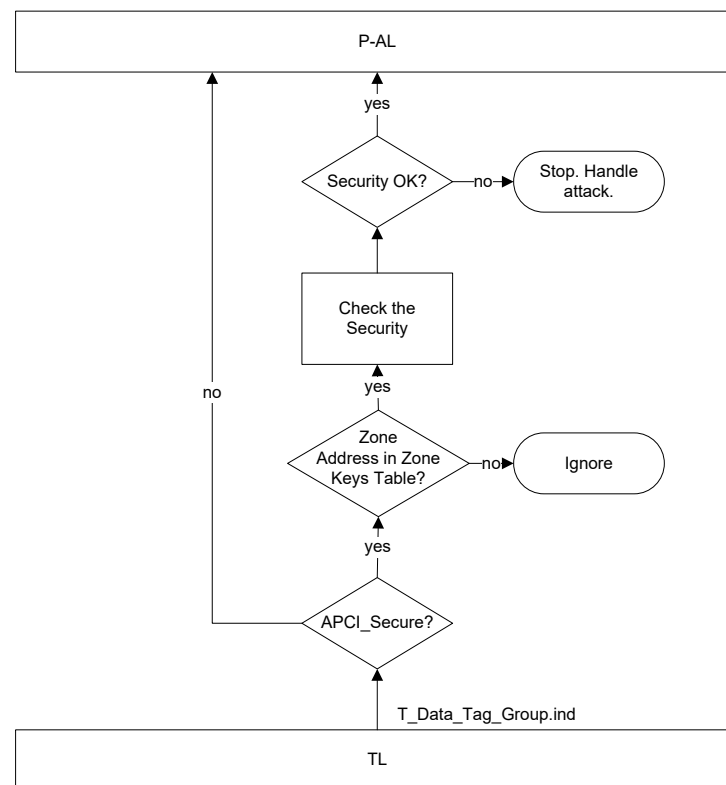


Figure 116 – Handling of T_Data_Tag_Group.ind by S-AL (informative)

5.5.4.3 Transmission – T_Data_Tag_Group.req

If the S-AL handles an APDU (APCI + Data) that shall be transmitted using a T_Data_Tag_Group.req then the S-AL shall conclude on the handling based on the security service parameters *par_auth* and *par_conf* of the AL-service according to the following decision tree.

1. The S-AL shall check the security service parameters *par_auth* and *par_conf* of the AL-service.
 - 1.1 If any of these parameters is set, then the S-AL shall search the TSAP (Extended Frame Format and Zone Address) of the AL-service in the *Zone Key Table*.
 - 1.1.1 If the TSAP is found in the Zone Key Table, then the S-AL shall check if the *link_index* requires the use of the Tool Key.

- 1.1.1.1 If the usage of the Tool Key is requested, then the S-AL shall confirm the AL-service negatively to the AIL.
- 1.1.1.2 If the usage of the Tool Key is not requested, then the S-AL shall apply the security (authentication and/or confidentiality) as requested in the AL-service parameter and build the Secure APDU (see 5.1.3.1), which shall be forwarded with a T_Data_Tag_Group.req to the TL.
- 1.1.2 If the TSAP is not found in the Zone Key Table, then the S-AL shall confirm the AL-service negatively to the AIL.
- 1.2 If none of the security service parameters is set, then the S-AL shall forward the APDU unchanged with a T_Data_Tag_Group.req to the TL.

Error – and exception handling

There is no specific error – or exception handling specified.

Flowchart example

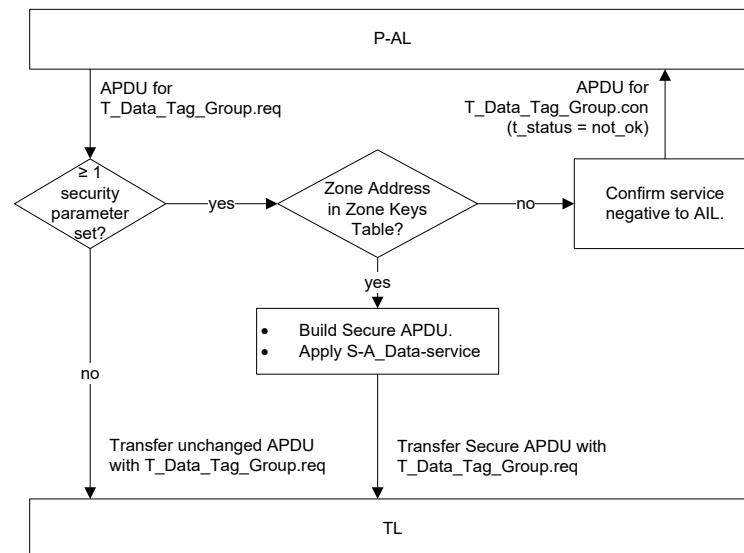


Figure 117 - Handling of T_Data_Tag_Group.req by S-AL (informative)

5.5.5 Secure AL-Service on point-to-point communication mode

5.5.5.1 Introduction

This clause is valid for the TL-services T_Data_Individual as well as T_Data_Connected.

For the specification of these services, please refer to [02].

NOTE 49 The TL-services T_Connect and T_Disconnect and the acknowledgement of the connection-oriented Transport Layer services (T_ACK-PDU and T_NAK-PDU) are handled by the TL itself. These do not contain an APDU (APCI) and can thus not be handled by the S-AL. This means that there is no security on the opening or closing of a TL-connection to a secure device or on the confirmations of connection-oriented services.

5.5.5.2 Reception – T_Data_Individual.ind and T_Data_Connected.ind

The below sequence specifies a decision tree to come to a single conclusion on handling the T_Data_Individual.ind and T_Data_Connected.ind.

1. If the TL-service primitive contains an S-A_Data-PDU then the S-AL shall check if the flag *Tool Access* is set in the SCF.

On KNX RF, additionally the KNX RF Domain Address of the sender shall be compared and be equal.

- 1.1 If the flag *Tool Access* indicates the use of the specific tool entry, then the S-AL shall use the Tool Key to try to decrypt the contained S-A_Data-PDU according to the security features indicated in the SCF and check the security of the message.
 - 1.1.1 If the security check results positive, then the S-AL shall inform the P-AL according to the contained AL-service primitive as decoded from the S-A_Data-service. The security service parameters *par_auth* and *par_conf* shall be set according to the use of authentication respectively confidentiality in the S-A_Data-SDU; the *link_index* shall indicate that the Tool Key is used.
 - 1.1.2 If the security check results negative, then the S-AL shall throw away the S-A_Data-PDU. The message shall not be forwarded to the P-AL. (See also error handling 1 above.)

The S-AL shall consider this as an attack and shall handle this. See also PID_SECURITY_FAILURES_LOG in [04].
- 1.2 If the flag *Tool Access* does not indicate the use of the specific tool entry, then the S-AL shall search the Source Address in the Security Individual Address Table (see PID_SECURITY_INDIVIDUAL_ADDRESS_TABLE in [04]).

If it is not found, then the message shall be ignored. This shall not be logged in the *Security Failures Log*.

If it is found, it shall look for the *IA_Index* where this is found in the Point-to-point Key Table (see PID_P2P_KEY_TABLE in [04]).

 - 1.2.1 If the *IA_Index* of the Source Address is found in the Point-to-point Key Table, then the S-AL shall try to decrypt the contained S-A_Data-SDU according to the security features indicated in the SCF and check the security of the message.
 - 1.2.1.1 If the security check results positive, then the S-AL shall inform the P-AL according to the contained AL-service primitive as decoded from the S-A_Data-service. The security service parameters *par_auth* and *par_conf* shall be set according to the use of authentication respectively confidentiality in the S-A_Data-SDU and the *IA_Index* of the secure link shall be forwarded as well.
 - 1.2.1.2 If the security check results negative, then the S-AL shall throw away the S-A_Data-PDU. The message shall not be forwarded to the P-AL. (See also error handling 1 above.)

The S-AL shall consider this as an attack and shall handle this. See also PID_SECURITY_FAILURES_LOG in [04].
 - 1.2.2 If the Source Address is not found through the Security Individual Address Table and the Point-to-point Key Table, then the S-A_Data-PDU shall be ignored. The message shall not be forwarded to the P-AL. This shall not be logged in the *Security Failures Log*.
2. If the TL-service primitive does not contain an S-A_Data-PDU then the S-AL shall forward the message unchanged to the P-AL. The security service parameters *par_auth* and *par_conf* shall both be cleared; the parameter *IA_Index* shall be “none”.

Error – and exception handling

The common error handling applies, as specified in 5.5.2.

1. There is no relation between a hack attempt and the TL state machine. If a security check of a T_Data_Connected.ind fails, then this has no repercussions on the TL State Machine. Any open Transport Layer connection is not closed.

Flowchart example

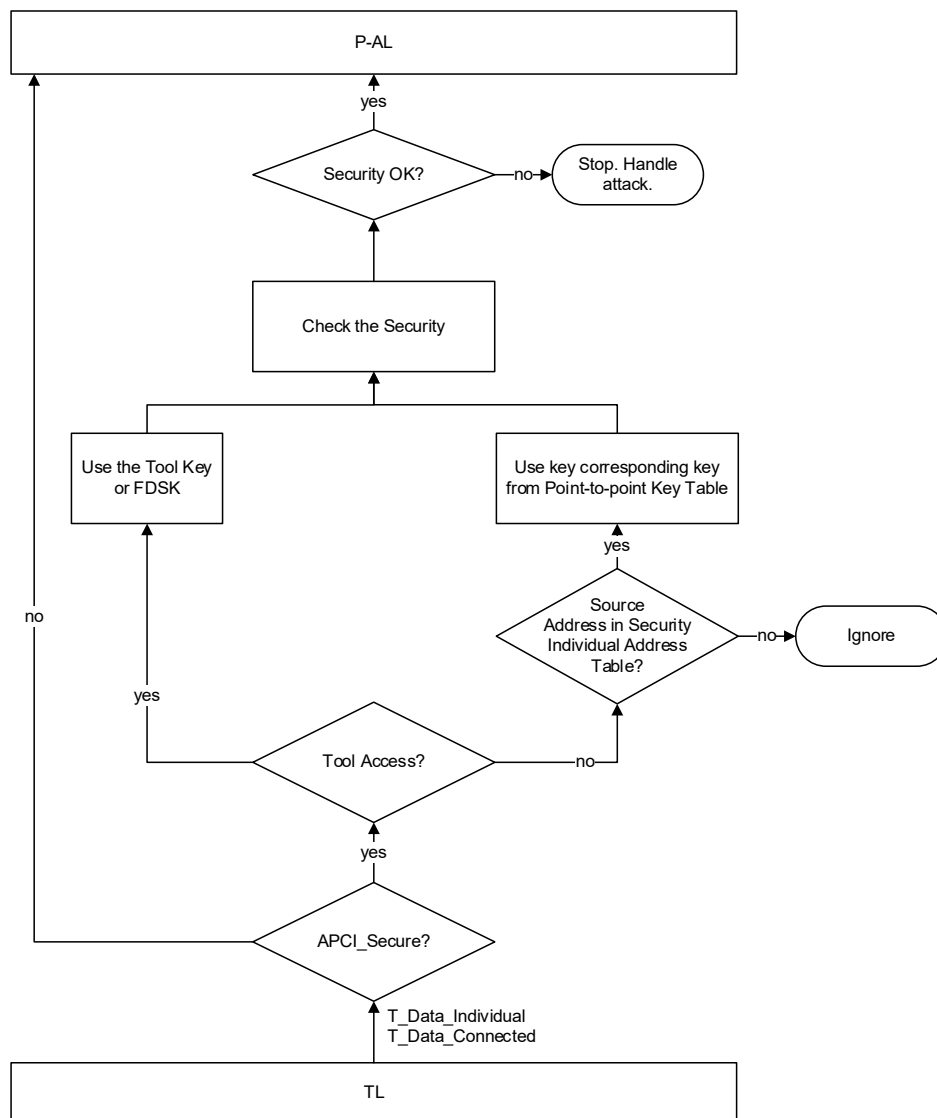


Figure 118 - Handling of T_Data_Individual.ind and T_Data_Connected.ind by S-AL (informative)

5.5.5.3 Transmission – T_Data_Individual.req and T_Data_Connected.req

If the S-AL handles an APDU (APCI + Data) that shall be transmitted using a T_Data_Individual.req or T_Data_Connected.req then the S-AL shall conclude on the handling based on the security service parameters *par_auth* and *par_conf* of the AL-service in the following decision tree.

1. The S-AL shall check the security service parameters *par_auth* and *par_conf* of the AL-service.
 - 1.1 If any of these parameters is set, then the S-AL shall check whether the link_index points to the use of the Tool Key.

- 1.1.1 If the Tool Key is used then the S-AL shall apply the security (authentication and/or confidentiality) as requested in the service and build the Secure APDU with the Tool Key (see 5.1.3.1), which shall be forwarded with a T_Data_Individual.req or T_Data_Connected.req to the TL.
- 1.1.2 If the Tool Key is not used then the S-AL shall check whether the TSAP (IA) used for the T_Data_Individual.req or T_Data_Connected.req is contained in the Security Individual Address Table and referred through its IA_Index in the Point-to-point Key Table.
 - 1.1.2.1 If both checks are positive, then the S-AL shall apply the security (authentication and/or confidentiality) as requested in the service and build the Secure APDU (see 5.1.3.1), which shall be forwarded with a T_Data_Individual.req or T_Data_Connected.req to the TL.
 - 1.1.2.2 If any of these checks is negative, then the S-AL shall confirm the AL-service negatively to the AIL.
- 1.2 If none of the security service parameters is set then the S-AL shall forward the APDU unchanged with a T_Data_Individual.req or T_Data_Connected.req to the TL.

Error – and exception handling

The common error handling applies, as specified in 5.5.2. There is no further error – and exception handling specific to the service primitives T_Data_Individual.req or T_Data_Connected.req.

Flowchart example

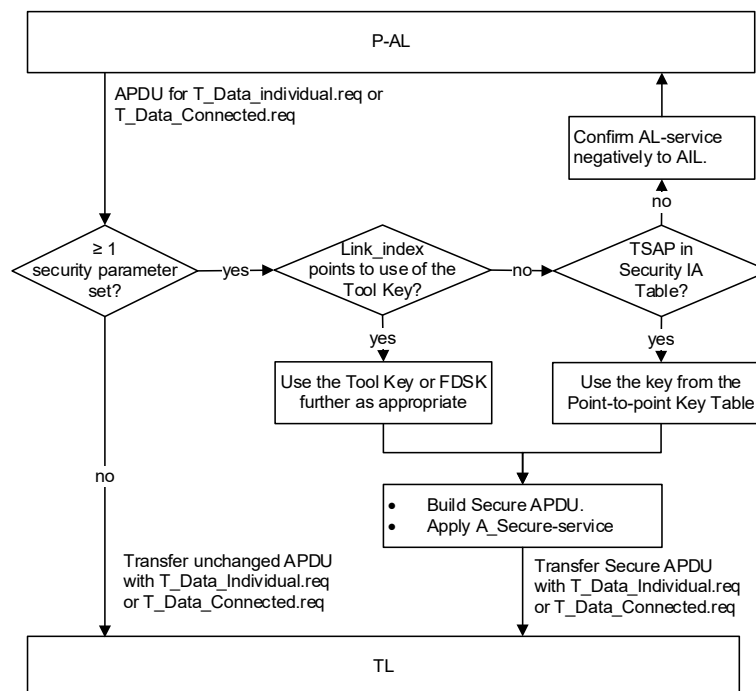


Figure 119 - Handling of T_Data_Individual.req and T_Data_Connected.req by S-AL (informative)

5.5.6 Secure AL-Service on system broadcast communication mode

5.5.6.1 References

This clause is valid for the T_Data_SystemBroadcast-service. For the specification of this service, please refer to [02].

No Resources are defined for the storage of keys or parameters for secure system broadcast communication, opposite to point-to-point communication (Point-to-point Key Table), group communication (Group Key Table) and LTE-Mode (Zone Key Table). Secure system broadcast communication can thus only be encrypted using the Tool Key by a MaC. End devices not using the Tool Key do not have the possibility to exchange messages using system broadcast communication mode.

5.5.6.2 Reception – T_Data_SystemBroadcast.ind

1. If the TL-service primitive contains an S-A_Data-PDU and the SBC-flag is set to “system broadcast” then the S-AL shall check if the flag *Tool Access* is set in the SCF.
 - 1.1 If the flag *Tool Access* is not set, then the S-AL shall ignore that S-A_Data-PDU³⁷⁾. The message shall not be forwarded to the P-AL.
 - 1.2 If the Tool Access is set, then the S-AL shall use the Tool Key to try to decrypt the contained S-A_Data-SDU according to the security features indicated in the SCF and check the security of the message.
 - 1.2.1 If the security check results positive, then the S-AL shall inform the P-AL according to the contained AL-service primitive as decoded from the S-A_Data-service. The security service parameters *par_auth* and *par_conf* shall be set according to the use of authentication respectively confidentiality in the S-A_Data-SDU; the link_index shall indicate that the Tool Key is used.
 - 1.2.2 If the security check results negative, then the S-AL shall throw the S-A_Data-PDU. The message shall not be forwarded to the P-AL. (See also error handling 1 above.)
2. If the TL-service primitive does not contain an S-A_Data-PDU then the S-AL shall forward the message unchanged to the P-AL. The security service parameters *par_auth* and *par_conf* shall both be cleared; the parameter link_index shall be “none”.

Error – and exception handling

The common error handling applies, as specified in 5.5.2.

5.5.6.3 Transmission - T_Data_SystemBroadcast.req

If the S-AL handles an APDU (APCI + Data) that shall be transmitted using a T_Data_SystemBroadcast.req then the S-AL shall conclude on the handling based on the security service parameters *par_auth* and *par_conf* of the AL-service in the following decision tree.

1. The S-AL shall check the security service parameters *par_auth* and *par_conf* of the AL-service.
 - 1.1 If any of these parameters is set, then the S-AL shall check whether the link_index points to the use of the Tool Key.
 - 1.1.1 If the Tool Key is used then the S-AL shall apply the security (authentication and/or confidentiality) as requested in the service and build the Secure APDU (see 5.1.3.1), which shall be forwarded with a T_Data_SystemBroadcast.req to the TL.

³⁷⁾ The MaC uses system broadcast – and broadcast communication mode for the Network Configuration, encrypted with the FDSK or the Tool Key of the single target device (see 2.6.3.4.3); all other devices will receive these messages and shall not consider these as hack attempts and shall not log these.

- 1.1.2 If the Tool Key is not used then then S-AL shall confirm the AL-service negatively to the AIL.
- 1.2 If none of the security service parameters is set then the S-AL shall forward the APDU unchanged with a T_Data_SystemBroadcast.req to the TL.

Error – and exception handling

The common error handling applies, as specified in 5.5.2.

5.5.7 Secure AL-services on broadcast communication mode

5.5.7.1 References

This clause is valid for the T_Data_Broadcast-service. For the specification of this service, please refer to [02].

The Resources for handling secure broadcast communication depend on the *Tool Access* flag in the message.

- If the Tool entry is not used, then the *Security Individual Address Table* (see PID_SECURITY_-INDIVIDUAL_ADDRESS_TABLE in [04]) and the *Point-to-point Key Table* (see PID_P2P_KEY_TABLE in [04]) shall be evaluated.
- The Tool entry is used, then the *Tool Key* (see PID_TOOL_KEY in [04]) or if that is not active, the *Factory Default Setup Key* (see Factory Default Setup Key in [04]) shall be evaluated.

5.5.7.2 Reception – T_Data_Broadcast.ind

1. If the TL-service primitive contains an S-A_Data-PDU and the SBC-flag is set to “by service” then the S-AL shall check if the flag *Tool Access* is set in the SCF.

NOTE 50 The handling of received T_Data_SystemBroadcast-PDUs requires the additional verification of the possibly contained KNX Serial Number. This is not required for the T_Data_Broadcast-PDUs, as the Frame will contain a DoA and this is already verified by the Layer-2.

- 1.1 If the flag *Tool Access* is not set, then the S-AL shall search the Source Address in the Security Individual Address Table (see PID_SECURITY_-INDIVIDUAL_ADDRESS_TABLE in [04]). If it is found, it shall look for the IA_Index where this is found in the Point-to-point Key Table (see PID_P2P_KEY_TABLE in [04]).

On KNX RF, additionally the KNX RF Domain Address of the sender shall be compared and be equal.

- 1.1.1 If the IA_Index of the Source Address is found in the Point-to-point Key Table then the S-AL shall decrypt the contained S-A_Data-SDU according to the security features indicated in the SCF and check the security of the message.
 - 1.1.1.1 If the security check results positive, then the S-AL shall inform the P-AL according to the contained AL-service primitive as decoded from the S-A_Data-service. The security service parameters *par_auth* and *par_conf* shall be set according to the use of authentication respectively confidentiality in the S-A_Data-SDU and the IA_Index of the secure link shall be forwarded as well.
 - 1.1.1.2 If the security check results negative, then the S-AL shall throw away the S-A_Data-PDU. The message shall not be forwarded to the P-AL. (See also error handling 1 above.)
The S-AL shall consider this as an attack and shall handle this. See also PID_SECURITY_FAILURES_LOG in [04].

- 1.1.2 If the Source Address is not found through the Security Individual Address Table and the Point-to-point Key Table then the S-A_Data-PDU shall be ignored. The message shall not be forwarded to the P-AL.
 - 1.2 If the flag *Tool Access* is set, then the S-AL shall use the Tool Key to try to decrypt the contained S-A_Data-SDU according to the security features indicated in the SCF and check the security of the message.
 - 1.2.1 If the security check results positive, then the S-AL shall inform the P-AL according to the contained AL-service primitive as decoded from the S-A_Data-service. The security service parameters *par_auth* and *par_conf* shall be set according to the use of authentication respectively confidentiality in the S-A_Data-SDU; the *link_index* shall indicate that the Tool Key is used.
 - 1.2.2 If the security check results negative, then the S-AL shall throw the S-A_Data-PDU. No message shall be forwarded to the P-AL. (See also error handling 1 above.)
2. If the TL-service primitive does not contain an S-A_Data-PDU then the S-AL shall forward the message unchanged to the P-AL. The security service parameters *par_auth* and *par_conf* shall both be cleared; the parameter *link_index* shall be “none”.

Error – and exception handling

The common error handling applies, as specified in 5.5.2.

5.5.7.3 Transmission - T_Data_Broadcast.req

If the S-AL handles an APDU (APCI + Data) that shall be transmitted using a T_Data_Broadcast.req then the S-AL shall conclude on the handling based on the security service parameters *link_index*, *par_auth* and *par_conf* of the AL-service in the following decision tree.

1. The S-AL shall check the security service parameters *par_auth* and *par_conf* of the AL-service.
 - 1.1 If any of these parameters is set, then the S-AL shall check whether the *link_index* points to the use of the Tool Key.
 - 1.1.1 If the Tool Key is used then the S-AL shall apply the security (authentication and/or confidentiality) as requested in the service and build the Secure APDU (see 5.1.3.1), which shall be forwarded with a T_Data_Broadcast.req to the TL.
 - 1.1.2 If the Tool Key is not used and the TSAP equals an IA_Index in the Point-to-point Key Table, then S-AL shall apply the security (authentication and/or confidentiality) as requested in the service and build the Secure APDU (see 5.1.3.1), which shall be forwarded with a T_Data_Broadcast.req to the TL.

If the IA_Index is not found in the Point-to-point Key Table, then S-AL shall confirm the AL-service negatively to the P-AL.
 - 1.2 If none of the security service parameters is set then the S-AL shall forward the APDU unchanged with a T_Data_Broadcast.req to the TL.

Error – and exception handling

The common error handling applies, as specified in 5.5.2.

5.5.8 Overview of allowed combinations of keys and communication modes

Table 10 – Allowed combinations of communication modes and Security Keys

The legend (M, X) is inherited from Volume 6 “Profiles”.

M: It shall be possible to use this key with this communication mode, if supported.

EXAMPLE 6 It shall be possible to use Group Keys for T_Data_Group, if at least the device supports secure group communication.

X: This key shall not be used for this communication mode. (Whether or not this is considered as a hack attack or should be logged is specified for the specific communication mode.)

Communication mode	Key			
	Group keys	Zone Key	FDSK or Tool Key	Point-to-point keys
T_Data_Group	M	X	X	X
T_Data_Tag_Group	X	M	X	X
T_Data_Individual	X	X	M	M
T_Data_Connected				
T_Data_Broadcast	X	X	M	M
T_Data_SystemBroadcast	X	X	M	X

5.6 Access Policies at service level

The Application Layer shall be the place where logically the Access Policies at service level are handled. Access Policies are defined in [03].

Table 11 lists the service primitives for which the Access Policies are specified “at service level”, so, commonly independent of the payload they carry. For other service primitives, the Access Policies depend on the data they carry or on the Resource that is accessed. Please refer to [03], [04] and [06] for more requirements on Access Policies in general and on Access Policies at data level specifically.

NOTE 51 For the services for which the acceptance is for all its use defined “at data level”, always 3FF/3FF is given, so that it is clear that the Plain Application Layer passes the message and the evaluation of the access is done by the Application Interface Layer.

Table 11 – Overview of the Access Policies at service level

Service primitive	Access Level	Access Policy
APCI_GroupValue_Read	at data level: configurable	3FF / 3FF
APCI_GroupValue_Response	at data level: configurable	3FF / 3FF
APCI_GroupValue_Write	at data level: configurable	3FF / 3FF
APCI_IndividualAddress_Write	at service level	3FF / 00C
APCI_IndividualAddress_Read	at service level	3FF / 3FF
APCI_IndividualAddress_Response	at service level	3FF / 3FF
APCI_ADC_Read	at service level	3FF / 00C
APCI_ADC_Response	at service level	3FF / 00C
APCI_SystemNetworkParameter_Read	at data level	3FF / 3FF
APCI_SystemNetworkParameter_Response	at data level	3FF / 3FF
APCI_SystemNetworkParameter_Write	at data level	3FF / 3FF
APCI_PropertyExtValue_Read-PDU	at data level	3FF / 3FF
APCI_PropertyExtValue_Response-PDU	at data level	3FF / 3FF
APCI_PropertyExtValue_WriteCon-PDU	at data level	3FF / 3FF
APCI_PropertyExtValue_WriteConRes-PDU	at data level	3FF / 3FF

Service primitive	Access Level	Access Policy
APCI_PropertyExtValue_WriteUnCon-PDU	at data level	3FF / 3FF
APCI_PropertyExtValue_InfoReport-PDU	at data level	3FF / 3FF
APCI_PropertyExtDescription_Read-PDU	at service level	3FF / 3FF
APCI_PropertyExtDescription_Response-PDU	at service level	3FF / 3FF
APCI_FunctionPropertyExtCommand-PDU	at data level	3FF / 3FF
APCI_FunctionPropertyExtState_Read-PDU	at data level	3FF / 3FF
APCI_FunctionPropertyExtState_Response-PDU	at data level	3FF / 3FF
APCI_MemoryExtended_Write	at data level	3FF / 3FF
APCI_MemoryExtended_WriteResponse	at data level	3FF / 3FF
APCI_MemoryExtended_Read	at data level	3FF / 3FF
APCI_MemoryExtended_ReadResponse	at data level	3FF / 3FF
APCI_Memory_Read	at data level	3FF / 3FF
APCI_Memory_Response	at data level	3FF / 3FF
APCI_Memory_Write	at data level	3FF / 3FF
APCI_UserMemory_Read	at data level	3FF / 3FF
APCI_UserMemory_Response	at data level	3FF / 3FF
APCI_UserMemory_Write	at data level	3FF / 3FF
APCI_UserMemoryBit_Write	at data level	3FF / 3FF
APCI_UserManufacturerInfo_Read	at data level	3FF / 3FF
APCI_UserManufacturerInfo_Response	at data level	3FF / 3FF
APCI_FunctionPropertyCommand	at data level	3FF / 3FF
APCI_FunctionPropertyState_Read	at data level	3FF / 3FF
APCI_FunctionPropertyState_Response	at data level	3FF / 3FF
APCI_DeviceDescriptor_Read	at data level	3FF / 3FF
APCI_DeviceDescriptor_Response	at data level	3FF / 3FF
APCI_Restart	at data level: see below	3FF / 3FF
A_Open_Routing_Table_Req-PDU	at service level	3FF / 0CC
A_Write_Router_Status_Req-PDU	at service level	3FF / 0CC
APCI_Read_Router_Status	at service level	3FF / 0CC
APCI_MemoryBit_Write	at data level	3FF / 3FF
APCI_Authorize_Request	at service level	3FF / 3FF
APCI_Authorize_Response	at service level	3FF / 3FF
APCI_Key_Write	at service level	3FF / 0CC
APCI_Key_Response	at service level	3FF / 0CC
APCI_PropertyValue_Read	at data level	3FF / 3FF
APCI_PropertyValue_Response	at data level	3FF / 3FF
APCI_PropertyValue_Write	at data level	3FF / 3FF
APCI_PropertyDescription_Read	at service level	3FF / 3FF
APCI_PropertyDescription_Response	at service level	3FF / 3FF
APCI_NetworkParameter_Read	at data level	3FF / 3FF
APCI_NetworkParameter_Response	at data level	3FF / 3FF
APCI_IndividualAddress_SerialNumber_Read	at service level	3FF / 3FF
APCI_IndividualAddress_SerialNumber_Response	at service level	3FF / 3FF
APCI_IndividualAddress_SerialNumber_Write	at service level	3FF / 00C
APCI_DomainAddress_Write	at service level	3FF / 00C
APCI_DomainAddress_Read	at service level	3FF / 3FF
APCI_DomainAddress_Response	at service level	3FF / 3FF
APCI_DomainAddress_Selective_Read	at service level	3FF / 3FF
APCI_NetworkParameter_Write	at data level	3FF / 3FF
APCI_Link_Read	void 38)	
APCI_Link_Response	void 38)	

³⁸⁾ These services will no longer be used in the Secure generations of E-Mode devices.

Service primitive	Access Level	Access Policy
APCI_Link_Write	void 38)	
APCI_GroupPropValue_Read	at data level: configurable	3FF / 3FF
APCI_GroupPropValue_Response	at data level: configurable	3FF / 3FF
APCI_GroupPropValue_Write	at data level: configurable	3FF / 3FF
APCI_GroupPropValue_InfoReport	at data level: configurable	3FF / 3FF
APCI_DomainAddressSerialNumber_Read	at data level: see below	3FF / 3FF
APCI_DomainAddressSerialNumber_Response	at data level	3FF / 3FF
APCI_DomainAddressSerialNumber_Write	at data level: see below	3FF / 3FF
APCI_FileStream_InfoReport	at data level	3FF / 3FF

Annex A

(informative)

Use of CCM

A.1 Goal

CCM is specified in [08] and [07]. The goal of this annex is not to redefine CCM, but to describe in an informative way how it is used in KNX. The KNX specific use of CCM, the contents of the variable, is given in the normative part of this specification in clause 5 “Secure Application Layer”.

A.2 Definitions

Firstly, the following operations and functions need to be defined.

Operations and functions

Symbol	Description
$X \parallel Y$	The concatenation of the two strings X and Y.
$X \oplus Y$	The bitwise exclusive-OR of the two bit strings X and Y of the same length.
$AES_K(X)$	The encryption of X according to the AES algorithm using the key K.
$LSB_n(X)$	The bit string consisting of the n right-most bits of the bit string X
$MSB_n(X)$	The bit string consisting of the n left-most bits of the bit string X
$PAD_n(X)$	Append 0-bits to the right of X until the length of the resulting bit string is equal to a multiple of n octets.

Fields

CCM protects a payload P and additional data A. The content of these variables is KNX specific and is therefore not given in this informative annex. Please refer to the clauses 5.1.3.4 “Authentication only” and 5.1.3.5 “Authentication and Confidentiality” for the definitions of A and P.

A_{len}	The length of A in bit.
a	The length of A in octets. (size of a = 16 bit)
P_{len}	The length of P in bit.
q	The length of P in octets.

A.3 CCM operation

- The composition of the first Block Counter B_0 is KNX specific. This is specified in clause 5.1.3.2 and in Figure 100.
- Build the blocks B_1 to B_n as follows
The blocks B_1 to B_n shall be built by concatenating a with A and P, padding to a multiple of 16 octets.

pseudo code:
 $B_1 \text{ to } B_n = PAD_{16}(a \parallel A \parallel P)$

NOTE 52 B_1 to B_n are thus always built in this way regardless of whether “authentication only” is used or “authentication and confidentiality” is used; however, the contents of A and P is different: see clause 5.1.3.4 and clause 5.1.3.5 respectively.

- Calculate the first output block Y_0 as AES128 encryption of the Block B_0 with the encryption key k.

pseudo code:
 $Y_0 = AES_K(B_0)$

4. Calculate the further output blocks Y_1 to Y_n as AES128 encryption with the key k of the exclusive-OR function of the block Y_0 to Y_{n-1} respectively

pseudo code:

$$Y_i = \text{AES}_K(Y_{i-1} \oplus B_i) \quad \text{for } i = 1 \dots n$$

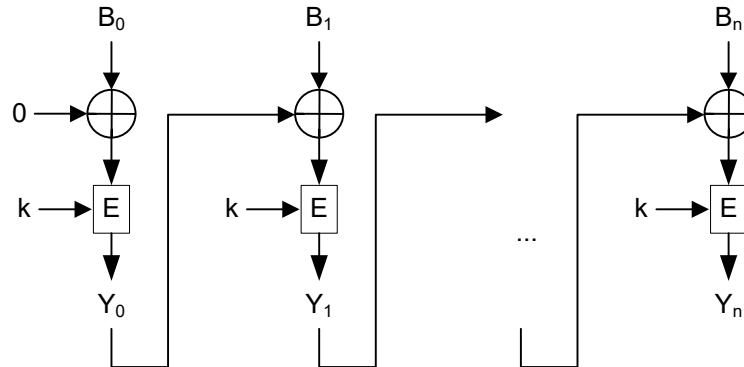


Figure 120 – Block diagram for AES-128 with CBC-MAC

5. The composition of the Block Counter Ctr_0 is KNX specific. This is specified in clause 5.1.3.2 and Figure 102

pseudo code:

$$\text{Ctr}_j = \text{SeqNr} \mid \text{SA} \mid \text{DA} \mid 0000000001h \mid j \quad // \text{ for } j = 0 \text{ to } n; n \leq 255$$

// This is, $\text{CTR}_j = \text{CTR}_{j-1} + 1$: the Ctr is incremented by 1 for each block.

6. Calculate the output blocks S_0 to S_n , which shall be the successively calculated cipher texts of the AES encryption of the Block Counters Ctr_0 to Ctr_n with the key k .

pseudo code:

$$S_j = \text{AES}_K(\text{Ctr}_j) \quad // \text{ for } j = 0 \text{ to } n; n \leq 255$$

7. The stream block S is built by concatenating the output blocks S_0 to S_n where from S_0 only the rightmost 12 octets are included (the leftmost 4 octets are used to calculate the MAC).

pseudo code:

$$S = \text{LSB}_{96}(S_0) \mid S_1 \mid S_2 \mid \dots \mid S_n$$

8. The encryption is done by bitwise XOR-ing the stream block S with the bits of the payload P , starting from the most significant bits and concatenating the result with the MAC (this is $\text{MSB}_{32}(Y_n) \oplus S_0$)

pseudo code:

$$C = (P \oplus \text{MSB}_{\text{plen}}(S)) \mid (\text{MSB}_{32}(Y_n) \oplus \text{MSB}_{32}(S_0))$$

NOTE 53 If “authentication only” is used, P is empty and only $(\text{MSB}_{32}(Y_n) \oplus \text{MSB}_{32}(S_0))$ remains.

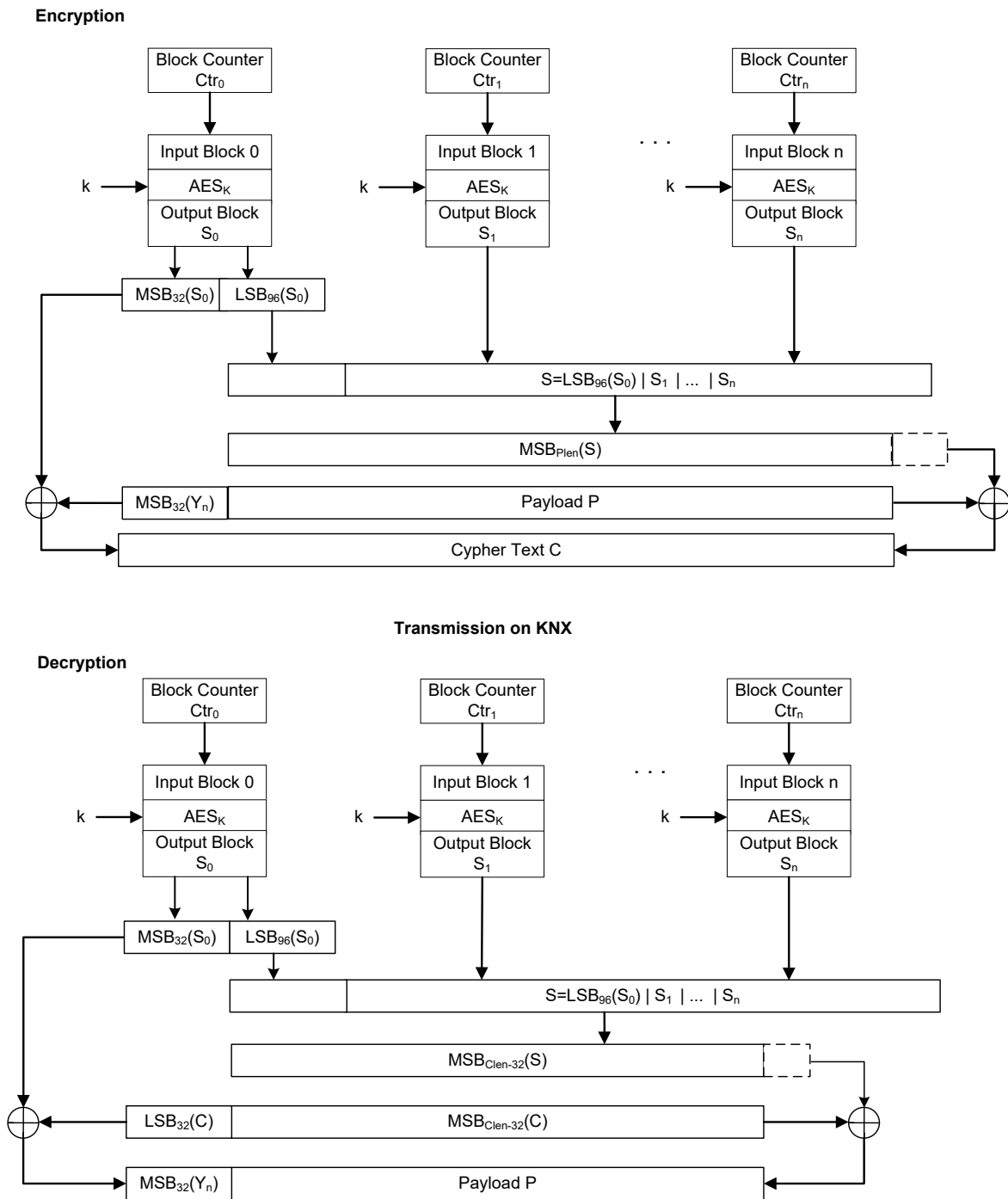


Figure 121 – Block diagram for AES-CTR Mode

Message verification

1. Decrypt the cipher text to obtain the Secure Data.

pseudo code:

$\text{Ctr}_j = \text{SeqNr} \mid \text{SA} \mid \text{DA} \mid 0000000001h \mid j$ // for $j = 0$ to n ; $n \leq 255$
 // This is, $\text{Ctr}_j = \text{Ctr}_{j-1} + 1$: the CTR is incremented by 1 for each block.

$S_j = \text{AES}_K(\text{Ctr}_j)$ // for $j = 0$ to n ; $n \leq 255$

$S = \text{LSB}_{96}(S_0) \mid S_1 \mid S_2 \mid \dots \mid S_n$

The received payload is then reconstructed as follows:
 C_{len} is the length of the received cipher text.

pseudo code:

$$P = MSB_{C_{len}-32}(C) \oplus MSB_{C_{len}-32}(S)$$

2. The received MAC (T_R) shall be decrypted as follows.

pseudo code:

$$T_R = LSB_{32}(C) \oplus MSB_{32}(S_0)$$

3. The composition of the first block B_0 is KNX specific. This is specified in Figure 100.

pseudo code:

$$B_0 = SeqNr | SA | DA | 00h | AT | TPCI | 3F1h | 00h | q$$

q = The length of the payload. q shall be one octet long.

4. Build the blocks B_1 to B_n as follows

The blocks B_1 to B_n shall be built by concatenating a with A and then with P, padding to a multiple of 16 octets.

pseudo code:

$$B_1 \text{ to } B_n = PAD16(a | A | P)$$

5. Calculate the first output block Y_0 as AES128 encryption of the Block B_0 with the encryption key k .

pseudo code:

$$Y_0 = AES_K(B_0)$$

6. Calculate the further output blocks Y_1 to Y_n as AES128 encryption with the key k of the exclusive-OR function of the block Y_0 to Y_{n-1} respectively

pseudo code:

$$Y_i = AES_K(Y_{i-1} \oplus B_i) \text{ for } i = 1 \dots n$$

7. The receiver shall then compare $MSB_{32}(Y_n)$ with the decrypted T_R and accept the message solely of both equal.

Annex B

(informative)

Background and motivation

B.1 Runtime - symmetric security algorithm

B.1.1 Algorithm

KNX security algorithm for runtime

Apart from the initial configuration, for runtime communication a symmetric key encryption algorithm shall be used. This allows sender and receiver to have the same key, for encryption as well as for decryption.

For encryption AES 128 shall be used. This requires 10^{28} MIPS years to attack. Hence it gives a protection lifetime of minimal 20 years.

Timings indications

Free sources in C for AES 128 have been implemented on a MSP430F2370 at 4MHz.

(<http://www.progressive-coding.com/tutorial.php?id=0>)

- 16 bytes encryption → 38 ms
- 16 bytes decryption → 38 ms

Other implementations exist.

A commercialized solution mentions for AES-128 for MSP430 (asm + C interfaces) mentions:

- Encryption : 5342 cycles (thus 1,4 ms at 4 MHz)
- Decryption : 8802 cycles (thus 2,2 ms at 4 MHz)

http://jce.iaik.tugraz.at/sic/products/crypto_software_for_microcontrollers/texas_instruments_msp430_microcontrollers

B.1.2 Operation Mode

Introduction

Any security algorithm may have several operation modes.

EXAMPLE 7 AES has 5 operation modes: ECB, CBC, CFB, OFB, CTR

B.2 Sequence Number field – calculation of size

This calculation is based on the communication possibilities of the KNX RF Physical Layer.

KNX RF uses a signalling speed of 32 768 cps. This allows to encode 16 384 bits per second ³⁹⁾.

A typical KNX Frame size counts 23 octets. These are 184 bits.

From this, the maximal number of KNX RF messages that can be sent per second can be calculated.

$$\frac{16\,384\text{ bits/second}}{184\text{ bits/message}} = 89\text{ messages/second}$$

According to the size of the sequence number, a wrap around occurs approximately after the duration below

³⁹⁾ This is the current data rate (April 2011). The KNX RF Multi is not used.

size random value (bits)	128	64	48	32	16
counter values possibilities	3,40282E+38	1,84467E+19	2,81475E+14	4294967296	65536
max Knx messages sent / sec	89	89	89	89	89
years number	1,21239E+29	6572386712	100286,6625	1,530253028	2,335E-05

Figure 122 - Calculation of the suitable size of the Sequence Number field

The above shows that a wraparound of counter happens after 1,5 year for a size of 32 bits. Therefore, for KNX Secure Communication, the size of the Sequence Number shall be 48 bit.

B.3 Timing calculation of KNX secure communication

Figure 123 shows the resulting timing of a secure communication. The calculation is done for the KNX RF communication medium.

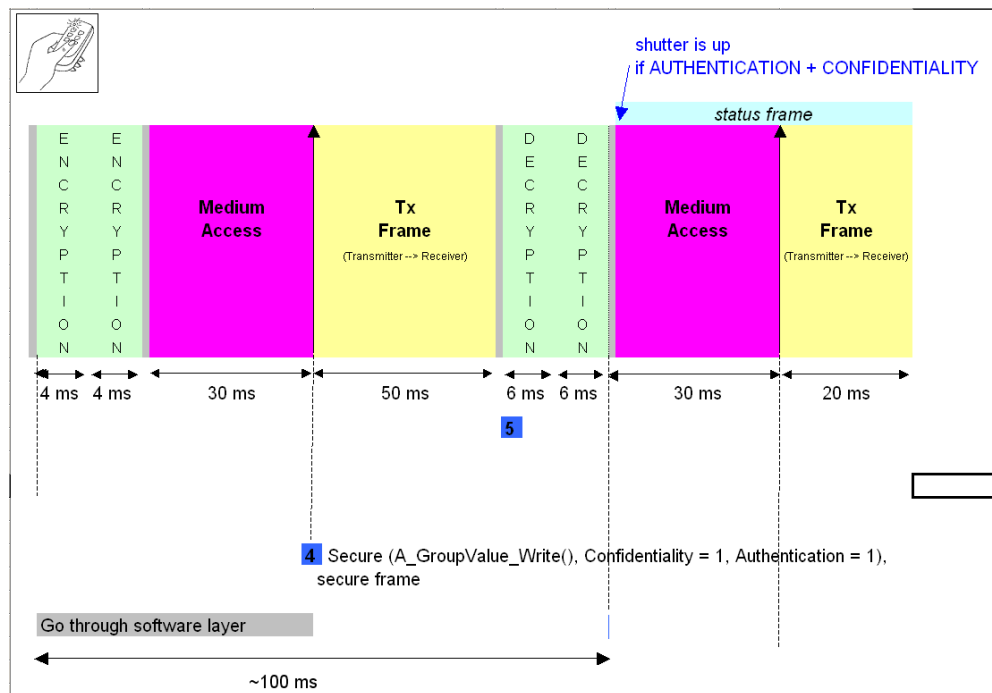


Figure 123 – Timing of secure communication in KNX RF

Annex C

(informative)

Examples

C.1 Full encoding of a KNX Secure APDU

A full example of a properly encrypted KNX Frame is provided here.

C.1.1 S-A_Data-PDU

Property Value Write PID_GRP_KEY_TABLE connectionless (Objet Idx = 5, PropId = 35h, Element Count = 1, Index = 1, Data = 20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F, A + C)

Input

SA = FF67h (15.15.103)

DA = FF00h (15.15.0)

TPCI/APCI = 03F1h (000000b | 1111110001b)
(T_Data_Individual-PDU, A_Secure-APDU)

SCF = 90h

(SAI = 1, Tool Access = 1, S-AL type = CCM S-A_data with A and C)

SeqNr = 000000000004h (Received in the preceding A_Sync_Res Frame)

Frame unencrypted: 3C 60 FF 67 FF 00 1E 03 F1 90 00 00 00 00 00 04 03
D7 05 35 10 01 20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F

(Attention, this is an Secure Frame without MAC and confidentiality.
This is only informative for better understanding, such frame will
never be send)

Security Parameters

Tool Key: 00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f

Calculations

A = SCF
= 90

P = 000000b | Plain APDU
= 03 D7 05 35 10 01 20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F

a = 00 01

q = 22d (=16h)

B0 = SeqNr | SA | DA | 00h | AT | TPCI/APCI | 00h | q
= 00 00 00 00 00 04 FF 67 FF 00 00 00 03 F1 00 16

B1 = PAD16(a | A | P)
= 00 01 90 03 D7 05 35 10 01 20 21 22 23 24 25 26

B2 = PAD16(P)
= 27 28 29 2A 2B 2C 2D 2E 2F 00 00 00 00 00 00 00

Y0 = AESk(B0)
= bd2161cb9dd615a443a627952afb1295

Y0 XOR B1 = bd20f1c84ad320b4428606b709df37b3

Y1 = AESk(Y0 XOR B1)
= f8a01c6930004f55b8fde3ac67f3d712

```

Y1 XOR B2 = df8835431b2c627b97fde3ac67f3d712
Y2 = AEsK(Y1 XOR B2)
    = 056a57e9950c98cae2991688ededa5d7
Ctr0 = SeqNr | SA | DA | 00 00 00 00 01 j
      = 00 00 00 00 00 04 FF 67 FF 00 00 00 00 00 01 00
Ctr1 = SeqNr | SA | DA | 00 00 00 00 01 j
      = 00 00 00 00 00 04 FF 67 FF 00 00 00 00 00 01 01
S0 = AEsK(Ctr0)
    = 08e5968164b0211f3309ea5783345004
S1 = AEsK(Ctr1)
    = 68c3e774bebbb359132a169a1ddb8d67
MAC = MSB32(Y2) XOR MSB32(S0)
     = d8fc168
S = LSB96(S0)
  = 64b0211f3309ea5783345004 68c3e774bebbb359132a
C = (P XOR MSB32(S)) | MAC
  = 6767242a2308ca76a1177421 4ee4cf5d94909f743d05 | 0d8fc168

```

Final encrypted S-A_Data-PDU

SCF	SeqNr	C
90	00 00 00 00 00 04	6767242a2308ca76a11774214ee4cf5d94909f743d050d8fc168

C.1.2 S-A_Data-PDU

Property Value Write Response PID_GRP_KEY_TABLE connectionless (Objet Idx = 5, PropId = 35h, Element Count = 1, Index = 1, Data = 20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F, A + C

Input

```

SA = FF00h (15.15.0)
DA = FF67h (15.15.103)
TPCI/APCI = 03F1h (000000b | 1111110001b)
(T_Data_Individual-PDU, A_Secure-APDU)
SCF = 90h
(SAI = 1, Tool Access = 1, S-AL type = CCM S-A_data with A and C)
SeqNr = 000000000003h (SeqNrSending from PID_SEQUENCE_NUMBER_SENDING)
Frame unencrypted: 3C 60 FF 00 FF 67 22 03 F1 90 00 00 00 00 00 03 03
D6 05 35 10 01 20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F

```

Security Parameters

Tool Key: 00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f

Calculations

```

A = SCF
  = 90
P = 000000b | Plain APDU
  = 03 D6 05 35 10 01 20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F

```

```

a = 00 01
q = 22d (=16h)
B0 = SeqNr | SA | DA | 00h | AT | TPCI/APCI | 00h | q
    = 00 00 00 00 00 03 FF 00 FF 67 00 00 03 F1 00 16
B1 = PAD16(a | A | P)
    = 00 01 90 03 D6 05 35 10 01 20 21 22 23 24 25 26
B2 = PAD16(P)
    = 27 28 29 2A 2B 2C 2D 2E 2F 00 00 00 00 00 00 00
Y0 = AESk(B0)
    = 6d04b5be0afcabf78dce709adfc719b9
Y0 XOR B1 = 6d0596bddcf99ee78cee51b8fce33c9f
Y1 = AESk(Y0 XOR B1)
    = 405375e828b4b3531b51177a373b7bee
Y1 XOR B2 = d151b2c07c8aa52909401d3b4b468d4a
Y2 = AESk(Y1 XOR B2)
    = 6433f89ee46dfb90bd39082c89c9ea90
Ctr0 = SeqNr | SA | DA | 00 00 00 00 01 j
      = 00 00 00 00 00 03 FF 00 FF 67 00 00 00 00 01 00
Ctr1 = SeqNr | SA | DA | 00 00 00 00 01 j
      = 00 00 00 00 00 03 FF 00 FF 67 00 00 00 00 01 01
S0 = AESk(Ctr0)
    = 0c7a5fb573b9560415511576e90800d4
S1 = AESk(Ctr1)
    = fb1333499dcb3bc1f89fbd4ae3217198
MAC = MSB32(Y2) XOR MSB32(S0)
     = 6849a72b
S = LSB96(S0)
  = 3b9560415511576e90800d4 fb1333499dcb3bc1f89f
C = (P XOR MSB32(S)) | MAC
  = 706f533105503557cb2b24f1 dd341b60b7e017ecd6b0 | 6849a72b

```

Final encrypted S-A_Data-PDU

SCF	SeqNr	C
23	00 00 00 00 00 03	706f533105503557cb2b24f1dd341b60b7e017ecd6b06849a72b

C.1.3 S-A_Sync.req

Input

```

SA = FF67h (15.15.103)
DA = FF00h (15.15.0)
TPCI/APCI = 43F1h (010000b | 1111110001b)
SCF = 92h

```

```
SeqNrlocal = 0000000000001h
SerialNumber = 0000000000000h
Challenge = 0000000000003h
```

Security Parameters

Tool Key: 00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f

Calculations

```
A = SCF | SerialNumber
  = 92 00 00 00 00 00 00

P = Challenge
  = 00 00 00 00 00 03

a = 00 07
q = 06

B0 = SeqNrlocal | SA | DA | 00h | AT | TPCI/APCI | 00h | q
    = 00 00 00 00 00 01 FF 67 FF 00 00 00 43 F1 00 06

B1 = PAD16(a | A | P)
    = 00 07 92 00 00 00 00 00 00 00 00 00 00 00 03 00

Y0 = AESk(B0)
    = 67 27 e9 e3 08 1e df 25 4d dc 99 f7 31 18 a1 3b

Y1 = AESk(Y0 XOR B1)
    = e7 98 07 b3 1b 35 86 e9 8e 56 1f ba 9e fa 8a 65

Ctr0 = SeqNrlocal | SA | DA | 00 00 00 00 01 00
      = 00 00 00 00 00 01 ff 67 ff 00 00 00 00 00 01 00

S0 = AESk(Ctr0)
    = 30 07 ac e6 c1 cf 45 06 f0 98 c9 a0 fc 83 62 f3

MAC = MSB32(Y1) XOR MSB32(S0)
     = d7 9f ab 55

S = LSB96(S0)
  = c1 cf 45 06 f0 98 c9 a0 fc 83 62 f3

C = (P XOR MSB48(S)) | MAC
  = c1 cf 45 06 f0 9b d7 9f ab 55
```

Final encrypted S_A_Sync_Req-PDU

SCF	SeqNr _{local}	SerialNumber	C
92	00 00 00 00 00 01	00 00 00 00 00 00	c1 cf 45 06 f0 9b d7 9f ab 55

C.1.4 S-A_Sync.res

Input

```
SA = FF00h (15.15.0)
DA = FF67h (15.15.103)
TPCI/APCI = 43F1h (010000b | 1111110001b)
SCF = 93h
```

SeqNr_{remote} = 0000000000003h (SeqNrSending from
PID_SEQUENCE_NUMBER_SENDING)
SeqNr_{local} = 0000000000004h (next valid SeqNr accepted for Tool Key)
Challenge = 0000000000003h
Random = AA AA AA AA AA AA

Security Parameters

Tool Key: 00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f

Calculations

A = SCF
= 93

P = SeqNr_{remote} | SeqNr_{local}
= 00 00 00 00 00 03 00 00 00 00 00 04

a = 00 01

q = 0C

B0 = Random | SA | DA | 00h | AT | TPCI/APCI | 00h | q
= aa aa aa aa aa aa ff 00 ff 67 00 00 43 f1 00 0c

B1 = PAD16(a | A | P)
= 00 01 93 00 00 00 00 00 03 00 00 00 00 00 04 00

Y0 = AESk(B0)
= e6 6e fb c5 b0 61 71 64 37 1e a0 7a 63 42 4c 7e

Y1 = AESk(Y0 XOR B1)
= 48 4a d4 b5 4a 17 6a cd 80 2d 6f ea a4 a6 40 cd

Ctr0 = Random | SA | DA | 00 00 00 00 01 00
= aa aa aa aa aa aa ff 00 ff 67 00 00 00 00 01 00

S0 = AESk(Ctr0)
= 13 3a 1e 71

MAC = MSB32(Y1) XOR MSB32(S0)
= 5b 70 ca c4

S = LSB96(S0)
= 13 3a 1e 71 9c 02 3a d2 5e 17 64 70 69 3e 63 89

C = (P XOR MSB32(S)) | MAC
= 9c 02 3a d2 5e 14 64 70 69 3e 63 8d 5b 70 ca c4

Final encrypted S_A_Sync_Res-PDU

SCF	Challenge XOR Random	C
93	aa aa aa aa aa a9	9c 02 3a d2 5e 14 64 70 69 3e 63 8d 5b 70 ca c4